

▼	reviews_book		CREATE TABLE "reviews_book" ("id" Integer NOT NULL PRIMARY KEY,
	id	Integer	"id" Integer NOT NULL
	title	varchar(70)	"title" varchar(70) NOT NULL
	publication_date	date	"publication_date" date NOT NULL
	isbn	varchar(20)	"isbn" varchar(20)
	publisher_id	bigint	"publisher_id" bigint NOT NULL
▼	reviews_bookcontributor		CREATE TABLE "reviews_bookcontributor" ("id" Integer NOT NULL PRI
	id	Integer	"id" Integer NOT NULL
	role	varchar(20)	"role" varchar(20) NOT NULL
	book_id	bigint	"book_id" bigint NOT NULL
	contributor_id	bigint	"contributor_id" bigint NOT NULL
▼	reviews_contributor		CREATE TABLE "reviews_contributor" ("id" Integer NOT NULL PRIMARY
	id	Integer	"id" Integer NOT NULL
	first_names	varchar(50)	"first_names" varchar(50) NOT NULL
	last_names	varchar(50)	"last_names" varchar(50)
	email	varchar(254)	"email" varchar(254)
▼	reviews_publisher		CREATE TABLE "reviews_publisher" ("id" Integer NOT NULL PRIMARY
	id	Integer	"id" Integer NOT NULL
	name	varchar(50)	"name" varchar(50) NOT NULL
	website	varchar(200)	"website" varchar(200)
	email	varchar(254)	"email" varchar(254)
▼	reviews_review		CREATE TABLE "reviews_review" ("id" Integer NOT NULL PRIMARY KEY
	id	Integer	"id" Integer NOT NULL
	content	text	"content" text NOT NULL
	rating	Integer	"rating" Integer NOT NULL
	date_created	datetime	"date_created" datetime NOT NULL
	date_edited	datetime	"date_edited" datetime
	book_id	bigint	"book_id" bigint NOT NULL
	creator_id	Integer	"creator_id" Integer NOT NULL

Figure 2.8: Database tables as defined in the reviews app

In this section, we learned more about Django models and migrations and how Python's simple classes can transform themselves into database tables. We also learned about how various class attributes translate into appropriate database columns following the defined field types. Later, we learned about primary keys and different types of relationships that can exist in a database. We also created models for the book review application and migrated those models, translating them into database tables. In the next section, we will learn about how to perform database CRUD operations using Django's ORM.

Django's database CRUD operations

As we have created the necessary database tables for the book review application, let's work on understanding the basic database operations with Django.

▼	reviews_book	CREATE TABLE "reviews_book" ("id" Integer NOT NULL PRIMARY KEY,
	id	Integer "id" Integer NOT NULL
	title	varchar(70) "title" varchar(70) NOT NULL
	publication_date	date "publication_date" date NOT NULL
	isbn	varchar(20) "isbn" varchar(20)
	publisher_id	bigint "publisher_id" bigint NOT NULL
▼	reviews_bookcontributor	CREATE TABLE "reviews_bookcontributor" ("id" Integer NOT NULL PRI
	id	Integer "id" Integer NOT NULL
	role	varchar(20) "role" varchar(20) NOT NULL
	book_id	bigint "book_id" bigint NOT NULL
	contributor_id	bigint "contributor_id" bigint NOT NULL
▼	reviews_contributor	CREATE TABLE "reviews_contributor" ("id" Integer NOT NULL PRIMARY
	id	Integer "id" Integer NOT NULL
	first_names	varchar(50) "first_names" varchar(50) NOT NULL
	last_names	varchar(50) "last_names" varchar(50)
	email	varchar(254) "email" varchar(254)
▼	reviews_publisher	CREATE TABLE "reviews_publisher" ("id" Integer NOT NULL PRIMARY
	id	Integer "id" Integer NOT NULL
	name	varchar(50) "name" varchar(50) NOT NULL
	website	varchar(200) "website" varchar(200)
	email	varchar(254) "email" varchar(254)
▼	reviews_review	CREATE TABLE "reviews_review" ("id" Integer NOT NULL PRIMARY KEY
	id	Integer "id" Integer NOT NULL
	content	text "content" text NOT NULL
	rating	Integer "rating" Integer NOT NULL
	date_created	datetime "date_created" datetime NOT NULL
	date_edited	datetime "date_edited" datetime
	book_id	bigint "book_id" bigint NOT NULL
	creator_id	Integer "creator_id" Integer NOT NULL

그림 2.8: 리뷰 앱에서 정의된 데이터베이스 테이블

이 섹션에서는 Django 모델과 마이그레이션에 대해 더 배우고 Python의 간단한 클래스가 어떻게 데이터베이스 테이블로 변환될 수 있는지에 대해 배웠습니다. 또한 다양한 클래스 속성이 정의된 필드 유형에 따라 적절한 데이터베이스 열로 변환되는 방법에 대해서도 배웠습니다. 이후에는 기본 키와 데이터베이스에 존재할 수 있는 다양한 유형의 관계에 대해 배웠습니다. 우리는 또한 도서 리뷰 애플리케이션을 위한 모델을 만들고 이러한 모델을 마이그레이션하여 데이터베이스 테이블로 변환했습니다. 다음 섹션에서는 Django의 ORM을 사용하여 데이터베이스 CRUD 작업을 수행하는 방법에 대해 배울 것입니다.

Django의 데이터베이스 CRUD 작업

우리가 도서 리뷰 애플리케이션을 위한 필요한 데이터베이스 테이블을 생성했으므로, Django를 사용하여 기본 데이터베이스 작업을 이해하는 데 집중해 봅시다.

CRUD is an acronym for **Create, Read, Update, Delete** – the four primary operations of a database system. In SQL, these operations are implemented with the `INSERT`, `SELECT`, `UPDATE`, and `DELETE` statements.

Django's ORM provides the same functionality without having to deal with SQL statements. Django's database operations are simple Python code, hence we overcome the hassle of maintaining SQL statements within the Python code. Let's take a look at how these are performed.

To execute the CRUD operations, we will enter Django's command-line shell by executing the following command:

```
python manage.py shell
```

When the interactive console starts, it looks as follows:

```
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>>
```

In the following exercise, we shall perform a create operation.

Exercise 2.01 – Creating an entry in the bookr database

In this exercise, we will create a new entry in the database by saving a model instance. In other words, we will create an entry in a database table without explicitly running a SQL query. Follow these steps to do this:

1. First, import the `Publisher` class/model from `reviews.models`:

```
>>> from reviews.models import Publisher
```

2. Create an object or an instance of the `Publisher` class by passing all the field values (name, website, and email) required by the `Publisher` model:

```
>>> publisher = Publisher(name='Packt Publishing',
                          website='https://www.packtpub.com',
                          email='info@packtpub.com')
```

CRUD는 데이터베이스 시스템의 네 가지 주요 작업인 생성, 읽기, 업데이트, 삭제를 나타내는 약어입니다. SQL에서는 이러한 작업이 INSERT, SELECT, UPDATE 및 DELETE 문으로 구현됩니다.

Django의 ORM은 SQL 문을 다루지 않고도 동일한 기능을 제공합니다. Django의 데이터베이스 작업은 간단한 Python 코드로 이루어져 있어 Python 코드 내에서 SQL 문을 유지하는 번거로움을 덜 수 있습니다. 이러한 작업이 어떻게 수행되는지 살펴보겠습니다.

CRUD 작업을 실행하기 위해 다음 명령을 실행하여 Django의 명령줄 셸에 들어갑니다:

```
python manage.py shell
```

대화형 콘솔이 시작되면 다음과 같이 보입니다:

```
자세한 정보는 "help", "copyright", "credits" 또는 "license"를 입력하세요. (InteractiveConsole)
```

```
>>>
```

다음 연습에서는 생성 작업을 수행할 것입니다.

연습 2.01 – bookr 데이터베이스에 항목 만들기

이 연습에서는 모델 인스턴스를 저장하여 데이터베이스에 새 항목을 생성합니다. 즉, SQL 쿼리를 명시적으로 실행하지 않고 데이터베이스 테이블에 idx_a175ff71entry를 생성합니다. 이를 수행하려면 다음 단계를 따르십시오:

1. 먼저, reviews.models에서 Publisher 클래스/모델을 가져옵니다:

```
>>> from reviews.models import Publisher
```

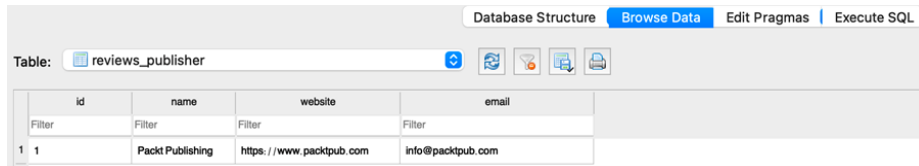
2. Publisher 모델에서 요구하는 모든 필드 값(이름, 웹사이트 및 이메일)을 전달하여 Publisher 클래스의 객체 또는 인스턴스를 생성합니다:

```
>>> publisher = Publisher(name='Packt Publishing',
                           website='https://www.packtpub.com',
                           email='info@packtpub.com')
```


3. Next, to write the object into the database, it is important to call the `save()` method, because until this is called, there will not be an entry created in the database:

```
>>> publisher.save()
```

Now we can see a new entry created in the database using DB Browser:



The screenshot shows the DB Browser interface with the 'reviews_publisher' table selected. The table has four columns: id, name, website, and email. A single row is displayed with the following values: id=1, name=Packt Publishing, website=https://www.packtpub.com, and email=info@packtpub.com.

	id	name	website	email
1	1	Packt Publishing	https://www.packtpub.com	info@packtpub.com

Figure 2.9: An entry created in the database

4. Use the object attributes to make any further changes to the object and save the changes to the database:

```
>>> publisher.email = 'info@packtpub.com'
>>> publisher.email = 'customersupport@packtpub.com'
>>> publisher.save()
```

We can see the changes using DB Browser as follows:



The screenshot shows the DB Browser interface with the 'reviews_publisher' table selected. The table has four columns: id, name, website, and email. A single row is displayed with the following values: id=1, name=Packt Publishing, website=https://www.packtpub.com, and email=customersupport@packtpub.com.

	id	name	website	email
1	1	Packt Publishing	https://www.packtpub.com	customersupport@packtpub.com

Figure 2.10: An entry with the updated email field

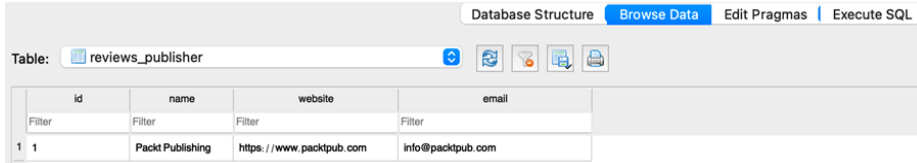
In this exercise, we created an entry in the database by creating an instance of the model object and used the `save()` method to write the model object into the database.

Note that by following the preceding method, the changes to the class instance are not saved until the `save()` method is called. However, if we use the `create()` method, Django saves the changes to the database in a single step. We'll use this method in the exercise that follows.

3. 다음으로, 객체를 데이터베이스에 쓰기 위해서는 `save()` 메서드를 호출하는 것이 중요합니다. 이 메서드가 호출되기 전까지는 데이터베이스에 항목이 생성되지 않습니다:

```
>>> publisher.save()
```

이제 DB Browser를 사용하여 데이터베이스에 생성된 새로운 항목을 볼 수 있습니다:



The screenshot shows the DB Browser interface with the 'reviews_publisher' table selected. The table has four columns: id, name, website, and email. A single record is displayed with id 1, name 'Packt Publishing', website 'https://www.packtpub.com', and email 'info@packtpub.com'.

	id	name	website	email
1	1	Packt Publishing	https://www.packtpub.com	info@packtpub.com

그림 2.9: 데이터베이스에 생성된 항목

4. 객체 `idx_635d4bb6attributes`를 사용하여 `idx_8af52fc8` 객체에 대한 추가 변경을 하고 변경 사항을 데이터베이스에 저장합니다:

```
>>> publisher.email = 'info@packtpub.com'
>>> publisher.email = 'customersupport@packtpub.com'
>>> publisher.save()
```

우리는 DB Browser를 사용하여 다음과 같이 변경 사항을 볼 수 있습니다:



The screenshot shows the DB Browser interface with the 'reviews_publisher' table selected. The table has four columns: id, name, website, and email. A single record is displayed with id 1, name 'Packt Publishing', website 'https://www.packtpub.com', and email 'customersupport@packtpub.com'.

	id	name	website	email
1	1	Packt Publishing	https://www.packtpub.com	customersupport@packtpub.com

그림 2.10: 업데이트된 이메일 필드가 있는 항목

이 연습에서는 모델 객체의 인스턴스를 생성하여 데이터베이스에 항목을 만들고 `save()` 메서드를 사용하여 모델 객체를 데이터베이스에 기록했습니다.

앞서 설명한 방법을 따르면 `save()` 메서드가 호출될 때까지 클래스 인스턴스에 대한 변경 사항이 저장되지 않습니다. 그러나 `create()` 메서드를 사용하면 Django가 단일 단계에서 데이터베이스에 변경 사항을 저장합니다. 우리는 다음 연습에서 이 메서드를 사용할 것입니다.

Exercise 2.02 – Using the create() method to create an entry

Unlike the previous exercise, where we executed two separate steps by first creating an object and later using the save() method to create an entry in the database, in this exercise, we will create a record in the contributor table using the create() method in a single step:

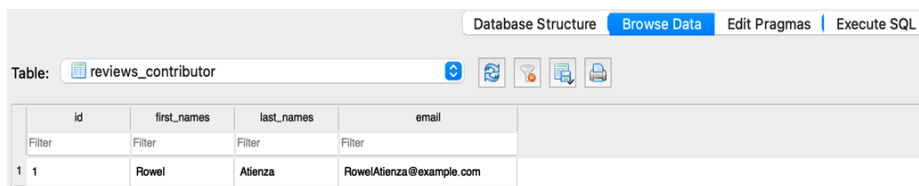
1. First, import the Contributor class as before:

```
>>> from reviews.models import Contributor
```

2. Invoke the create() method to create an object in the database in a single step. Ensure that you pass all the required parameters (first_names, last_names, and email):

```
>>> contributor = Contributor.objects.create(
    first_names="Rowel", last_names="Atienza",
    email="RowelAtienza@example.com")
```

3. Use DB Browser to verify that the contributor record has been created in the database. If your DB Browser is not already open, open the db.sqlite3 database file as we just did in the previous section. Click **Browse Data** and select the desired table – in this case, the reviews_contributor table from the **Table** drop-down menu, as shown in the screenshot – and verify the newly created database record:



The screenshot shows the DB Browser interface with the 'Browse Data' tab selected. The 'Table' dropdown menu is set to 'reviews_contributor'. Below the table name, there are icons for refreshing, filtering, and other actions. The table itself has four columns: 'id', 'first_names', 'last_names', and 'email'. There is one row of data with the following values: '1' for id, 'Rowel' for first_names, 'Atienza' for last_names, and 'RowelAtienza@example.com' for email.

id	first_names	last_names	email
1	Rowel	Atienza	RowelAtienza@example.com

Figure 2.11: Verifying the creation of the record in DB Browser

In this exercise, we learned that using the create() method, we can create a record for a model in a database in a single step.

Creating an object with a foreign key

Similar to how we created a record in the Publisher and Contributor tables, let's now create one for the Book table. If you recall, the Book model has a foreign key to Publisher that cannot have a null value. So, a way to populate the publisher's foreign key is by providing the created publisher object in the book's publisher field, as shown in the following exercise.

연습 2.02 – create() 메서드를 사용하여 항목 만들기

이전 연습과 달리, 객체를 먼저 생성한 다음 save() 메서드를 사용하여 데이터베이스에 항목을 만드는 두 개의 별도 단계를 실행했던 것과는 달리, 이 연습에서는 create() 메서드를 사용하여 기여자 테이블에 레코드를 단일 단계로 생성할 것입니다:

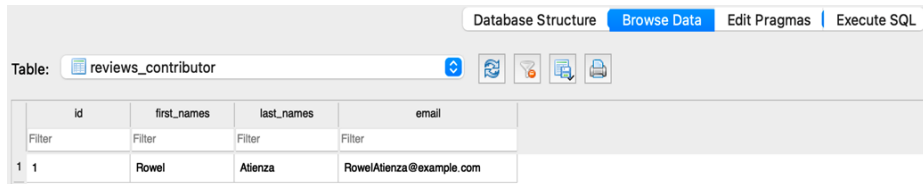
1. 먼저, 이전과 같이 Contributor 클래스를 가져옵니다:

```
>>> from reviews.models import Contributor
```

2. 단일 단계로 데이터베이스에 객체를 생성하기 위해 create() 메서드를 호출합니다. 모든 필수 매개변수(이름, 성, 이메일)를 전달해야 합니다:

```
>>> contributor = Contributor.objects.create(
    first_names="Rowel", last_names="Atienza",
    email="RowelAtienza@example.com")
```

3. DB 브라우저를 사용하여 기여자 레코드가 데이터베이스에 생성되었는지 확인하십시오. DB 브라우저가 이미 열려 있지 않다면, 이전 섹션에서 했던 것처럼 db.sqlite3 데이터베이스 파일을 엽니다. 데이터 탐색을 클릭하고 원하는 테이블을 선택합니다. 이 경우, 스크린샷에 표시된 대로 테이블 드롭다운 메뉴에서 reviews_contributor 테이블을 선택하고 새로 생성된 데이터베이스 레코드를 확인합니다:



The screenshot shows a database browser interface with tabs for 'Database Structure', 'Browse Data', 'Edit Pragma', and 'Execute SQL'. The 'Browse Data' tab is active, and the table 'reviews_contributor' is selected. Below the table name, there are icons for refresh, search, and other actions. The table structure is shown with columns: id, first_names, last_names, and email. A single record is displayed with the following values:

id	first_names	last_names	email
1	Rowel	Atienza	RowelAtienza@example.com

그림 2.11: DB 브라우저에서 레코드 생성 확인

이번 연습에서는 create() 메서드를 사용하여 데이터베이스에서 모델의 레코드를 한 번에 생성할 수 있다는 것을 배웠습니다.

외래 키로 객체 생성하기

출판사 및 기여자 테이블에 레코드를 생성한 방법과 유사하게, 이제 책 테이블에 대한 레코드를 생성해 보겠습니다. 기억하시겠지만, 책 모델은 null 값을 가질 수 없는 출판사에 대한 외래 키를 가지고 있습니다. 따라서 출판사의 외래 키를 채우는 방법은 다음 연습에서 보여준 것처럼 책의 출판사 필드에 생성된 출판사 객체를 제공하는 것입니다.

Exercise 2.03 – Creating records for a many-to-one relationship

In this exercise, we will create a record in the Book table, including a foreign key to the Publisher model. As you already know, the relationship between Book and Publisher is a many-to-one relationship, so you have to first fetch the Publisher object and then use it while creating the book record. To do that, follow these steps:

1. First, import the Publisher class:

```
>>> from reviews.models import Book, Publisher
```

2. Retrieve the publisher object from the database using the following command. The `get()` method is used to retrieve an object from the database. We still haven't explored database read operations. For now, use the following command; we will go deeper into database read/retrieve in the next section:

```
>>> publisher = Publisher.objects.get(
    name='Packt Publishing')
```

3. When creating a book, we need to supply a date object, as `publication_date` is a date field in the Book model. So, import date from datetime so that a date object can be supplied when creating the book object, as shown in the following code:

```
>>> from datetime import date
```

4. Use the `create()` method to create a record of the book in the database. Ensure that you pass all the fields, namely `title`, `publication_date`, `isbn`, and the `publisher` object:

```
>>> book = Book.objects.create(
    title="Advanced Deep Learning with Keras",
    publication_date=date(2018, 10, 31),
    isbn="9781788629416",
    publisher=publisher)
```

Note that since `publisher` is a foreign key and it is not nullable (cannot hold a null value), it is mandatory to pass a publisher object. When the mandatory foreign key object `publisher` is not provided, the database will throw an integrity error.

연습 2.03 – 다대일 관계를 위한 레코드 생성

이 연습에서는 Book 테이블에 레코드를 생성하고, Publisher 모델에 대한 외래 키를 포함합니다. 이미 아시다시피, Book과 Publisher 간의 idx_e72c0f 관계는 다대일 관계이므로, 먼저 Publisher 객체를 가져온 다음 책 레코드를 생성할 때 사용해야 합니다. 이를 위해 다음 단계를 따르세요:

1. 먼저, Publisher 클래스를 가져옵니다:

```
>>> from reviews.models import Book, Publisher
```

2. 다음 명령어를 사용하여 데이터베이스에서 퍼블리셔 객체를 검색합니다. get() 메서드는 데이터베이스에서 객체를 검색하는 데 사용됩니다. 우리는 아직 데이터베이스 읽기 작업을 탐색하지 않았습니니다. 지금은 다음 명령어를 사용하세요; 다음 섹션에서 데이터베이스 읽기/검색에 대해 더 깊이 다룰 것입니다:

```
>>> publisher = Publisher.objects.get(
    name='팩트 퍼블리싱')
```

3. 책을 생성할 때, Book 모델에서 publication_date가 datefield이므로 날짜 객체를 제공해야 합니다. 따라서 책 객체를 생성할 때 날짜 객체를 제공할 수 있도록 datetime에서 date를 가져와야 하며, 이는 다음 코드와 같이 나타납니다:

```
>>> from datetime import date
```

4. create() 메서드를 사용하여 데이터베이스에 책의 레코드를 생성하세요. 다음을 확인하세요. 당신은 모든 필드, 즉 제목(title), 출판일(publication_date), ISBN, 그리고 출판사(publisher)를 전달합니다.

객체:

```
>>> book = Book.objects.create(
    ---
    ---
    title="케라스를 이용한 고급 딥러닝", publication_date=
    date(2018, 10, 31), isbn="9781788629416", publisher=
    publisher)
    ---
```

게시자가 외래 키이고 null 값을 가질 수 없기 때문에 게시자 객체를 반드시 전달해야 한다는 점에 유의하세요. 필수 외래 키 객체인 게시자가 제공되지 않으면 데이터베이스에서 무결성 오류가 발생합니다.

Figure 2.12 shows the Book table, where the first entry is created. Notice that the foreign key field (`publisher_id`) points to the `id` (primary key) of the Publisher table. The `publisher_id` entry in the book's record is pointing to a Publisher record that has `id` (primary key) 1, as shown in the following two screenshots:

The figure consists of two screenshots from a database browser interface. The top screenshot shows the 'reviews_book' table with the following data:

	id	title	publication_date	isbn	publisher_id
1	1	Advanced Deep Learning with Keras	2018-10-31	9781788629416	1

The bottom screenshot shows the 'reviews_publisher' table with the following data:

	id	name	website	email
1	1	Packt Publishing	https://www.packtpub.com/	customersupport@packtpub.com

Figure 2.12: The foreign key `publisher_id` of `reviews_book` points to the primary key of `reviews_publisher`

In this exercise, we learned that while creating a database record, an object can be assigned to a field if it is a foreign key. We know that the Book model also has a many-to-many relationship with the Contributor model. Let's now explore the ways to establish many-to-many relations as we create records in the database.

Exercise 2.04 – Creating records with many-to-many relationships

In this exercise, we will create a many-to-many relationship between Book and Contributor using the BookContributor relationship model:

1. In case you have restarted the shell and lost the publisher and book objects, retrieve them from the database by using the following set of Python statements:

```
>>> from reviews.models import Book
>>> from reviews.models import Contributor
>>> contributor = Contributor.objects.get(
    first_names='Rowel')
>>> book = Book.objects.get(
    title='Advanced Deep Learning with Keras')
```

2. The way to establish a many-to-many relationship is by storing the information about the relationship in the intermediary model or the relationship model; in this case, BookContributor. Since we have already fetched the book and the contributor records

그림 2.12는 첫 번째 항목이 생성된 Book 테이블을 보여줍니다. 외래 키 필드 (publisher_id)가 Publisher 테이블의 id(기본 키)를 가리키고 있음을 주목하세요. 책의 레코드에 있는 publisher_id 항목은 다음 두 스크린샷에 표시된 대로 id(기본 키)가 1인 Publisher 레코드를 가리키고 있습니다:

The first screenshot shows the 'reviews_book' table with the following data:

	id	title	publication_date	isbn	publisher_id
1	1	Advanced Deep Learning with Keras	2018-10-31	9781788629416	1

The second screenshot shows the 'reviews_publisher' table with the following data:

	id	name	website	email
1	1	Packt Publishing	https://www.packtpub.com/	customersupport@packtpub.com

그림 2.12: reviews_book의 외래 키 publisher_id는 reviews_publisher의 기본 키를 가리킵니다

이번 연습에서는 데이터베이스 레코드를 생성할 때 객체가 외래 키인 경우 필드에 할당될 수 있음을 배웠습니다. Book 모델은 Contributor 모델과도 다대다 관계를 가지고 있다는 것을 알고 있습니다. 이제 idx_837aac09 데이터베이스에서 레코드를 생성하면서 다대다 관계를 설정하는 방법을 탐색해 보겠습니다.

연습 2.04 – 다대다 관계로 레코드 생성하기

이번 연습에서는 Book과 Contributor 간의 다대다 idx_e95b3e76 관계를 BookContributor 관계 모델을 사용하여 생성할 것입니다:

1. 셸을 재시작하고 idx_1fe6384e의 publisher 및 book 객체를 잃어버린 경우, 다음의 Python 문을 사용하여 데이터베이스에서 다시 가져오세요:

```
>>> from reviews.models import Book
>>> from reviews.models import Contributor
>>> contributor = Contributor.objects.get(
    first_names='Rowel')
>>> book = Book.objects.get(
    title="Advanced Deep Learning with Keras")
```

2. 다대다 관계를 설정하는 방법은 중간 모델 또는 관계 모델에 관계에 대한 정보를 저장하는 것입니다. 이 경우, BookContributor입니다. 이미 책과 기여자 레코드를 가져왔기 때문에

from the database, let's use these objects while creating a record for the BookContributor relationship model. To do so, first, create an instance of the BookContributor relationship class and then save the object to the database. While doing so, ensure you pass the required fields, namely the book object, the contributor object, and role:

```
>>> from reviews.models import BookContributor
>>> book_contributor = BookContributor(book=book,
    contributor=contributor, role='AUTHOR')
>>> book_contributor.save()
```

Notice that we specified the role as AUTHOR while creating the book_contributor object. This is a classic example of storing relationship data while establishing a many-to-many relationship. The role can be AUTHOR, CO_AUTHOR, or EDITOR.

This established the relationship between the *Advanced Deep Learning with Keras* book and the contributor Rowel (Rowel being the author of the book).

In this exercise, we established a many-to-many relationship between Book and Contributor using the BookContributor relationship model. With regards to the verification of the many-to-many relationship that we just created, we will see this in detail in a few exercises later on in this chapter.

Exercise 2.05 – A many-to-many relationship using the add() method

In this exercise, we will establish a many-to-many relationship using the add() method. When we don't use the relationship to create objects, we can use through_default to pass in a dictionary with the parameters defining the required fields. Continuing from the previous exercise, let's add one more contributor to the book titled *Advanced Deep Learning with Keras*. This time, the contributor is an editor of the book:

1. If you have restarted the shell, run the following two commands to import and fetch the desired book instance:

```
>>> from reviews.models import Book, Contributor
>>> book = Book.objects.get(
    title="Advanced Deep Learning with Keras")
```

데이터베이스에서, theBookContributor 관계 모델의 레코드를 생성할 때 이러한 객체를 사용합니다. 이를 위해 먼저 theBookContributor 관계 클래스의 인스턴스를 생성한 다음, 객체를 데이터베이스에 저장합니다. 이때, 필요한 필드인 책 객체, 기여자 객체 및 역할을 전달해야 합니다:

```
>>> from reviews.models import BookContributor
>>> book_contributor = BookContributor(book=book,
    contributor=contributor, role='AUTHOR')
>>> book_contributor.save()
```

책_기여자 객체를 생성할 때 역할을 AUTHOR로 지정했음을 주목하세요. 이는 다대다 관계를 설정하면서 관계 데이터를 저장하는 전형적인 예입니다. 역할은 AUTHOR, CO_AUTHOR 또는 EDITOR일 수 있습니다.

이것은 Advanced Deep Learning with Keras 책과 기여자 Rowel(책의 저자 Rowel) 간의 관계를 설정했습니다.

이 연습에서는 Book과 Contributor 간의 다대다 관계를 BookContributor 관계 모델을 사용하여 설정했습니다. 우리가 방금 생성한 다대다 관계의 검증에 관해서는, 이 장의 후속 연습에서 자세히 살펴보겠습니다.

연습 2.05 – add() 메서드를 사용한 다대다 관계

이 연습에서는 add() 메서드를 사용하여 다대다 관계를 설정합니다. 관계를 사용하여 객체를 생성하지 않을 때는 through_default를 사용하여 필요한 필드를 정의하는 idx_f875b403 매개변수를 포함하는 사전을 전달할 수 있습니다. 이전 연습에서 계속하여, 제목이 Advanced Deep Learning with Keras인 책에 한 명의 기여자를 더 추가해 보겠습니다. 이번에는 기여자가 책의 편집자입니다:

1. 셸을 재시작한 경우, 원하는 책 인스턴스를 가져오기 위해 다음 두 명령을 실행하세요:

```
>>> from reviews.models import Book, Contributor
>>> book = Book.objects.get(
    title="Advanced Deep Learning with Keras")
```

2. Use the `create()` method to create a contributor, as shown here:

```
>>> contributor = Contributor.objects.create(
    first_names='Packt',
    last_names='Example Editor',
    email='PacktEditor@example.com')
```

3. Add the newly created contributor to the book using the `add()` method. Ensure you provide the role relationship parameter as dict. Enter the following code:

```
>>> book.contributors.add(contributor,
    through_defaults={'role': 'EDITOR'})
```

Thus, we used the `add()` method to establish a many-to-many relationship between the book and the contributor while storing the relationship data role as Editor. Let's now take a look at other ways of doing this.

Using the `create()` and `set()` methods for many-to-many relationships

Assume the *Advanced Deep Learning with Keras* book has a total of two editors. Let's use the following method to add another editor to the book. If the contributor is not already present in the database, then we can use the `create()` method to simultaneously create an entry, as well as to establish the relation with the book:

```
>>> book.contributors.create(first_names='Packtp',
    last_names='Editor Example',
    email='PacktEditor2@example.com',
    through_defaults={'role': 'EDITOR'})
```

Similarly, we can also use the `set()` method to add a list of contributors for a book. Let's create a publisher, a set of two contributors who are the co-authors, and a book object. First, import the Publisher model, if not already imported, using the following code:

```
>>> from reviews.models import Publisher
```

The following code will help us do so:

```
>>> publisher = Publisher.objects.create(
    name='Pocket Books',
```

2. 다음과 같이 `create()` 메서드를 사용하여 기여자를 생성합니다:

```
>>> contributor = Contributor.objects.create(
    first_names='Packt',
    last_names='Example Editor',
    email='PacktEditor@example.com')
```

3. 새로 생성된 기여자를 `add()` 메서드를 사용하여 책에 추가합니다. 역할 관계 매개변수를 `dict`로 제공해야 합니다. 다음 코드를 입력하세요:

```
>>> book.contributors.add(contributor,
    through_defaults={'role': 'EDITOR'})
```

따라서 우리는 `add()` 메서드를 사용하여 책과 기여자 간의 다대다 관계를 설정하고 `idx_7f1b5466` 관계 데이터 역할을 편집자로 저장했습니다. 이제 `idx_881c3a81`를 수행하는 다른 방법을 살펴보겠습니다.

다대다 관계를 위한 `create()` 및 `set()` 메서드 사용

Advanced Deep Learning with Keras 책에는 총 두 명의 편집자가 있다고 가정합니다. 다음 방법을 사용하여 책에 또 다른 편집자를 추가하겠습니다. 기여자가 데이터베이스에 이미 존재하지 않는 경우, `create()` 메서드를 사용하여 동시에 `idx_8982a2ee` 항목을 생성하고 책과의 관계를 설정할 수 있습니다:

```
>>> book.contributors.create(first_names='Packtp',
    last_names='Editor Example',
    email='PacktEditor2@example.com',
    through_defaults={'role': 'EDITOR'})
```

유사하게, `set()` 메서드를 사용하여 책에 대한 기여자 목록을 추가할 수도 있습니다. 먼저 출판사를 생성하고, 두 명의 공동 저자인 기여자 세트를 만들고, 책 객체를 생성하겠습니다. 먼저 다음 코드를 사용하여 `Publisher` 모델을 가져옵니다:

```
>>> from reviews.models import Publisher
```

다음 코드는 이를 수행하는 데 도움이 될 것입니다:

```
>>> publisher = Publisher.objects.create(
    name='Pocket Books',
```

```
website='https://pocketbookssampleurl.com',
email='pocketbook@example.com')
>>> contributor1 = Contributor.objects.create(
    first_names='Stephen', last_names='Stephen',
    email='StephenKing@example.com')
>>> contributor2 = Contributor.objects.create(
    first_names='Peter', last_names='Straub',
    email='PeterStraub@example.com')
>>> book = Book.objects.create(title='The Talisman',
    publication_date=date(2012, 9, 25),
    isbn='9781451697216', publisher=publisher)
```

Since this is a many-to-many relationship, we can add a list of objects in just one go using the `set()` method. We can use `through_defaults` to specify the role of the contributors; in this case, they are co-authors:

```
>>> book.contributors.set([contributor1, contributor2],
    through_defaults={'role': 'CO_AUTHOR'})
```

Read operations

Django provides us with methods that allow us to read/retrieve from the database. We can retrieve a single object from the database using the `get()` method. We have already created a few records in the previous sections, so let's use the `get()` method to retrieve an object.

Exercise 2.06 – Using the `get()` method to retrieve an object

In this exercise, we will retrieve an object from the database using the `get()` method. Follow these steps:

1. Fetch a `Publisher` object that has a `name` field with the `Pocket Books` value:

```
>>> from reviews.models import Publisher
>>> publisher = Publisher.objects.get(
    name='Pocket Books')
```

2. Re-enter the retrieved publisher object and press *Enter*:

```
>>> publisher
<Publisher: Pocket Books>
```

```

website='https://pocketbookssampleurl.com',
email='pocketbook@example.com')
>>> contributor1 = Contributor.objects.create(
    first_names='Stephen', last_names='Stephen',
    email='StephenKing@example.com')
>>> contributor2 = Contributor.objects.create(
    first_names='Peter', last_names='Straub',
    email='PeterStraub@example.com')
>>> book = Book.objects.create(title='The Talisman',
    publication_date=date(2012, 9, 25),
    isbn='9781451697216', publisher=publisher)

```

이것은 다대다 idx_a0a8eb77 관계이므로, 우리는 theset() 메서드를 사용하여 한 번에 객체 목록을 추가할 수 있습니다. 우리는 through_defaults를 사용하여 기여자의 역할 idx_3a21f81b을 지정할 수 있습니다; 이 경우, 그들은 idx_cbf3cd84 공동 저자입니다:

```

>>> book.contributors.set([contributor1, contributor2],
    through_defaults={'role': 'CO_AUTHOR'})

```

읽기 작업

장고는 데이터베이스에서 읽거나 검색할 수 있는 메서드를 제공합니다. 우리는 get() 메서드를 사용하여 데이터베이스에서 단일 객체 idx_fc0ab041를 검색할 수 있습니다. 우리는 이미 이전 섹션에서 몇 개의 레코드를 생성했으므로, get() 메서드를 사용하여 객체를 검색해 보겠습니다.

연습 2.06 – get() 메서드를 사용하여 객체 검색하기

이 연습에서는 get() 메서드를 사용하여 데이터베이스에서 idx_21fc7810 객체를 검색할 것입니다. 다음 단계를 따르세요:

1. 이름 필드가 Pocket Books 값인 Publisher 객체를 가져옵니다:

```

>>> from reviews.models import Publisher
>>> publisher = Publisher.objects.get(
    name='Pocket Books')

```

2. 가져온 퍼블리셔 객체를 다시 입력하고 Enter를 누르세요:

```

>>> publisher
<Publisher: Pocket Books>

```

Notice that the output is displayed in the shell. This is called a string representation of an object. It is the result of adding the `__str__()` model method as we did in the *Model methods* section for the `Publisher` class.

3. Upon retrieving the object, we have access to all the object's attributes. Since this is a Python object, the attributes of the object can be accessed by using dot notation (`.` followed by the attribute name). So, we can retrieve the publisher's name with the following command:

```
>>> publisher.name
'Pocket Books'
```

4. Similarly, to retrieve the publisher's website, use the following:

```
>>> publisher.website
'https://pocketbookssampleurl.com'
```

The publisher's email address can be retrieved as follows:

```
>>> publisher.email
'pocketbook@example.com'
```

In this exercise, we learned how to fetch a single object using the `get()` method. There are several disadvantages to using this method, though. Let's find out what they are next.

Returning an object using the `get()` method

It is important to note that the `get()` method can only fetch one object. If there is another object carrying the same value as the field mentioned, then we can expect a *returned more than one* error message. For example, if there are two entries in the `Publisher` table with the same value for the `name` field, we can expect an error. In such cases, there are alternate ways to retrieve those objects, which we will explore in the subsequent sections.

We can also get a *matching query does not exist* error message when there are no objects returned from the `get()` query. The `get()` method can be used with any of the object's fields to retrieve a record. In the following case, we are using the `website` field:

```
>>> publisher = Publisher.objects.get(
    website='https://pocketbookssampleurl.com')
```

출력이 셸에 표시된다는 점에 유의하십시오. 이것은 객체의 문자열 표현이라고 합니다. 이는 Publisher 클래스의 Modelmethods 섹션에서 했던 것처럼 `__str__()` 모델 메서드를 추가한 결과입니다.

3. 객체를 검색하면 모든 객체의 속성에 접근할 수 있습니다. 이것이 Python 객체이므로, `idx_4d20a049` 객체의 속성은 점 표기법(속성 이름 뒤에 `.`이 오는)을 사용하여 접근할 수 있습니다. 따라서 다음 명령어로 출판사의 이름을 검색할 수 있습니다:

```
>>> publisher.name
'Pocket Books'
```

4. 유사하게, 출판사의 웹사이트를 검색하려면 다음을 사용하십시오:

```
>>> publisher.website
'https://pocketbookssampleurl.com'
```

출판사의 이메일 주소는 다음과 같이 검색할 수 있습니다:

```
>>> publisher.email
'pocketbook@example.com'
```

이번 연습에서는 `get()` 메서드를 사용하여 단일 객체를 가져오는 방법을 배웠습니다. 그러나 이 방법에는 몇 가지 단점이 있습니다. 다음에 그것들이 무엇인지 알아보시다.

get() 메서드를 사용하여 객체 반환하기

`get()` 메서드는 단일 객체만 가져올 수 있다는 점에 유의하는 것이 중요합니다. 만약 `idx_efa415bf` 언급된 필드와 동일한 값을 가진 다른 객체가 있다면, 우리는 하나 이상의 오류 메시지가 반환될 것으로 예상할 수 있습니다. 예를 들어, Publisher 테이블에 이름 필드에 대해 동일한 값을 가진 두 개의 항목이 있다면, 우리는 오류를 예상할 수 있습니다. 이러한 경우, 우리는 후속 섹션에서 탐색할 대체 방법으로 해당 객체를 검색할 수 있습니다.

`get()` 쿼리에서 반환된 객체가 없을 때 일치하는 쿼리가 존재하지 않는 오류 메시지도 받을 수 있습니다. `get()` 메서드는 객체의 필드 중 어떤 것이든 사용하여 레코드를 검색할 수 있습니다. 다음 경우에는 웹사이트 필드를 사용하고 있습니다:

```
>>> publisher = Publisher.objects.get(
    website='https://pocketbookssampleurl.com')
```


After retrieving the object, we can still get the publisher's name, as shown here:

```
>>> publisher.name
'Pocket Books'
```

Another way to retrieve an object is by using its primary key, `pk`, as can be seen here:

```
>>> Publisher.objects.get(pk=2)
<Publisher: Pocket Books>
```

Using `pk` for the primary key is a more generic way of using the primary key field. But for the `Publisher` table, since we know that `id` is the primary key, we can simply use the `id` field name to create our `get()` query:

```
>>> Publisher.objects.get(id=2)
<Publisher: Pocket Books>
```

For `Publisher` and all the other tables, the primary key is `id`, which Django automatically created. This happens when a primary key field is not mentioned at the time of the creation of the table. But there can be instances where a field can be explicitly declared as a primary key.

In the next exercise, we will see how to query and retrieve all the objects for a given model.

Exercise 2.07 – Using the `all()` method to retrieve a set of objects

We can use the `all()` method to retrieve a set of all objects. In this exercise, we will use this method to retrieve the names of all contributors. To do that, follow these steps:

1. Add the following code to retrieve all the objects from the `Contributor` table:

```
>>> from reviews.models import Contributor

>>> Contributor.objects.all()
<QuerySet [<Contributor: Rowel>, <Contributor: Packt>, <Contributor:
Packtp>, <Contributor: Stephen>, <Contributor: Peter>]>
```

Upon execution, we will get `QuerySet` of all the objects.

객체를 검색한 후에도 출판사의 이름을 여전히 얻을 수 있습니다. 여기에서 볼 수 있습니다:

```
>>> publisher.name
'Pocket Books'
```

객체를 검색하는 또 다른 방법은 기본 키인 pk를 사용하는 것입니다. 여기에서 볼 수 있습니다:

```
>>> Publisher.objects.get(pk=2)
<Publisher: Pocket Books>
```

pk를 기본 키로 사용하는 것은 기본 키 필드를 사용하는 보다 일반적인 방법입니다. 그러나 Publisher 테이블의 경우 id가 기본 키라는 것을 알고 있으므로 id 필드 이름을 사용하여 get() 쿼리를 생성할 수 있습니다:

```
>>> Publisher.objects.get(id=2)
<Publisher: Pocket Books>
```

퍼블리셔와 다른 모든 테이블에 대해 기본 키는 id이며, 이는 Django가 자동으로 생성합니다. 이는 테이블 생성 시 aidx_dc855fb0 기본 키 필드가 언급되지 않을 때 발생합니다. 그러나 필드를 명시적으로 기본 키로 선언할 수 있는 경우도 있습니다.

다음 연습에서는 주어진 모델에 대한 모든 객체를 쿼리하고 검색하는 방법을 살펴보겠습니다.

연습 2.07 – all() 메서드를 사용하여 객체 집합 검색하기

all() 메서드를 사용하여 모든 객체의 집합을 검색할 수 있습니다. 이 연습에서는 이 메서드를 사용하여 모든 기여자의 이름 idx_e87fe281을 검색합니다. 그렇게 하려면 idx_8c1c7133 다음 단계를 따르세요:

1. Contributor 테이블에서 모든 객체를 검색하기 위해 다음 코드를 추가하세요:

```
>>> from reviews.models import Contributor

>>> Contributor.objects.all()
<QuerySet [<Contributor: Rowel>, <Contributor: Packt>, <Contributor: Packtp>, <Contributor: Stephen>, <Contributor: Peter>]>
```

실행 시, 모든 객체의 QuerySet을 얻을 수 있습니다.

2. We can use list indexing to look up a specific object or to iterate over the list using a loop to do any other operation:

```
>>> contributors = Contributor.objects.all()
```

3. Since Contributor is a list of objects, we can use indexing to access any element in the list, as shown in the following command:

```
>>> contributors[0]
<Contributor: Rowel>
```

In this case, the first element in the list is a contributor with a `first_names` value of 'Rowel' and a `last_names` value of 'Atienza', as we can see from the following code:

```
>>> contributors[0].first_names
'Rowel'
>>> contributors[0].last_names
'Atienza'
```

In this exercise, we learned how to retrieve all the objects using the `all()` method and how to use the retrieved set of objects as a list.

Retrieving objects by filtering

If we have more than one object for a field value, we cannot use the `get()` method since the `get()` method can return only one object. For such cases, we have the `filter()` method, which can retrieve all the objects that match a specified condition.

Exercise 2.08 – Using the `filter()` method to retrieve objects

In this exercise, we will use the `filter()` method to get a specific set of objects for a certain condition. Specifically, we will retrieve all the contributors' names whose first name is Peter. To do that, follow these steps:

1. First, create two more contributors:

```
>>> from reviews.models import Contributor
>>> Contributor.objects.create(first_names='Peter',
                               last_names='Wharton',
                               email='PeterWharton@example.com')
>>> Contributor.objects.create(first_names='Peter',
```

2. 특정 객체를 조회하거나 루프를 사용하여 목록을 반복하여 다른 작업을 수행하기 위해 목록 인덱싱을 사용할 수 있습니다:

```
>>> contributors = Contributor.objects.all()
```

3. Contributor는 객체 목록이므로 다음 명령에서 볼 수 있듯이 목록의 모든 요소에 접근하기 위해 인덱싱을 사용할 수 있습니다:

```
>>> contributors[0]
<Contributor: Rowel>
```

이 경우 목록의 첫 번째 요소 idx_98d9dad3은 'Rowel'이라는 first_names 값과 'Atienza'라는 last_names 값을 가진 기여자입니다. 다음 코드에서 확인할 수 있습니다:

```
>>> contributors[0].first_names
'로웰'
>>> contributors[0].last_names
'아텐자'
```

이 연습에서는 all() 메서드를 사용하여 모든 객체를 검색하는 방법과 검색된 객체 집합을 목록으로 사용하는 방법을 배웠습니다.

객체 필터링으로 검색하기

필드 값에 대해 객체가 하나 이상 있는 경우, get() 메서드를 사용할 수 없습니다. get() 메서드는 하나의 객체만 반환할 수 있기 때문입니다. 이러한 경우에는 filter() 메서드가 있으며, 이는 지정된 조건과 일치하는 모든 객체를 검색할 수 있습니다.

연습 2.08 – filter() 메서드를 사용하여 객체 검색하기

이 연습에서는 filter() 메서드를 사용하여 특정 조건에 대한 특정 객체 집합을 가져올 것입니다. 구체적으로, 우리는 첫 번째 이름이 피터인 모든 기여자의 이름을 검색할 것입니다. 그렇게 하려면 다음 단계를 따르세요:

1. 먼저, 두 명의 기여자를 더 만듭니다:

```
>>> from reviews.models import Contributor
>>> Contributor.objects.create(first_names='Peter',
    last_names='Wharton',
    email='PeterWharton@example.com')
>>> Contributor.objects.create(first_names='Peter',
```

```
last_names='Tyrrell',  
email='PeterTyrrell@example.com')
```

2. To retrieve those contributors who have a `first_names` value of Peter, add the following code:

```
>>> Contributor.objects.filter(first_names='Peter')  
<QuerySet [<Contributor: Peter>, <Contributor: Peter>,  
<Contributor: Peter>]>
```

3. The `filter()` method returns the object even if there is only one. We can see this here:

```
>>> Contributor.objects.filter(first_names='Rowel')  
<QuerySet [<Contributor: Rowel>]>
```

4. Furthermore, the `filter()` method returns an empty `QuerySet` if none matches the query. This can be seen here:

```
>>> Contributor.objects.filter(first_names='Nobody')  
<QuerySet []>
```

In this exercise, we saw the use of filters to retrieve a set of a few objects filtered by a certain condition. In the next section, we will learn about filtering by using field lookups.

Filtering by field lookups

Now, let's suppose we want to filter and query a set of objects using the objects' fields by providing certain conditions. In a case, we can use a **double-underscore lookup**. For example, the `Book` object has a `publication_date` field; let's say we want to filter and fetch all the books that were published after 01-01-2014. We can easily look these up by using the double-underscore method. To do this, we will first import the `Book` model:

```
>>> from reviews.models import Book  
>>> book = Book.objects.filter(  
    publication_date__gt=date(2014, 1, 1))
```

Here, `publication_date__gt` indicates the publication date, which is greater than (`gt`) a certain specified date – in this case, 01-01-2014. Similar to this, we have the following abbreviations:

- `lt`: Less than

```
last_names='Tyrrell',
email='PeterTyrrell@example.com')
```

2. 기여자 idx_7d0ccd2d를 검색하려면 first_names 값이 Peter인 기여자를 추가하세요.

다음 코드를 입력하세요:

```
>>> Contributor.objects.filter(first_names='Peter')
<QuerySet [<Contributor: Peter>, <Contributor: Peter>,
<Contributor: Peter>]>
```

3. filter() 메서드는 하나만 있어도 객체를 반환합니다. 여기에서 이를 볼 수 있습니다:

```
>>> Contributor.objects.filter(first_names='Rowel')
<QuerySet [<Contributor: Rowel>]>
```

4. 또한, filter() 메서드는 일치하는 항목이 없으면 빈 QuerySet을 반환합니다. 여기에서 이를 볼 수 있습니다:

```
>>> Contributor.objects.filter(first_names='Nobody')
<QuerySet []>
```

이번 연습에서는 특정 조건으로 필터링된 몇 개의 객체 집합을 검색하기 위해 필터를 사용하는 방법을 보았습니다. 다음 섹션에서는 필드 조회를 사용한 필터링에 대해 배울 것입니다.

필드 조회에 의한 필터링

이제, 객체의 필드를 사용하여 특정 조건을 제공함으로써 객체 집합을 필터링하고 쿼리하려고 한다고 가정해 보겠습니다. aidx_21fd591c의 경우, 더블 언더스코어 조회를 사용할 수 있습니다. 예를 들어, Book 객체에는 publication_date 필드가 있습니다. 2014년 1월 1일 이후에 출판된 모든 idx_dbde613c 책을 필터링하고 가져오고 싶다고 가정해 보겠습니다. 더블 언더스코어 메서드를 사용하여 쉽게 조회할 수 있습니다. 이를 위해 먼저 Book 모델을 가져옵니다:

```
>>> from reviews.models import Book
>>> book = Book.objects.filter(
    publication_date__gt=date(2014, 1, 1))
```

여기서 publication_date__gt는 특정 지정된 날짜보다 큰 (gt) 출판 날짜를 나타냅니다. 이 경우 2014년 1월 1일입니다. 이와 유사하게 다음과 같은 약어가 있습니다:

- lt: 미만

- lte: Less than or equal to
- gte: Greater than or equal to

The result after filtering can be seen here:

```
>>> book
<QuerySet [(<Book: Advanced Deep Learning with Keras>)]>
```

Here is the publication date of the book that is part of the query set, which confirms that the publication date was after 01-01-2014:

```
>>> book[0].publication_date
datetime.date(2018, 10, 31)
```

Using pattern matching for filtering operations

For filtered results, we can also look up whether the parameter contains a part of the string we are looking for:

```
>>> book = Book.objects.filter(
    title__contains='Deep learning')
```

Here, title__contains looks for all those objects with titles containing 'Deep learning' as a part of the string:

```
>>> book
<QuerySet [(<Book: Advanced Deep Learning with Keras>)]>

>>> book[0].title
'Advanced Deep Learning with Keras'
```

Similarly, we can use icontains if the string match needs to be case-insensitive. Using startswith matches any string starting with the specified string.

- lte: 작거나 같음
- gte: 크거나 같음 필터링 후 결

과는 여기에서 볼 수 있습니다:

```
>>> 책
<QuerySet [<Book: Advanced Deep Learning with Keras>]>
```

쿼리 세트의 일부인 theidx_b45d3fba 책의 출판 날짜는 theidx_0223a8d8 출판 날짜가 2014-01-01 이후임을 확인합니다:

```
>>> 책
[0].publication_datedatetime.date
(2018, 10, 31)
```

필터링 작업을 위한 패턴 매칭 사용

필터링된 결과를 위해, 우리는 idx_7909b736 매개변수가 우리가 찾고 있는 문자열의 일부를 포함하는지 확인할 수 있습니다:

```
>>> book = Book.objects.filter(
    title__contains='Deep learning')
```

여기서 title__contains는 'Deep learning'을 문자열의 일부로 포함하는 모든 객체를 찾습니다:

```
>>> book
<QuerySet [<Book: Advanced Deep Learning with Keras>]>

>>> book[0].title
'Advanced Deep Learning with Keras'
```

유사하게, idx_87baa2bc 문자열 일치가 대소문자를 구분하지 않아야 하는 경우 icontains를 사용할 수 있습니다. startswith는 지정된 문자열로 시작하는 모든 문자열과 일치합니다.

Retrieving objects by using the `exclude()` method

In the previous section, we learned about fetching a set of objects by matching a certain condition. Now, suppose we want to do the opposite; that is, we want to fetch all those objects that do not match a certain condition. In such cases, we can use the `exclude()` method to exclude a certain condition and fetch all the required objects. This will be clearer with an example. The following is a list of all contributors:

```
>>> Contributor.objects.all()
<QuerySet [<Contributor: Rowel>, <Contributor: Packt>,
<Contributor: Packtp>, <Contributor: Stephen>,
<Contributor: Peter>, <Contributor: Peter>,
<Contributor: Peter>]>
```

Now, from this list, we will exclude all those contributors who have the `first_names` value of Peter:

```
>>> Contributor.objects.exclude(first_names='Peter')
<QuerySet [<Contributor: Rowel>, <Contributor: Packt>,
<Contributor: Packtp>, <Contributor: Stephen>]>
```

We see here that the query returned all those contributors whose first name is not Peter.

Retrieving objects using the `order_by()` method

We can retrieve a list of objects while ordering by a specified field using the `order_by()` method. For example, in the following code snippet, we order the books by their publication date:

```
>>> books = Book.objects.order_by("publication_date")
>>> books
<QuerySet [<Book: The Talisman>, <Book: Advanced Deep Learning with Keras>]>
```

Let's examine the order of the query. Since the query set is a list, we can use indexing to check the publication date of each book:

```
>>> books[0].publication_date
datetime.date(2012, 9, 25)
>>> books[1].publication_date
datetime.date(2018, 10, 31)
```

exclude() 메서드를 사용하여 객체 검색하기

이전 섹션에서는 특정 조건에 맞는 객체 집합을 가져오는 방법에 대해 배웠습니다. 이제, 반대로 특정 조건에 맞지 않는 모든 객체를 가져오고 싶다고 가정해 보겠습니다. 이러한 경우, `exclude()` 메서드를 사용하여 특정 조건을 제외하고 필요한 모든 객체를 가져올 수 있습니다. 예를 들어 설명하겠습니다. 다음은 모든 기여자의 목록입니다:

```
>>> Contributor.objects.all()
<QuerySet [
  <Contributor: Rowel>, <Contributor: Packt>,
  <Contributor: Packtp>, <Contributor: Stephen>,
  <Contributor: Peter>, <Contributor: Peter>,
  <Contributor: Peter>]>
```

이 목록에서, 우리는 `first_names` 값이 `Peter`인 모든 기여자를 제외할 것입니다:

```
>>> Contributor.objects.exclude(first_names='Peter')
<QuerySet [
  <Contributor: Rowel>, <Contributor: Packt>,
  <Contributor: Packtp>, <Contributor: Stephen>]>
```

여기서 우리는 `queryidx_44df47db`가 이름이 `Peter`가 아닌 모든 기여자를 반환했음을 봅니다.

order_by() 메서드를 사용하여 객체 검색하기

`order_by()` 메서드를 사용하여 지정된 필드로 정렬된 객체 목록을 검색할 수 있습니다. 예를 들어, `idx_5538c67afollowing` 코드 스니펫에서 우리는 `idx_51561755` 책을 출판 날짜로 정렬합니다:

```
>>> books = Book.objects.order_by("publication_date")
>>> books
<QuerySet [
  <Book: The Talisman>, <Book: Advanced Deep Learning with Keras>]>
```

쿼리의 순서를 살펴보겠습니다. 쿼리 세트는 목록이므로 인덱싱을 사용하여 각 책의 출판 날짜를 확인할 수 있습니다:

```
>>> books
[0].publication_datedatetime.
date(2012, 9, 25)>>> books
[1].publication_datedatetime.
date(2018, 10, 31)
```

Notice that the publication date of the first book with a 0 index is older than the publication date of the second book with a 1 index. So, this confirms that the queried list of books has been properly ordered as per their publication dates. We can also use a prefix with the negative sign for the field parameter to order results in descending order. This can be seen from the following code snippet:

```
>>> books = Book.objects.order_by("-publication_date")
>>> books

<QuerySet [
<Book: Advanced Deep Learning with Keras>,
<Book: The Talisman>]>
```

Since we have prefixed a negative sign to the publication date, notice that the queried set of books has now been returned in the opposite order, where the first book object with a 0 index has a more recent date than the second book:

```
>>> books[0].publication_date
datetime.date(2018, 10, 31)

>>> books[1].publication_date
datetime.date(2012, 9, 25)
```

We can also order by using a string field or a numerical one. For example, the following code can be used to order books by their primary key or id:

```
>>> books = Book.objects.order_by('id')
>>> books

<QuerySet [
<Book: Advanced Deep Learning with Keras>,
<Book: The Talisman>]>
```

The queried set of books has been ordered as per book id in ascending order:

```
>>> books[0].id
1
>>> books[1].id
2
```

첫 번째 책의 0 인덱스 출판 날짜가 두 번째 책의 1 인덱스 출판 날짜보다 오래되었음을 주목하십시오. 따라서, 이는 요청된 책 목록이 출판 날짜에 따라 올바르게 정렬되었음을 확인합니다. 또한, 필드 매개변수에 음수 기호를 접두사로 사용하여 결과를 내림차순으로 정렬할 수 있습니다. 이는 다음 코드 스니펫에서 볼 수 있습니다:

```
>>> books = Book.objects.order_by("-publication_date")
>>> books

<QuerySet [
<Book: Advanced Deep Learning with Keras>,
<Book: The Talisman]>]
```

출판 날짜에 음수 기호를 접두사로 추가했기 때문에, 요청된 책 세트가 이제 idx_8dc75ab1 반대 순서로 반환되었음을 주목하십시오. 여기서 0 인덱스를 가진 첫 번째 책 객체는 두 번째 책보다 더 최근 날짜를 가지고 있습니다:

```
>>> books[0].publication_date
datetime.date(2018, 10, 31)

>>> books[1].publication_date
datetime.date(2012, 9, 25)
```

문자열 필드나 숫자 필드를 사용하여 정렬할 수도 있습니다. 예를 들어, 다음 코드를 사용하여 책을 idx_e4d27825 기본 키 또는 id로 정렬할 수 있습니다:

```
>>> books = Book.objects.order_by('id')
>>> books

<QuerySet [
<Book: Advanced Deep Learning with Keras>,
<Book: The Talisman]>]
```

쿼리된 책 세트는 책 id에 따라 오름차순으로 정렬되었습니다:

```
>>> books[0].id
1
>>> books[1].id
2
```

Again, to order in descending order, the negative sign can be used as a prefix, as follows:

```
>>> books = Book.objects.order_by('-id')
>>> books
<QuerySet [ <Book: The Talisman>, <Book: Advanced Deep Learning with Keras>]>
```

Now, the queried set of books has been ordered as per book id in descending order:

```
>>> books[0].id
2
>>> books[1].id
1
```

To order by a string field in alphabetical order, we can do something like this:

```
>>> books = Book.objects.order_by('title')
>>> books
<QuerySet [ <Book: Advanced Deep Learning with Keras>, <Book: The Talisman>]>
```

Since we have used the title of the book to order by, the query set has been ordered in alphabetical order. We can see this as follows:

```
>>> books[0]
<Book: Advanced Deep Learning with Keras>
>>> books[1]
<Book: The Talisman>
```

Similar to what we've seen for the previous ordering types, the negative sign prefix can help us sort in reverse alphabetical order, as we can see here:

```
>>> books = Book.objects.order_by('-title')
>>> books
<QuerySet [ <Book: The Talisman>, <Book: Advanced Deep Learning with Keras>]>
```

This will lead to the following output:

```
>>> books[0]
<Book: The Talisman>
>>> books[1]
<Book: Advanced Deep Learning with Keras>
```

다시 말해, 내림차순으로 정렬하기 위해 idx_4697f09b 부호를 접두사로 사용할 수 있습니다, 다음과 같이:

```
>>> books = Book.objects.order_by('-id')
>>> books
<QuerySet [<Book: The Talisman>, <Book: Advanced Deep Learning with Keras>]>
```

이제 쿼리된 책 세트는 책 ID에 따라 내림차순으로 정렬되었습니다:

```
>>> books[0].id
2
>>> books[1].id
1
```

문자 필드를 알파벳 순서로 정렬하려면, 다음과 같은 방법을 사용할 수 있습니다:

```
>>> books = Book.objects.order_by('title')
>>> books
<QuerySet [<Book: Advanced Deep Learning with Keras>, <Book: The Talisman>]>
```

책의 titleidx_e236f402를 사용하여 정렬했기 때문에, 쿼리 세트는 알파벳 순서로 정렬되었습니다. 다음과 같이 확인할 수 있습니다:

```
>>> books[0]
<Book: Advanced Deep Learning with Keras>
>>> books[1]
<Book: The Talisman>
```

이전 주문 유형에서 본 것과 유사하게, 음수 기호 접두사는 역 알파벳 순서로 정렬하는데 도움이 될 수 있습니다. 여기에서 볼 수 있습니다:

```
>>> books = Book.objects.order_by('-title')
>>> books
<QuerySet [<Book: The Talisman>, <Book: Advanced Deep Learning with Keras>]>
```

이것은 다음과 같은 출력을 초래합니다:

```
>>> books[0]
<Book: The Talisman>
>>> books[1]
<Book: Advanced Deep Learning with Keras>
```

Yet another useful method offered by Django is `values()`. It helps us get a query set of dictionaries instead of objects. In the following code snippet, we're using this for a `Publisher` object:

```
>>> publishers = Publisher.objects.all().values()

>>> publishers
<QuerySet [{ 'id': 1, 'name': 'Packt Publishing', 'website':
'https://www.packtpub.com', 'email':
'customersupport@packtpub.com'}, { 'id': 2, 'name':
'Pocket Books', 'website':
'https://pocketbookssampleurl.com',
'email': 'pocketbook@example.com'}]>
>>> publishers[0]
{ 'id': 1, 'name': 'Packt Publishing', 'website':
'https://www.packtpub.com', 'email':
'customersupport@packtpub.com'}
>>> publishers[0]
{ 'id': 1, 'name': 'Packt Publishing', 'website':
'https://www.packtpub.com', 'email':
'customersupport@packtpub.com'}
```

In the next few sections, we will explore how to query across relationships.

Querying across relationships

As we have studied in this chapter, the reviews app has two kinds of relationships: many-to-one and many-to-many. So far, we have learned various ways of making queries using `get()`, filters, field lookups, and so on. Now let's study how to perform queries across relationships. There are several ways to go about this – we could use foreign keys, object instances, and more. Let's explore these with the help of some examples.

Django에서 제공하는 또 다른 유용한 방법인 `values()`가 있습니다. 이는 객체 대신 사전의 쿼리 집합을 가져오는 데 도움이 됩니다. 다음 코드 조각에서는 `Publisher` 객체에 대해 이를 사용하고 있습니다:

```
>>> publishers = Publisher.objects.all().values()

>>> publishers
<QuerySet [{ 'id': 1, 'name': 'Packt Publishing', 'website':
'https://www.packtpub.com', 'email':
'customersupport@packtpub.com'}, { 'id': 2, 'name':
'Pocket Books', 'website':
'https://pocketbookssampleurl.com',
'email': 'pocketbook@example.com'}]>
>>> publishers[0]
{ 'id': 1, 'name': 'Packt Publishing', 'website':
'https://www.packtpub.com', 'email':
'customersupport@packtpub.com'}
>>> publishers[0]
{ 'id': 1, 'name': 'Packt Publishing', 'website':
'https://www.packtpub.com', 'email':
'customersupport@packtpub.com'}
```

다음 섹션에서는 `weidx_a406fa20`가 관계를 통해 쿼리하는 방법을 탐구할 것입니다.

관계 간 쿼리하기

이 장에서 공부한 바와 같이, 리뷰 앱에는 두 가지 종류의 관계가 있습니다: 다대일 및 다대다. 지금까지 우리는 `get()`, `filters`, `field lookups`, 및 `soidx_a7eea8ce on`을 사용하여 쿼리를 만드는 다양한 방법을 배웠습니다. 이제 관계 간 쿼리를 수행하는 방법을 공부해 봅시다. 이를 수행하는 방법에는 여러 가지가 있습니다 – 외래 키, 객체 인스턴스 등을 사용할 수 있습니다. 몇 가지 예제를 통해 이를 탐구해 봅시다.

Querying using foreign keys

When we have relationships across two models/tables, Django provides a way to perform a query using the relationships. The command shown in this section will retrieve all the books published by Packt Publishing by performing a query using model relationships. Similar to what we've seen previously, this is done using the double-underscore lookup. For example, the Book model has a foreign key of publisher pointing to the Publisher model. Using this foreign key, we can perform a query using double underscores and the name field in the Publisher model. This can be seen from the following code:

```
>>> Book.objects.filter(publisher__name='Packt Publishing')
<QuerySet [ <Book: Advanced Deep Learning with Keras> ]>
```

Querying using the model name

Another way of querying is using a relationship to do the query backward, using the model name in lowercase. For instance, let's say we want to query the publisher who published the *Advanced Deep Learning with Keras* book using model relationships in the query. For this, we can execute the following statement to retrieve the Publisher information object:

```
>>> Publisher.objects.get(
    book__title='Advanced Deep Learning with Keras')
<Publisher: Packt Publishing>
```

Here, book is the model's name in lowercase. As we already know, the Book model has a publisher foreign key with a name value of Packt Publishing.

Querying across foreign key relationships using the object instance

We can also retrieve the information using the object's foreign key. Suppose we want to query the publisher's name for the title *The Talisman*:

```
>>> book = Book.objects.get(title='The Talisman')
>>> book.publisher
<Publisher: Pocket Books>
```

외래 키를 사용한 쿼리

두 모델/테이블 간에 관계가 있을 때, Django는 관계를 사용하여 쿼리를 수행하는 방법을 제공합니다. 이 섹션에 표시된 명령은 모델 관계를 사용하여 Packt Publishing에서 발행한 모든 idx_8218f640책을 검색합니다. 이전에 본 것과 유사하게, 이는 이중 언더스코어 조화를 사용하여 수행됩니다. 예를 들어, Book 모델은 Publisher 모델을 가리키는 publisher의 외래 키를 가지고 있습니다. 이 외래 키를 사용하여, 우리는 Publisher 모델의 name 필드와 이중 언더스코어를 사용하여 쿼리를 수행할 수 있습니다. 다음 코드에서 확인할 수 있습니다:

```
>>> Book.objects.filter(publisher__name='Packt Publishing')
<QuerySet [Book: Advanced Deep Learning with Keras]>
```

모델 이름을 사용한 쿼리

쿼리하는 또 다른 방법은 모델 이름을 소문자로 사용하여 관계를 통해 쿼리를 역으로 수행하는 것입니다. 예를 들어, Advanced Deep Learning with Keras 책을 출판한 출판사를 쿼리하고 싶다고 가정해 보겠습니다. 이를 위해 쿼리에서 idx_ee6f8128모델 관계를 사용하여 다음 문장을 실행하여 Publisher 정보 객체를 검색할 수 있습니다:

```
>>> Publisher.objects.get(
    book__title='Advanced Deep Learning with Keras')
<Publisher: Packt Publishing>
```

여기서 book은 소문자로 된 모델의 이름입니다. 이미 알고 있듯이, Book 모델은 Packt Publishing이라는 이름 값을 가진 apublisher 외래 키를 가지고 있습니다.

객체 인스턴스를 사용하여 외래 키 관계를 통해 쿼리하기

idx_bfa1a8c9 객체의 외래 키를 사용하여 정보를 검색할 수도 있습니다. 제목 The Talisman의 출판사 이름을 쿼리하고 싶다고 가정해 보겠습니다:

```
>>> book = Book.objects.get(title='The Talisman')
>>> book.publisher
<Publisher: Pocket Books>
```

Using the object here is an example of where we use the reverse direction to get all the books published by a publisher by using the `set.all()` method:

```
>>> publisher = Publisher.objects.get(name='Pocket Books')
>>> publisher.book_set.all()
<QuerySet [<Book: The Talisman>]>
```

We can also create queries using chains of queries:

```
>>> Book.objects.filter(
    publisher__name='Pocket Books').filter(
    title='The Talisman')
<QuerySet [<Book: The Talisman>]>
```

Let's perform some more exercises to shore up our knowledge of the various kinds of queries we have learned about so far.

Exercise 2.09 – Querying across a many-to-many relationship using the field lookup

We know that `Book` and `Contributor` have a many-to-many relationship. In this exercise, without creating an object, we will perform a query to retrieve all the contributors who contributed to writing the book titled *The Talisman*:

1. First, import the `Contributor` class:

```
>>> from reviews.models import Contributor
```

2. Now, add the following code to query for the set of contributors on *The Talisman*:

```
>>> Contributor.objects.filter(
    book__title='The Talisman')
```

We should see the following:

```
<QuerySet [<Contributor: Stephen>, <Contributor: Peter>]>
```

From the preceding output, we can see that Stephen and Peter are the contributors who contributed to writing the book *The Talisman*. The query uses the book model (written in lowercase) and does a field lookup for the title field using the double underscore, as shown in the command.

여기서 객체를 사용하는 것은 idx_5affe0f4에 의해 출판사가 발행한 모든 책을 가져오기 위해 역방향을 사용하는 예입니다. `set.all()` 메서드를 사용하여:

```
>>> publisher = Publisher.objects.get(name='Pocket Books')
>>> publisher.book_set.all()
<QuerySet [<Book: The Talisman>]>
```

또한 쿼리 체인을 사용하여 idx_f67574ed 쿼리를 생성할 수 있습니다:

```
>>> Book.objects.filter(
    publisher__name='Pocket Books').filter(
    title='The Talisman')
<QuerySet [<Book: The Talisman>]>
```

지금까지 배운 다양한 종류의 쿼리에 대한 지식을 강화하기 위해 몇 가지 더 연습을 해봅시다.

연습 2.09 – 필드 조회를 사용하여 다대다 관계에서 쿼리하기

Book과 Contributor는 다대다 관계를 가지고 있다는 것을 알고 있습니다. 이 연습에서는 idx_71514880 객체를 생성하지 않고, idx_61cd10b9a 쿼리를 수행하여 The Talisman이라는 제목의 책을 쓰는 데 기여한 모든 기여자를 검색합니다:

1. 먼저, Contributor 클래스를 가져옵니다:

```
>>> from reviews.models import Contributor
```

2. 이제 The Talisman의 기여자 집합을 쿼리하기 위해 다음 코드를 추가합니다:

```
>>> Contributor.objects.filter(
    book__title='The Talisman')
```

다음 내용을 확인해야 합니다:

```
<QuerySet [<Contributor: Stephen>, <Contributor: Peter>]>
```

앞선 출력에서, 우리는 Stephen과 Peter가 책 The Talisman을 집필한 기여자임을 알 수 있습니다. 쿼리는 책 모델(소문자로 작성됨)을 사용하고, 명령어에 표시된 대로 더블 언더스코어를 사용하여 제목 필드에 대한 필드 조회를 수행합니다.

In this exercise, we learned how to perform queries across many-to-many relationships using a field lookup. Let's now look at using another method to carry out the same task.

Exercise 2.10 – A many-to-many query using objects

In this exercise, using a Book object, search for all the contributors who contributed to writing the book with the title *The Talisman*. The following steps will help us do that:

1. Import the Book model:

```
>>> from reviews.models import Book
```

2. Retrieve a book object with the title *The Talisman*, by adding the following line of code:

```
>>> book = Book.objects.get(title='The Talisman')
```

3. Then retrieve all the contributors who worked on the book *The Talisman* using the book object. Add the following code to do so:

```
>>> book.contributors.all()  
<QuerySet [<Contributor: Stephen>, <Contributor: Peter>]>
```

Again, we can see that Stephen and Peter are the contributors who worked on the book *The Talisman*. Since the book has a many-to-many relationship with contributors, we have used the `contributors.all()` method to get a query set of all those contributors who worked on the book. Now, let's try using the set method to perform a similar task.

Exercise 2.11 – A many-to-many query using the set() method

In this exercise, we will use a contributor object to fetch all the books written by the contributor named Rowel:

1. Import the Contributor model:

```
>>> from reviews.models import Contributor
```

이 연습에서는 필드 조회를 사용하여 다대다 관계에서 쿼리를 수행하는 방법을 배웠습니다. 이제 같은 작업을 수행하기 위해 다른 방법을 사용하는 것을 살펴보겠습니다.

연습 2.10 – 객체를 사용한 다대다 쿼리

이 연습에서는 Book 객체를 사용하여 제목이 The Talisman인 책을 집필하는 데 기여한 모든 idx_b3f99a87 기여자를 검색합니다. 다음 단계가 도움이 될 것입니다:

1. Book 모델을 가져옵니다:

```
>>> from reviews.models import Book
```

2. 다음 코드 줄을 추가하여 제목이 The Talisman인 책 객체를 검색합니다:

```
>>> book = Book.objects.get(title='The Talisman')
```

3. 그런 다음 책 The Talisman에 대해 작업한 모든 기여자를 책 객체를 사용하여 검색합니다. 이를 수행하기 위해 다음 코드를 추가합니다:

```
>>> book.contributors.all()
<QuerySet [<Contributor: Stephen>, <Contributor: Peter>]>
```

다시 말해, Stephen과 Peter가 책 The Talisman에 대해 작업한 기여자라는 것을 알 수 있습니다. 이 책은 기여자와 다대다 관계를 가지고 있으므로, `book.contributors.all()` 메서드를 사용하여 이 책에 대해 작업한 모든 기여자의 쿼리 세트를 가져왔습니다. 이제 set 메서드를 사용하여 유사한 작업을 수행해 보겠습니다.

연습 2.11 – set() 메서드를 사용한 다대다 쿼리

이 연습에서는 기여자 객체를 사용하여 Rowel이라는 이름의 기여자가 쓴 모든 책을 가져옵니다:

1. Contributor 모델을 가져옵니다:

```
>>> from reviews.models import Contributor
```

2. Fetch a contributor object whose `first_names` is 'Rowel' using the `get()` method:

```
>>> contributor = Contributor.objects.get(
    first_names='Rowel')
```

3. Using the contributor object and the `book_set()` method, get all those books written by the contributor:

```
>>> contributor.book_set.all()
<QuerySet [Book: Advanced Deep Learning with Keras]>
```

Since `Book` and `Contributor` have a many-to-many relationship, we can use the `set()` method to query a set of objects associated with the model. In this case, `contributor.book_set.all()` returned all the books written by the contributor.

Exercise 2.12 – Using the `update()` method

In this exercise, we will use the `update()` method to update an existing record:

1. Change `first_names` for a contributor who has the last name `Tyrrell`:

```
>>> from reviews.models import Contributor
>>> Contributor.objects.filter(
    last_names='Tyrrell').update(
    first_names='Mike')
```

1

The return value shows the number of records that have been updated. In this case, one record has been updated.

2. Fetch the contributor that was just modified using the `get()` method and verify that the first name has been changed to Mike:

```
>>> Contributor.objects.get(
    last_names='Tyrrell').first_names
'Mike'
```

Note

If the filter operation has more than one record, then the `update()` method will update the specified field in all the records returned by the filter.

2. `get()` 메서드를 사용하여 `first_names`가 'Rowel'인 기여자 객체를 가져옵니다:

```
>>> contributor = Contributor.objects.get(
    first_names='Rowel')
```

3. 기여자 객체와 `book_set()` 메서드를 사용하여 작성된 모든 책을 가져오세요. 기여자에 의해:

```
>>> contributor.book_set.all()
<QuerySet [<Book: Advanced Deep Learning with Keras>]>
```

`Book`과 `Contributor`는 다대다 관계이므로, `set()` 메서드를 사용하여 모델과 관련된 객체 집합을 쿼리할 수 있습니다. 이 경우, `contributor.book_set.all()`은 기여자가 쓴 모든 책을 반환했습니다.

연습 2.12 – `update()` 메서드 사용하기

이 연습에서는 `update()` 메서드를 사용하여 기존의 `idx_572ee4d0` 레코드를 `idx_725d7377` 업데이트할 것입니다:

1. 성이 `Tyrrell`인 기여자의 `first_names`를 변경합니다:

```
>>> from reviews.models import Contributor
>>> Contributor.objects.filter(
    last_names='Tyrrell').update(
    first_names='Mike')
```

1

반환 값은 업데이트된 레코드의 수를 보여줍니다. 이 경우, 하나의 레코드가 업데이트되었습니다.

2. `get()` 메서드를 사용하여 방금 수정된 기여자 `idx_eb6a64e7`을 가져오고 `idx_5422fdc1`의 이름이 `Mike`로 변경되었는지 확인합니다:

```
>>> Contributor.objects.get(
    last_names='Tyrrell').first_names
'Mike'
```

참고

필터 작업이 하나 이상의 레코드를 반환하는 경우, `update()` 메서드는 필터에 의해 반환된 모든 레코드에서 지정된 필드를 업데이트합니다.

In this exercise, we learned how to use the `update()` method to update a record in the database. Now, finally, let's try deleting a record from the database using the `delete()` method.

Exercise 2.13 – Using the `delete()` method

An existing record in the database can be deleted using the `delete()` method. In this exercise, we will delete a record from the `contributors` table that has a `last_name` value of `Wharton`:

1. Fetch the object using the `get` method and use the `delete` method, as shown here:

```
>>> from reviews.models import Contributor
>>> Contributor.objects.get(
    last_names='Wharton').delete()
(1, {'reviews.Contributor': 1})
```

Notice that we called the `delete()` method without assigning the contributor object to a variable. Since the `get()` method returns a single object, we can access the object's method without actually creating a variable for it.

2. Verify that the contributor object with `last_name` as `'Wharton'` has been deleted:

```
>>> Contributor.objects.get(last_names='Wharton')
Traceback (most recent call last):
File "<console>", line 1, in <module>
File "../site-packages/django/db/models/manager.py",
line 82, in manager_method
return getattr(self.get_queryset(), name)(*args,
**kwargs)
File "../site-packages/django/db/models/query.py",
line 417, in get
self.model._meta.object_name
reviews.models.Contributor.DoesNotExist: Contributor
matching query does not exist.
```

As you can see, upon running the query, we got an *object does not exist* error. This is expected since the record has been deleted. In this exercise, we learned how to use the `delete` method to delete a record from the database.

이번 연습에서는 `update()` 메서드를 사용하여 데이터베이스의 레코드를 업데이트하는 방법을 배웠습니다. 이제 마지막으로 `delete()` 메서드를 사용하여 데이터베이스에서 레코드를 삭제해 보겠습니다.

연습 2.13 – `delete()` 메서드 사용하기

`idx_0fa7791b` 데이터베이스의 기존 레코드는 `delete()` 메서드를 사용하여 삭제할 수 있습니다. 이번 연습에서는 `last_name` 값이 `Wharton`인 `contributors` 테이블의 레코드를 삭제할 것입니다:

1. `get` 메서드를 사용하여 객체를 가져오고, 여기에서 보여준 대로 `delete` 메서드를 사용합니다:

```
>>> from reviews.models import Contributor
>>> Contributor.objects.get(
    last_name='Wharton').delete()
(1, {'reviews.Contributor': 1})
```

`삭제()` 메서드를 변수에 기여자 객체를 할당하지 않고 호출했다는 점에 유의하세요. `get()` 메서드는 단일 객체를 반환하므로, 실제로 변수를 생성하지 않고도 객체의 메서드에 접근할 수 있습니다.

2. 성(`last_name`)이 `'Wharton'`인 기여자 객체가 삭제되었는지 확인하세요:

```
>>> Contributor.objects.get(last_name='Wharton')
Traceback (most recent call last):
파일 "<console>", 1행, <module>에서
파일 "../site-packages/django/db/models/manager.py",
82행, manager_method에서
return getattr(self.get_queryset(), name)(*args,
**kwargs)
파일 "../site-packages/django/db/models/query.py",
line 417, in get
self.model._meta.object_namereviews.models.Contribut
or.DoesNotExist: Contributormatching query does not
exist.
```

보시다시피, `idx_e211bf47` 쿼리를 실행할 때 객체가 존재하지 않는 오류가 발생했습니다. 이는 레코드가 삭제되었기 때문에 예상된 결과입니다. 이 연습을 통해 데이터베이스에서 레코드를 삭제하기 위해 `delete` 메서드를 사용하는 방법을 배웠습니다.

Bulk create and bulk update operations

When we have a large set of records that needs to be created or updated, we can perform bulk create and bulk update operations using the `bulk_create()` and `bulk_update()` methods, respectively. When we have a large number of records to be created or updated, using methods such as these can be efficient while performing database operations.

Typically, here is how a `bulk_create` method is called by supplying a list of objects:

```
Person.objects.bulk_create([
    Person(name='Robert',
          address='5 Falcon Street, Byron Bay, NSW 2481'),
    Person(name='Mark',
          address="31 West Street, Orange, NSW 2800"),])
```

Likewise, the bulk update is also performed on a list of similar objects. The object's attributes can be changed as shown in the following example and updated in the database in a single command using the `bulk_update()` method, as shown in the following snippet:

```
persons[0].address = "2 Church Street, Byron Bay, NSW 2481"
persons[1].address = "35 Steel Street, Orange, NSW 2800"
Person.objects.bulk_update(persons, ["address"])
```

We will see further examples of both of these in the following exercises.

Exercise 2.14 – Create multiple records using `bulk_create`

In this exercise, we will use the `bulk_create()` method to create multiple entries in the `Publisher` table in one go. To do that, follow these steps:

1. Import the `Publisher` model if you have not already imported it into your Django shell:

```
>>> from reviews.models import Publisher
```

2. Create multiple records in the `Publisher` table by passing a list of the `Publisher` objects into the `bulk_create()` method:

```
>>> Publisher.objects.bulk_create([
    Publisher(name="New Town Publisher",
             website="www.newtown.example.com",
             email='newtown@example.com'),
```

대량 생성 및 대량 업데이트 작업

생성하거나 업데이트해야 할 대량의 레코드가 있을 때, 각각 `bulk_create()` 및 `bulk_update()` 메서드를 사용하여 대량 생성 및 대량 업데이트 작업을 수행할 수 있습니다. 생성하거나 업데이트해야 할 레코드가 많을 때, 이러한 메서드를 사용하는 것이 데이터베이스 작업을 수행하는 데 효율적일 수 있습니다.

일반적으로, 다음은 객체 목록을 제공하여 `bulk_create` 메서드를 호출하는 방법입니다:

```
Person.objects.bulk_create([
    Person(name='Robert',
          address='5 Falcon Street, Byron Bay, NSW 2481'),
    Person(name='Mark',
          address="31 West Street, Orange, NSW 2800"),])
```

마찬가지로, 대량 업데이트는 유사한 객체 목록에서도 수행됩니다. 객체의 속성은 다음 예와 같이 변경될 수 있으며, `bulk_update()` 메서드를 사용하여 단일 명령으로 데이터베이스에 업데이트됩니다. 다음 코드 조각과 같이:

```
persons[0].address = "2 Church Street, Byron Bay, NSW 2481"
persons[1].address = "35 Steel Street, Orange, NSW 2800"
Person.objects.bulk_update(persons, ["address"])
```

다음 연습에서 이 두 가지의 추가 예를 볼 것입니다.

연습 2.14 – `bulk_create`를 사용하여 여러 레코드 만들기

이 연습에서는 `bulk_create()` 메서드를 사용하여 Publisher 테이블에 여러 항목을 한 번에 생성합니다. 이를 위해 다음 단계를 따르세요:

1. Django 셸에 Publisher 모델을 아직 가져오지 않았다면 가져옵니다:

```
>>> from reviews.models import Publisher
```

2. 게시자 테이블에 Publisher 객체 목록을 `bulk_create()` 메서드에 전달하여 여러 레코드를 생성합니다:

```
>>> Publisher.objects.bulk_create([
    Publisher(name="New Town Publisher",
              website="www.newtown.example.com",
              email='newtown@example.com'),
```

```
Publisher(name="Byron Bay Press",
website="www.byronbay.example.org",
email='byronbay@example.org'),
Publisher(name="Katoomba Publisher",
website="www.katoomba.example.net",
email='katoomba@example.net']])

[<Publisher: New Town Publisher>, <Publisher: Byron Bay Press>, <Publisher:
Katoomba Publisher>]
```

The method returns a list of objects created as entries in the database. If a large number of entries need to be created, this might be one of the most efficient ways of carrying out the task.

Next, we shall try to bulk update records efficiently using the `bulk_update` method.

Exercise 2.15 – Update multiple records using `bulk_update`

Similar to the previous exercise, we can update multiple records in one go by using the `bulk_update` method:

1. First, import the `Publisher` model if you have not already imported it:

```
>>> from reviews.models import Publisher
```

2. In the next step, we will pick a couple of `Publisher` objects as a list.
3. Assuming both of these publishers got merged into a single company, and they need to share the same website; now let us update both the objects in a query set:

```
>>> publishers = [
    Publisher.objects.get(name='New Town Publisher'),
    Publisher.objects.get(name='Byron Bay Press')]
>>> publishers[0].website = "nswpublisher.example.com"
>>> publishers[1].website = "nswpublisher.example.com"

>>> Publisher.objects.bulk_update(publishers,
    ["website"])
2
```

Upon running the `bulk_update` method, it returns the number of objects updated; in this case, it is two objects. Again, this is an efficient way to update more than one object in one go.

```
Publisher(name="Byron Bay Press",
website="www.byronbay.example.org",
email='byronbay@example.org'),
Publisher(name="Katoomba Publisher",
website="www.katoomba.example.net",
email='katoomba@example.net]])

[<Publisher: New Town Publisher>, <Publisher: Byron Bay Press>,
<Publisher:Katoomba Publisher>]
```

이 메서드는 데이터베이스에 항목으로 생성된 객체 목록을 반환합니다. 많은 수의 항목을 생성해야 하는 경우, 이는 작업을 수행하는 가장 효율적인 방법 중 하나일 수 있습니다.

다음으로, `bulk_update` 메서드를 사용하여 레코드를 효율적으로 일괄 업데이트해 보겠습니다.

연습 2.15 – `bulk_update`를 사용하여 여러 레코드 업데이트

이전 `idx_13d20ddd` 연습과 유사하게, `bulk_update` 메서드를 사용하여 한 번에 여러 레코드를 업데이트할 수 있습니다:

1. 먼저, `Publisher` 모델을 아직 가져오지 않았다면 가져옵니다:

```
>>> from reviews.models import Publisher
```

2. 다음 단계에서는 `Publisher` 객체 몇 개를 목록으로 선택할 것입니다.
3. 이 두 출판사가 하나의 회사로 합병되었고, 동일한 웹사이트를 공유해야 한다고 가정해 보겠습니다. 이제 쿼리 세트에서 두 객체를 업데이트해 보겠습니다:

```
>>> publishers = [
    Publisher.objects.get(name='New Town Publisher'),
    Publisher.objects.get(name='Byron Bay Press')]
>>> publishers[0].website = "nswpublisher.example.com"
>>> publishers[1].website = "nswpublisher.example.com"

>>> Publisher.objects.bulk_update(publishers,
    ["website"])
```

2

`bulk_update` 메서드를 실행하면 `itidx_c6c959bf`는 업데이트된 객체의 수를 반환합니다. 이 경우 두 개의 객체입니다. 다시 말해, 이는 한 번에 여러 객체를 업데이트하는 효율적인 방법입니다.

Performing complex lookups using Q objects

Q objects are used to perform complex queries, especially when a query involves the AND or OR operations in a WHERE clause. For instance, if we need to do a query similar to the SQL query shown here:

```
SELECT * FROM Person
WHERE name LIKE 'Rob%' OR name LIKE 'Bob%';
```

The preceding SQL statement queries for any person whose name either starts with Rob or Bob.

LIKE 'Rob%' – Here, the LIKE keyword is pattern-matching a string to check whether the string starts with the specified Rob value.

Q objects use the & and | operators for the AND and OR operations while combining the WHERE clauses. The preceding query can be written as follows using Q objects:

```
Person.objects.get(Q(name__startswith='Rob'), | Q(name__startswith='Bob'))
```

Exercise 2.16 – Perform a complex query using a Q object

Next, using the concept learned before, Q objects let us perform a query for two of the publishers:

1. Import Publisher if you have not already done so before, and also import Q from `django.db.models`, as shown in the following command:

```
>>> from django.db.models import Q
>>> from reviews.models import Publisher
```

2. Execute the Q object query as shown next. Here, we check whether any of the Publisher objects has name starting with either New or Idea:

```
>>> Publisher.objects.filter(Q(name__startswith="New")
    | Q(name__startswith="Idea"))
<QuerySet [Publisher: New Town Publisher]>>
```

Since there is no publisher with name starting with Idea and there was an object with name starting with New, one object was returned.

Q 객체를 사용하여 복잡한 조회 수행하기

Q 객체는 특히 WHERE 절에서 AND 또는 OR 연산이 포함된 쿼리를 수행하는 데 사용됩니다. 예를 들어, 다음과 같은 SQL 쿼리와 유사한 쿼리를 수행해야 하는 경우:

```
SELECT * FROM Person
WHERE name LIKE 'Rob%' OR name LIKE 'Bob%';
```

앞의 SQL 문은 이름이 Rob 또는 Bob으로 시작하는 모든 사람을 조회합니다.

LIKE 'Rob%' – 여기서 LIKE 키워드는 문자열이 지정된 Rob 값으로 시작하는지 확인하기 위해 문자열을 패턴 매칭합니다.

Q 객체는 WHERE 절을 결합할 때 AND 및 OR 연산을 위해 & 및 | 연산자를 사용합니다. 앞의 쿼리는 다음과 같이 Q 객체를 사용하여 작성할 수 있습니다:

```
Person.objects.get(Q(name__startswith='Rob'), | Q(name__startswith='Bob'))
```

연습 2.16 – Q 객체를 사용하여 복잡한 쿼리 수행하기

다음으로, 이전에 배운 conceptidx_abbe5913을 사용하여 Q 객체를 통해 두 개의 publishers에 대한 쿼리를 수행할 수 있습니다:

1. Publisher를 가져오고 idx_3499a6b8가 아직 가져오지 않았다면, 다음 명령어와 같이 django.db.models에서 Q를 가져옵니다:

```
>>> from django.db.models import Q
>>> from reviews.models import Publisher
```

2. 다음과 같이 Q 객체 쿼리를 실행합니다. 여기서 우리는 Publisher 객체 중 이름이 New 또는 Idea로 시작하는 것이 있는지 확인합니다:

```
>>> Publisher.objects.filter(Q(name__startswith="New")
    | Q(name__startswith="Idea"))
<QuerySet [Publisher: New Town Publisher>]>
```

아이디어로 시작하는 이름의 출판사가 없고, 뉴로 시작하는 이름의 객체가 있었기 때문에 하나의 객체가 반환되었습니다.

This is an example Q object query with the AND operator. In this query, we are querying a Publisher object whose name starts with New and also whose name ends with Publisher:

```
>>> Publisher.objects.filter(Q(name__startswith="New") &
                             Q(name__endswith="Publisher"))
<QuerySet [<Publisher: New Town Publisher>]>
```

In this case, only one object was returned, which is New Town Publisher, which satisfies the conditions stated in the query. This way, multiple such where clauses can be combined using the AND or OR operators to get the desired results.

Exercise 2.17 – Verify whether a queryset contains a given object

In this exercise, we will use a contains method to check whether the query contains a specified object:

1. Import the Publisher model if you have not already imported it:

```
>>> from reviews.models import Publisher
```

2. Run a query, as shown next. In this example, we are executing a Q object query, and the result has two publishers:

```
>>> publishers = Publisher.objects.filter(
    Q(name__startswith="New")
    | Q(name__endswith="Publisher"))
>>> publishers
<QuerySet [<Publisher: New Town Publisher>, <Publisher: Katoomba
Publisher>]>
```

3. Fetch a new publisher object and verify whether it is a part of the previous query set:

```
>>> new_town_publisher = Publisher.objects.get(
    name='New Town Publisher')
>>> new_town_publisher
<Publisher: New Town Publisher>
>>> publishers.contains(new_town_publisher)
True
```

If the object is present, it returns a Boolean value such as True or False.

이것은 AND 연산자를 사용한 Q 객체 쿼리의 예입니다. 이 쿼리에서는 이름이 New로 시작하고 이름이 Publisher로 끝나는 aPublisher 객체를 쿼리하고 있습니다:

```
>>> Publisher.objects.filter(Q(name__startswith="New") &
    Q(name__endswith="Publisher"))<QuerySet
[<Publisher: New Town Publisher>]>
```

이 경우, 쿼리에서 명시된 조건을 만족하는 New Town Publisher라는 하나의 객체만 반환되었습니다. 이렇게 여러 개의 그러한 where 절을 AND 또는 OR 연산자를 사용하여 결합하여 원하는 결과를 얻을 수 있습니다.

연습 2.17 – 쿼리셋에 주어진 객체가 포함되어 있는지 확인하기

이 연습에서는 contains 메서드를 사용하여 쿼리에 지정된 객체가 포함되어 있는지 확인할 것입니다:

1. 이미 Publisher 모델을 가져오지 않았다면 가져옵니다:

```
>>> from reviews.models import Publisher
```

2. 다음과 같이 쿼리를 실행합니다. 이 예제에서는 Q 객체 쿼리를 실행하고 있으며, 결과에는 두 개의 퍼블리셔가 있습니다:

```
>>> publishers = Publisher.objects.filter(
    Q(name__startswith="New")
    | Q(name__endswith="Publisher"))
>>> publishers
<QuerySet [<Publisher: New Town Publisher>, <Publisher:
KatoombaPublisher>]>
```

3. 새 퍼블리셔 객체 idx_82d5eba1을 가져오고 이전 쿼리 세트의 일부인지 확인합니다:

```
>>> new_town_publisher = Publisher.objects.get(
    name='New Town Publisher')
>>> new_town_publisher
<Publisher: New Town Publisher>>>>
publishers.contains(new_town_publisher)
True
```

객체가 존재하면 True 또는 False와 같은 부울 값을 반환합니다.

Here is an example when a certain query set does not have the specified object, in which case the result returned is `False`:

```
>>> publishers.contains(Publisher.objects.get(
    name='Byron Bay Press'))
False
```

In this exercise, we learned how a simple method, `contains()`, can be used to check whether an object is part of a query set.

Overall, in this entire section, we learned about and used Django's command-line interactive shell. Using the models created in the previous section for the book review application, we used the Django interactive shell to perform CRUD operations. We also explored various ways to filter and query (Read) from the database. All of these database operations will come in handy while developing any Django application. Using the concepts learned so far in this chapter, in the next section, we will create models for a sample application and perform some of the CRUD operations.

Activity 2.01 – Create models for a project management application

Imagine we are developing a project management application called Juggler. Juggler is an application that can track multiple projects, and each project can have multiple tasks associated with it. The following steps will help us complete this activity:

1. Using the techniques we have learned so far, create a Django project called `juggler`.
2. Create a Django app called `projectp`.
3. Add the app projects in the `juggler/settings.py` file.
4. Create two related model classes called `Project` and `Task` in `projectp/models.py`.
5. Create migration scripts and migrate the models' definitions to the database.
6. Open the Django shell and import the models.
7. Populate the database with an example and write a query displaying the list of tasks associated with a given project.

Note

The full solution for this activity can be found at in the `Chapter02` folder in this book's GitHub repo.

특정 쿼리 집합에 지정된 객체가 없을 때의 예입니다. 이 경우 반환되는 결과는 False 입니다:

```
>>> publishers.contains(Publisher.objects.get(
    name='Byron Bay Press'))

False
```

이 연습에서는 간단한 메서드인 `contains()`를 사용하여 객체가 쿼리 집합의 일부인지 확인하는 방법을 배웠습니다.

전반적으로 이 섹션 전체에서 우리는 Django의 명령줄 대화형 셸에 대해 배우고 사용했습니다. 책 리뷰 애플리케이션을 위해 이전 섹션에서 생성된 모델을 사용하여 Django 대화형 셸을 통해 CRUD 작업을 수행했습니다. 또한 데이터베이스에서 필터링하고 쿼리(Read)하는 다양한 방법을 탐색했습니다. 이러한 모든 데이터베이스 작업은 Django 애플리케이션을 개발하는 데 유용할 것입니다. 이 장에서 배운 개념을 사용하여 다음 섹션에서는 샘플 애플리케이션을 위한 모델을 만들고 일부 CRUD 작업을 수행할 것입니다.

활동 2.01 – 프로젝트 관리 애플리케이션을 위한 모델 만들기

우리가 Juggler라는 프로젝트 관리 애플리케이션을 개발하고 있다고 상상해 보십시오. Juggler는 여러 프로젝트를 추적할 수 있는 애플리케이션이며, 각 프로젝트는 여러 작업과 연결될 수 있습니다. 다음 단계는 이 활동을 완료하는 데 도움이 될 것입니다:

1. 지금까지 배운 기술을 사용하여 juggler라는 Django 프로젝트를 생성합니다.
2. `projectp`라는 Django 앱을 만듭니다.
3. `juggler/settings.py` 파일에 `projects` 앱을 추가합니다.
4. `projectp/models.py`에 `Project`와 `Task`라는 두 개의 관련 모델 클래스를 만듭니다.
5. 마이그레이션 스크립트를 생성하고 모델 정의를 데이터베이스로 마이그레이션합니다.
6. Django 셸을 열고 모델을 가져옵니다.
7. 주어진 프로젝트와 관련된 작업 목록을 표시하는 쿼리를 작성하고 예제로 데이터베이스를 채우십시오.

참고

이 활동에 대한 전체 솔루션은 이 책의 GitHub 리포지토리의 Chapter02 폴더에서 찾을 수 있습니다.

Populating the Bookr project's database

Although we know how to create individual database records, it will be good to populate the Bookr database with a dataset that exemplifies real-world publications. We have included a JSON file that can be used to load publication data and reviews into the database using the `loaddata` subcommand of the `manage.py` script.

Note

You will find the `reviews.json` file in the `Chapter02` folder in this book's GitHub repo.

Follow the next few steps to populate the project's database:

1. Copy the `reviews.json` file into the `reviews` app folder. This file can be found in the GitHub repository for this book.
2. Now, let's recreate a fresh database. Delete your SQL database file present in the project folder:

```
rm db.sqlite3
```

3. To create a fresh database with the correct schema, execute the Django migrate command:

```
python manage.py migrate
```

Now we can see the newly created `db.sqlite3` file under the `bookr` folder.

4. Execute the `loaddata` management command to populate the database:

```
python manage.py loaddata reviews/reviews.json
```

5. Using DB Browser for SQLite, verify that all the tables created by the `bookr` project are populated.

With this, we have successfully populated our application's database, which will come in handy to work with in the upcoming chapters of this book.

Bookr 프로젝트의 데이터베이스 채우기

개별 데이터베이스 레코드를 생성하는 방법을 알고 있지만, 실제 출판물을 예시로 하는 데이터 세트로 Bookr 데이터베이스를 채우는 것이 좋습니다. publication 데이터와 리뷰를 데이터베이스에 로드하는 데 사용할 수 있는 JSON 파일을 포함했습니다. manage.py 스크립트의 loaddata 서버커맨드를 사용하여 idx_2ccd6827 로드할 수 있습니다.

참고

이 책의 GitHub 리포지토리의 Chapter02 폴더에서 reviews.json 파일을 찾을 수 있습니다.

프로젝트의 데이터베이스를 채우기 위해 다음 몇 가지 단계를 따르세요:

1. reviews.json 파일을 reviews 앱 폴더에 복사하세요. 이 파일은 이 책의 GitHub 저장소에서 찾을 수 있습니다.
2. 이제 새 데이터베이스를 다시 만들어 보겠습니다. 프로젝트 폴더에 있는 SQL 데이터베이스 파일을 삭제하세요:

```
rm db.sqlite3
```

3. 올바른 스키마로 새로운 idx_920a7dab 데이터베이스를 생성하려면 Django migrate를 실행하세요.

명령:

```
python manage.py migrate
```

이제 bookr 폴더 아래에 새로 생성된 db.sqlite3 파일이 보입니다. 데

4. 데이터베이스를 채우기 위해 loaddata 관리 명령어를 실행하세요:

```
python manage.py loaddata reviews/reviews.json
```

5. DB Browser for SQLite를 사용하여 bookr 프로젝트에서 생성된 모든 테이블이 데이터로 채워져 있는지 확인하세요.

이로써 우리는 애플리케이션의 데이터베이스를 성공적으로 채웠으며, 이는 이 책의 다음 장에서 작업하는 데 유용할 것입니다.

Summary

In this chapter, we learned how Django provides a valuable abstraction layer called the ORM that helps us interact seamlessly with relational databases using simple Python code without having to compose SQL commands. As a part of the ORM, we learned about Django models, migrations, and how they help propagate changes to Django models in a database.

We shored up our knowledge of databases by learning about database relationships and their key types in relational databases. We also worked with the Django shell, using Python code to perform the same CRUD queries we performed earlier using SQL. Later, we learned how to retrieve our data in a more refined manner using pattern matching and field lookups. As we learned about these concepts, we also made considerable progress on our Bookr application. We created models for our reviews app and gained all the skills we needed to interact with the data stored inside the app's database. In the next chapter, we will learn how to create Django views, URL routing, and templates.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

요약

이 장에서는 Django가 SQL 명령을 작성하지 않고도 간단한 Python 코드를 사용하여 관계형 데이터베이스와 원활하게 상호작용할 수 있도록 도와주는 ORM이라는 귀중한 추상화 계층을 제공하는 방법을 배웠습니다. ORM의 일환으로, 우리는 Django 모델, 마이그레이션 및 데이터베이스에서 Django 모델에 대한 변경 사항을 전파하는 방법에 대해 배웠습니다.

우리는 데이터베이스 관계와 관계형 데이터베이스의 주요 유형에 대해 배우면서 데이터베이스에 대한 지식을 강화했습니다. 또한 Django 셸을 사용하여 이전에 SQL을 사용하여 수행했던 동일한 CRUD 쿼리를 수행하기 위해 Python 코드를 사용했습니다. 이후에는 패턴 매칭 및 필드 조회를 사용하여 데이터를 보다 정교하게 검색하는 방법을 배웠습니다. 이러한 개념을 배우면서 우리는 Bookr 애플리케이션에서도 상당한 진전을 이루었습니다. 우리는 리뷰 앱을 위한 모델을 만들었고 앱의 데이터베이스에 저장된 데이터와 상호작용하는 데 필요한 모든 기술을 습득했습니다. 다음 장에서는 Django 뷰, URL 라우팅 및 템플릿을 만드는 방법을 배울 것입니다.

이 책의 PDF 사본, 코드 번들 등을 받으세요

QR 코드를 스캔하거나 packtpub.com/unlock으로 이동하세요. 이 책의 이름으로 검색하고, 판을 확인한 후 페이지의 지침을 따르세요.



UNLOCK NOW

참고: 송장 사본을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

3

Django Views, URL Configuration, and Templates

In *Chapter 2*, we were introduced to databases, and we learned how to store, retrieve, update, and delete records from a database. We also learned how to create Django models and apply database migrations.

However, these database operations alone cannot display the application's data to a user. We need a way to display all the stored information in a meaningful way to the user – for example, displaying all the books present in our Bookr application's database, in a browser and in a presentable format. This is where Django views, URL configuration, and templates come into play. Views are one of the most important parts of a Django application, where the application logic is written. This application logic controls interactions with the database, such as creating, reading, updating, or deleting records from the database. Server requests are routed to views using URL mapping. Views also control how the data can be displayed to the user. This is done with the help of Django HTML templates.

You will start by exploring the two main types of views in Django: function-based views and class-based views. Next, you will learn the basics of Django template language and template inheritance. Using these concepts, you will create a page to display the list of all the books in the Bookr application. You will also create another page to display the details, review comments, and ratings of books.

In this chapter, we will be studying the following topics:

- Working with Django views
- URL configuration
- Working with Django templates

3

장고 뷰, URL 구성 및 템플릿

2장에서는 데이터베이스에 대해 소개하고, 데이터베이스에서 레코드를 저장, 검색, 업데이트 및 삭제하는 방법을 배웠습니다. 또한 장고 모델을 생성하고 데이터베이스 마이그레이션을 적용하는 방법도 배웠습니다.

그러나 이러한 데이터베이스 작업만으로는 애플리케이션의 데이터를 사용자에게 표시할 수 없습니다. 우리는 사용자가 의미 있는 방식으로 저장된 모든 정보를 표시할 수 있는 방법이 필요합니다. 예를 들어, 브라우저에서 Bookr 애플리케이션의 데이터베이스에 있는 모든 책을 표시하는 것입니다. 여기서 장고 뷰, URL 구성 및 템플릿이 작용합니다. 뷰는 장고 애플리케이션의 가장 중요한 부분 중 하나로, 애플리케이션 로직이 작성되는 곳입니다. 이 애플리케이션 로직은 데이터베이스와의 상호작용을 제어하며, 데이터베이스에서 레코드를 생성, 읽기, 업데이트 또는 삭제하는 작업을 포함합니다. 서버 요청은 URL 매핑을 사용하여 뷰로 라우팅됩니다. 뷰는 또한 데이터가 사용자에게 어떻게 표시될 수 있는지를 제어합니다. 이는 장고 HTML 템플릿의 도움으로 이루어집니다.

장고에서 두 가지 주요 유형의 뷰인 함수 기반 뷰와 클래스 기반 뷰를 탐색하는 것으로 시작합니다. 다음으로, 장고 템플릿 언어와 템플릿 상속의 기초를 배울 것입니다. 이러한 개념을 사용하여 Bookr 애플리케이션의 모든 책 목록을 표시하는 페이지를 생성할 것입니다. 또한 책의 세부 정보, 리뷰 댓글 및 평점을 표시하는 또 다른 페이지를 생성할 것입니다.

이번 장에서는 다음 주제를 공부할 것입니다:

- 장고 뷰 작업 URL 구성
-
- 장고 템플릿 작업

By using and learning about these topics, by the end of this chapter, you will have implemented a way to list all the books and display details of any given book in the **Bookr** application.

Technical requirements

Find the solution in the `Chapter03` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Working with Django views

Views are the part of a Django application that takes in a web request, processes it and provides a web response. For example, a user clicks on a **Product details** link:

The browser sends a request to a specific URL (e.g. `/products/12/`), which Django maps to a view.

The view goes to the database to fetch product 12 and prepares data from that object to be used in the view's template. This data is referred to as context data and is used alongside the template to create HTML for the response.

The response takes the rendered template to display the HTML in the user's browser. Django views can be broadly classified into two types: **function-based views** and **class-based views**. Let's take a look at each in detail.

Note

In this chapter, we will learn more about function-based views in Django. Class-based views, which are a more advanced topic, will be discussed in detail in *Chapter 11*. Asynchronous views will be discussed in *Chapter 17*.

Understanding function-based views

As the name implies, function-based views are implemented as *Python functions*. To understand how they work, consider the following snippet, which shows a simple view function named `home_page`:

```
from django.http import HttpResponseRedirect

def home_page(request):
    message = "<html><h1>Welcome to my Website</h1></html>"
    return HttpResponseRedirect(message)
```

이 장이 끝날 무렵에는 이러한 주제를 사용하고 배우면서 Bookr 애플리케이션에서 모든 책을 나열하고 주어진 책의 세부 정보를 표시하는 방법을 구현하게 될 것입니다.

기술 요구 사항

이 책의 GitHub 리포지토리의 Chapter03 폴더에서 솔루션을 찾으십시오. 리포지토리 링크에 접근하려면 서문에 있는 예제 코드 파일 다운로드 섹션의 단계를 따르십시오.

Django 뷰 작업하기

뷰는 웹 요청을 받아들이고 처리하여 웹 응답을 제공하는 Django 애플리케이션의 `idx_c8c0c675` 부분입니다. 예를 들어, 사용자가 제품 세부 정보 링크를 클릭합니다:

브라우저는 특정 URL(예: `/products/12/`)에 요청을 보내고, Django는 이를 뷰에 매핑합니다.

뷰는 데이터베이스에 가서 제품 12를 가져오고 해당 객체에서 데이터를 준비하여 뷰의 템플릿에서 사용합니다. 이 데이터는 컨텍스트 데이터라고 하며, 응답을 위한 HTML을 생성하기 위해 템플릿과 함께 사용됩니다.

응답은 렌더링된 템플릿을 사용하여 사용자의 브라우저에 HTML을 표시합니다. Django 뷰는 `idx_04d46af2` `broadly` `idx_e8367c79` 두 가지 `idx_07e2e163` 유형으로 분류될 수 있습니다: 함수 기반 뷰와 클래스 기반 뷰입니다. 각 유형을 자세히 살펴보겠습니다.

메모

이 장에서는 Django의 함수 기반 뷰에 대해 더 배울 것입니다. 클래스 기반 뷰는 더 고급 주제로, 11장에서 자세히 논의될 것입니다. 비동기 뷰는 17장에서 논의될 것입니다.

함수 기반 뷰 이해하기

이름에서 알 수 있듯이, 함수 기반 `idx_c399dcd4` 뷰는 Python 함수로 구현됩니다. 이들이 어떻게 작동하는지 이해하기 위해, 다음 스니펫을 고려해 보십시오. 이 스니펫은 `home_page`라는 간단한 뷰 함수를 보여줍니다:

```
from django.http import HttpResponse

def home_page(request):
    message = "<html><h1>내 웹사이트에 오신 것을 환영합니다</h1></html>"
    return HttpResponse(message)
```

The view function defined here, named `home_page`, takes a request object as an argument and returns an `HttpResponse` object, having the `Welcome to my Website` message. The advantage of using function-based views is that, since they are implemented as simple Python functions, they are easier to learn and also easily readable for other programmers. The major disadvantage of function-based views is that the code cannot be reused and made as concisely as class-based views for generic use cases. The next section is a brief introduction to class-based views.

Understanding class-based views

As the name implies, class-based views are implemented as *Python classes*. Using the principles of class inheritance, these classes are implemented as subclasses of Django's generic view classes. Unlike function-based views, where all the view logic is expressed explicitly in a function, Django's generic view classes come with various pre-built properties and methods that can provide shortcuts to writing clean, reusable views. This property comes in handy quite often during web development; for example, developers often need to render an HTML page without needing any data inserted from the database or any customization specific to the user. In this case, it is possible to simply inherit from Django's `TemplateView` and specify the path of the HTML file. The following is an example of a class-based view that can display the same message as in the function-based view example:

```
from django.views.generic import TemplateView

class HomePage(TemplateView):
    template_name = 'home_page.html'
```

In the preceding code snippet, `HomePage` is a class-based view inheriting Django's `TemplateView` from the `django.views.generic` module. The `template_name` class attribute defines the template to render when the view is invoked. For the template, we add an HTML file to our templates folder with the following content:

```
<html><h1>Welcome to my Website</h1></html>
```

This is a very basic example of class-based views, which will be explored further in *Chapter 11*. The major advantage of using class-based views is that fewer lines of code need to be used to implement the same functionality as compared to function-based views. Also, by inheriting Django's generic views, we can keep the code concise and avoid the duplication of code. However, a disadvantage of class-based views is that the code is often less readable for someone new to Django, which means that learning about these is usually a longer process compared to function-based views.

여기 정의된 뷰 함수인 `home_page`는 요청 객체를 인수로 받아들이고, 내 웹사이트에 오신 것을 환영합니다 메시지를 가진 `HttpResponse` 객체를 반환합니다. 함수 기반 뷰를 사용하는 장점은 간단한 파이썬 함수로 구현되기 때문에 배우기 쉽고 다른 프로그래머들이 읽기에도 용이하다는 것입니다. 함수 기반 뷰의 주요 단점은 코드가 재사용될 수 없고 일반적인 사용 사례에 대해 클래스 기반 뷰만큼 간결하게 만들 수 없다는 것입니다. 다음 섹션은 클래스 기반 뷰에 대한 간략한 소개입니다.

클래스 기반 뷰 이해하기

이름에서 알 수 있듯이, 클래스 기반 뷰는 파이썬 클래스로 구현됩니다. 클래스 상속의 원칙을 사용하여, 이러한 클래스는 장고의 일반 뷰 클래스의 하위 클래스로 구현됩니다. 모든 뷰 로직이 함수 내에서 명시적으로 표현되는 함수 기반 뷰와 달리, 장고의 일반 뷰 클래스는 깨끗하고 재사용 가능한 뷰를 작성하는 데 도움이 되는 다양한 미리 구축된 속성과 메서드를 제공합니다. 이 속성은 웹 개발 중에 자주 유용하게 사용됩니다. 예를 들어, 개발자는 종종 데이터베이스에서 삽입된 데이터나 사용자에 대한 특정 사용자 정의 없이 HTML 페이지를 렌더링해야 합니다. 이 경우, 장고의 `TemplateView`에서 단순히 상속받고 HTML 파일의 경로를 지정하는 것이 가능합니다. 다음은 함수 기반 뷰 예제와 동일한 메시지를 표시할 수 있는 클래스 기반 뷰의 예입니다:

```
from django.views.generic import TemplateView

class HomePage(TemplateView):
    template_name = 'home_page.html'
```

앞의 코드 조각에서 `HomePage`는 `django.views.generic` 모듈의 Django의 `TemplateView`를 상속받는 클래스 기반 뷰입니다. `template_name` 클래스 속성은 뷰가 호출될 때 렌더링할 템플릿을 정의합니다. 템플릿을 위해, 우리는 다음 내용을 가진 HTML 파일을 템플릿 폴더에 추가합니다:

```
<html><h1>내 웹사이트에 오신 것을 환영합니다</h1></html>
```

이것은 클래스 기반 뷰의 매우 기본적인 예로, 11장에서 더 자세히 탐구할 것입니다. 클래스 기반 뷰를 사용하는 주요 장점은 함수 기반 뷰에 비해 동일한 기능을 구현하는 데 필요한 코드 줄 수가 적다는 것입니다. 또한, Django의 일반 뷰를 상속함으로써 코드를 간결하게 유지하고 코드 중복을 피할 수 있습니다. 그러나 클래스 기반 뷰의 단점은 Django에 익숙하지 않은 사람에게는 코드가 종종 덜 읽기 쉽다는 점입니다. 이는 이러한 내용을 배우는 과정이 일반적으로 함수 기반 뷰에 비해 더 긴 과정을 의미합니다.

In the next section, we will learn about URL configuration.

URL configuration

Django views cannot work on their own in a web application. When a web request is made to the application, Django's **URL configuration** takes care of routing the request to the appropriate view function to process the request. A typical URL configuration in the `urls.py` file in Django looks like this:

```
from . import views

urlpatterns = [
    path('url-path/' views.my_view, name='my-view'),
]
```

Here, `urlpatterns` is the variable defining the list of URL paths, and `'url-path/'` defines the path to match.

`views.my_view` is the view function to invoke when there is a URL match, and `name='my-view'` is the name of the view function used to refer to the view. There may be a situation wherein, elsewhere in the application, we want to get the URL of this view. We wouldn't want to hardcode the value, as it would then have to be specified twice in the code base. Instead, we can access the URL by using the name of the view, as follows:

```
from django.urls import reverse

url = reverse('my-view')
```

If needed, we can also use a regular expression in a URL path to match string patterns using `re_path()`:

```
urlpatterns = [
    re_path(r'^url-path/(?P<name>pattern)/$',
           views.my_view, name='my-view')
]
```

다음 섹션에서는 URL 구성에 대해 배울 것입니다.

URL 구성

Django 뷰는 웹 애플리케이션에서 독립적으로 작동할 수 없습니다. 웹 요청이 애플리케이션에 이루어지면 Django의 URL 구성이 요청을 처리할 적절한 뷰 함수로 라우팅하는 역할을 합니다. Django의 `urls.py` 파일에서 일반적인 URL 구성은 다음과 같습니다:

```
from . import views

urlpatterns = [
    path('url-path/', views.my_view, name='my-view'),
]
```

여기서 `urlpatterns`는 URL 경로 목록을 정의하는 변수이며, `'url-path/'`는 일치할 경로를 정의합니다.

`views.my_view`는 URL이 일치할 때 호출되는 뷰 함수이며, `name='my-view'`는 뷰를 참조하는 데 사용되는 뷰 함수의 이름입니다. 애플리케이션의 다른 곳에서 이 뷰의 URL을 가져오고 싶을 수 있는 상황이 있을 수 있습니다. 값을 하드코딩하고 싶지 않으므로, 코드베이스에서 두 번 지정해야 합니다. 대신, 다음과 같이 뷰의 이름을 사용하여 URL에 접근할 수 있습니다:

```
from django.urls import reverse

url = reverse('my-view')
```

필요한 경우, `re_path()`를 사용하여 문자열 패턴과 일치시키기 위해 URL 경로에 정규 표현식을 사용할 수도 있습니다:

```
urlpatterns = [
    re_path(r'^url-path/(?P<name>pattern)/$',
           views.my_view, name='my-view')
]
```


Here, `name` refers to the pattern name, which can be any Python regular expression pattern, and this needs to be matched before calling the defined view function. You can also pass parameters from the URL into the view itself, as shown in this example:

```
urlpatterns = [  
    path('url-path/<int:id>/', views.my_view,  
        name='my-view')  
]
```

In the preceding example, `<int:id>` tells Django to look for URLs that contain an integer at this position in the string, and to assign the value of that integer to the `id` argument. This means that if the user navigates to `/url-path/14/`, the `id=14` keyword argument is passed to the view. This is often useful when a view needs to look up a specific object in the database and return corresponding data. For example, suppose we had a `User` model, and we wanted the view to display the user's name. The view could be written as follows:

```
def my_view(request, id):  
    user = User.objects.get(id=id)  
    return HttpResponse(f"This user's name is { user.first_name }  
    { user.last_name }")
```

When the user accesses `/url-path/14/`, `my_view` is called, and the `id=14` argument is passed into the function.

Here is the typical workflow when a URL such as `http://0.0.0.0:8000/url-path/` is invoked using a web browser:

1. An HTTP request would be made to the running application for the URL path. Upon receiving the request, the server reaches for the `ROOT_URLCONF` setting present in the `settings.py` file:

```
ROOT_URLCONF = 'project_name.urls'
```

This determines the URL configuration file that will be used first. In this case, it is the URL file present in the project directory, `project_name/urls.py`.

2. Next, Django goes through the list named `urlpatterns`, and once it matches `url-path/` with the path present in the URL, `http://0.0.0.0:8000/url-path/`, it invokes the corresponding view function.

여기서 name은 패턴 이름을 나타내며, 이는 임의의 Python 정규 표현식 패턴일 수 있으며, 정의된 뷰 함수를 호출하기 전에 일치해야 합니다. 또한 이 예제에서 보여주듯이 URL에서 뷰 자체로 매개변수를 전달할 수 있습니다:

```
urlpatterns = [
    path('url-path/<int:id>/', views.my_view,
         name='my-view')
]
```

앞의 예에서 <int:id>는 장고에게 문자열의 이 위치에 정수가 포함된 URL을 찾고, 그 정수의 값을 id 인수에 할당하라고 지시합니다. 이는 사용자가 /url-path/14/로 이동할 경우 id=14 키워드 인수가 뷰에 전달된다는 것을 의미합니다. 이는 뷰가 데이터베이스에서 특정 객체를 조회하고 해당 데이터를 반환해야 할 때 유용합니다. 예를 들어, User 모델이 있다고 가정하고 뷰가 사용자의 이름을 표시하도록 하려면 뷰를 다음과 같이 작성할 수 있습니다:

```
def my_view(request, id):
    user = User.objects.get(id=id)
    return HttpResponse(f"이 사용자의 이름은 { user.first_name } 입니다")
    { user.last_name }")
```

사용자가 /url-path/14/에 접근하면, my_view가 호출되고 id=14 인수가 함수에 전달됩니다.

웹 브라우저를 사용하여 http://0.0.0.0:8000/url-path/와 같은 URL이 호출될 때의 일반적인 작업 흐름은 다음과 같습니다:

1. 실행 중인 애플리케이션에 URL 경로에 대한 HTTP 요청이 이루어집니다. 요청을 받으면 서버는 settings.py 파일에 있는 ROOT_URLCONF 설정을 참조합니다:

```
ROOT_URLCONF = 'project_name.urls'
```

이것은 먼저 사용될 URL 구성 파일을 결정합니다. 이 경우, 프로젝트 디렉토리인 project_name/urls.py에 있는 URL 파일입니다.

2. 다음으로, Django의 b4b3fcbd는 urlpatterns라는 리스트를 순회하며, URL에 존재하는 경로인 http://0.0.0.0:8000/url-path/와 url-path/가 일치하면 해당 뷰 함수를 호출합니다.

URL configuration is sometimes also referred to as **URL conf** or **URL mapping**, and these terms are often used interchangeably. Overall, we can say that URL configuration is present to route the URL request posted on a browser to an appropriate view function. To understand views and URL configuration better, let's start with a simple exercise.

Exercise 3.01 – Implementing a simple function-based view

In this exercise, we will write a very basic function-based view and use the associated URL configuration to display the `Welcome to Bookr!` message in a web browser. We will also tell the user how many books we have in the database. Let's get started with the steps:

1. First, ensure that `ROOT_URLCONF` in `bookr/settings.py` is pointing to the project's URL file by adding the following command:

```
ROOT_URLCONF = 'bookr.urls'
```

2. Open the `bookr/reviews/views.py` file and add the following code snippet:

```
from django.http import HttpResponse
from .models import Book

def home(request):
    message = f"""<html><h1>Welcome to Bookr!</h1>
    <p>{Book.objects.count()} books and counting!</p></html>"""
    return HttpResponse(message)
```

In the preceding snippet, we first import the `HttpResponse` class from the `django.http` module. Next, we define the `home` function, which can display the `Welcome to Bookr!` message in a web browser. The `request` object is a function parameter that carries the HTTP request object. The next line defines the `message` variable, which contains HTML that displays the header, followed by a line that counts the number of books available in the database.

In the last line, we return an `HttpResponse` object with the string associated with the `message` variable. When the `home` view function is called, it will display the message `Welcome to Bookr! 2 Books and counting!` in the web browser.

URL 구성은 때때로 URL conf 또는 URL 매핑이라고도 하며, 이러한 용어는 종종 서로 바꿔 사용됩니다. 전반적으로, 우리는 URL 구성이 브라우저에 게시된 URL 요청을 적절한 뷰 함수로 라우팅하기 위해 존재한다고 말할 수 있습니다. 뷰와 URL 구성을 더 잘 이해하기 위해 간단한 연습부터 시작해 보겠습니다.

연습 3.01 – 간단한 함수 기반 뷰 구현

이 연습에서는 매우 기본적인 함수 기반 뷰를 작성하고 관련된 URL 구성을 사용하여 웹 브라우저에 'Bookr에 오신 것을 환영합니다!' 메시지를 표시합니다. 또한 데이터베이스에 몇 권의 책이 있는지 사용자에게 알려줄 것입니다. 단계별로 시작해 보겠습니다:

1. 먼저, bookr/settings.py의 ROOT_URLCONF가 프로젝트의 URL 파일을 가리키고 있는지 확인하고 다음 명령을 추가합니다:

```
ROOT_URLCONF = 'bookr.urls'
```

2. bookr/reviews/views.py 파일을 열고 다음 코드 스니펫을 추가합니다:

```
from django.http import HttpResponse
from .models import Book

def home(request):
    message = f"""<html><h1>Bookr에 오신 것을 환영합니다!</h1>
    <p>{Book.objects.count()} 권의 책이 있습니다!</p></html>"""
    return HttpResponse(message)
```

위의 코드 조각에서는 먼저 django.httpmodule에서 HttpResponse 클래스를 가져옵니다. 다음으로, 웹 브라우저에 'Bookr에 오신 것을 환영합니다!' 메시지를 표시할 수 있는 home 함수를 정의합니다. request 객체는 HTTP 요청 객체를 전달하는 함수 매개변수입니다. 다음 줄에서는 데이터베이스에 있는 책의 수를 세는 HTML 헤더를 표시하는 message 변수를 정의합니다.

마지막 줄에서는 message 변수와 연결된 문자열로 HttpResponse 객체를 반환합니다. home 뷰 함수가 호출되면 웹 브라우저에 'Bookr에 오신 것을 환영합니다! 2 권의 책이 있습니다!'라는 메시지가 표시됩니다.

3. Now, create the URL configuration to call the newly created view function. Open the project URL file, `bookr/urls.py`, and add the list of `urlpatterns` as follows:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('reviews.urls'))
]
```

The first line in the list of `urlpatterns`, that is, `path('admin/', admin.site.urls)` routes to the admin URLs if `admin/` is present in the URL path (for example, `http://0.0.0.0:8000/admin`).

Similarly, consider the second line, `path('', include('reviews.urls'))`. Here, the path mentioned is an empty string, `''`. If the URL does not have any specific path after `http://hostname:port-number/` (for example, `http://0.0.0.0:8000/`), it includes `urlpatterns` present in `review.urls`.

The `include` function is a shortcut that allows you to combine URL configurations. It is common to keep one URL configuration per application in your Django project. Here, we've created a separate URL configuration for the reviews app and have added it to our project-level URL configuration.

4. Since we do not have the `reviews.urls` URL module yet, create a file called `bookr/reviews/urls.py`, and add the following lines of code:

```
from django.contrib import admin
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

3. 이제 새로 생성된 뷰 함수를 호출하기 위한 URL 구성을 만듭니다. 프로젝트 URL 파일인 `bookr/urls.py`를 열고 다음과 같이 `urlpatterns` 목록을 추가합니다:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('reviews.urls'))
]
```

목록의 `urlpatterns`에서 첫 번째 줄, 즉 `path('admin/', admin.site.urls)`는 URL 경로에 `admin/`가 포함되어 있으면 관리자 URL로 라우팅됩니다 (예: `http://0.0.0.0:8000/admin`).

유사하게, 두 번째 줄인 `path("", include('reviews.urls'))`를 고려해 보십시오. 여기서 언급된 경로는 빈 문자열, "입니다. URL이 `http://hostname:port-number/` (예: `http://0.0.0.0:8000/`) 뒤에 특정 경로가 없으면 `review.urls`에 있는 `urlpatterns`를 포함합니다.

`include` 함수는 URL 구성을 결합할 수 있는 단축키입니다. Django 프로젝트에서 애플리케이션당 하나의 URL 구성을 유지하는 것이 일반적입니다. 여기서는 리뷰 앱을 위한 별도의 URL 구성을 만들고 이를 프로젝트 수준의 URL 구성에 추가했습니다.

4. 우리는 아직 `reviews.urls` URL 모듈이 없으므로 `bookr/reviews/urls.py`라는 파일을 만들고 다음 코드 줄을 추가하십시오:

```
from django.contrib import admin
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

Here, we have used an empty string again for the URL path. So, when `http://0.0.0.0:8000/` is invoked, after getting routed from `bookr/urls.py` into `bookr/reviews/urls.py`, this pattern invokes the home view function.

5. After making changes to the two files, we have the necessary URL configuration ready to call the home view. Now, start the Django server with `./manage.py runserver` and type `http://0.0.0.0:8000` or `http://127.0.0.1:8000` in your web browser. You should be able to see the **Welcome to Bookr!** message:

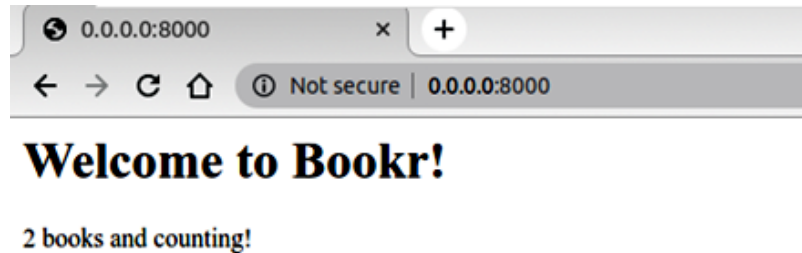


Figure 3.1 – Displaying “Welcome to Bookr!” and the number of books on the home page

Note

If there is no URL match, Django invokes error handling, such as displaying a **404 Page not found** message or something similar.

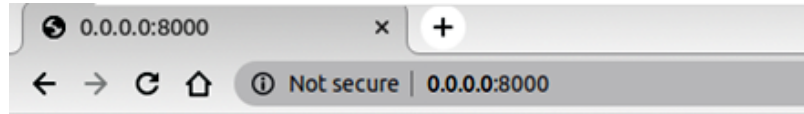
In development, with the `DEBUG` setting enabled, Django's error page will provide information to help understand the error and help fix it. In the example of a 404, it will show the URL patterns that it has tried to match against to find a route to a view for the request.

In this exercise, we learned how to write a basic view function and do the associated URL configuration. We have created a web page that displays a simple message to the user and reports how many books are currently in our database.

However, the astute reader will have noticed that it doesn't look very nice to have HTML code sitting inside our Python function as in the preceding example. As our views get bigger, this will become even more unsustainable. Therefore, in the next section, we will turn our attention to where our HTML code is supposed to be – inside templates.

여기서 우리는 URL 경로에 대해 다시 빈 문자열을 사용했습니다. 따라서 `http://0.0.0.0:8000/`가 호출되면 `bookr/urls.py`에서 `bookr/reviews/urls.py`로 라우팅된 후 이 패턴이 홈 뷰 함수를 호출합니다.

5. 두 파일에 변경을 한 후, 홈 뷰를 호출할 수 있는 필요한 URL 구성이 준비되었습니다. 이제 `./manage.py runserver`로 Django 서버를 시작하고 웹 브라우저에 `http://0.0.0.0:8000` 또는 `http://127.0.0.1:8000`을 입력하세요. 'Welcome to Bookr!' 메시지를 볼 수 있어야 합니다:



Welcome to Bookr!

2 books and counting!

그림 3.1 - 홈 페이지에 "Welcome to Bookr!"와 책의 수를 표시하기

참고

URL이 일치하지 않으면 Django는 404 페이지를 찾을 수 없다는 메시지와 같은 오류 처리를 호출합니다.

개발 중 DEBUG 설정이 활성화되면 Django의 오류 페이지는 오류를 이해하고 수정하는 데 도움이 되는 정보를 제공합니다. 404의 예에서는 요청에 대한 뷰로 가는 경로를 찾기 위해 시도한 URL 패턴을 보여줍니다.

이번 연습에서는 기본 뷰 함수를 작성하고 관련된 URL 구성을 하는 방법을 배웠습니다. 우리는 사용자에게 간단한 메시지를 표시하고 현재 데이터베이스에 몇 개의 책이 있는지 보고하는 웹 페이지를 만들었습니다.

그러나 날카로운 독자는 앞서의 예와 같이 HTML 코드가 우리의 Python 함수 안에 있는 것이 그리 보기 좋지 않다는 것을 눈치챈 것입니다. 우리의 뷰가 커질수록, 이것은 더욱 지속 불가능해질 것입니다. 따라서 다음 섹션에서는 우리의 HTML 코드가 있어야 할 곳, 즉 템플릿 안으로 우리의 주의를 돌릴 것입니다.

Working with Django templates

In *Exercise 3.01, Implementing a simple function-based view*, we saw how to create a view, configure the URL, and display a message in the browser. But if you recall, we hardcoded the Welcome to Bookr! HTML message in the view function itself and returned an `HttpResponse` object, as follows:

```
message = f"""<html><h1>Welcome to Bookr!</h1>
<p>{Book.objects.count()} books and counting!</p></html>"""
return HttpResponse(message)
```

Hardcoding of HTML inside Python modules is not a good practice, because as the content to be rendered on a web page increases, so does the amount of HTML code we need to write for it. Having a lot of HTML code among Python code can make the code hard to read and maintain in the long run.

For this reason, Django **templates** provide us with a better way to write and manage HTML templates. Django's templates not only work with static HTML content but also with dynamic HTML templates.

Django's template configuration is done in the `TEMPLATES` variable present in the `settings.py` file. This is how the default configuration looks:

```
TEMPLATES = [
    {
        'BACKEND':
            'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]
```

장고 템플릿 작업하기

연습 3.01, 간단한 함수 기반 뷰 구현에서, 우리는 뷰를 생성하고, URL을 구성하며, `idx_3c0d6326` 브라우저에 메시지를 표시하는 방법을 보았습니다. 하지만 기억하신다면, 우리는 뷰 함수 자체에 `Welcome to Bookr!` HTML 메시지를 하드코딩하고 `HttpResponse` 객체를 반환했습니다. 다음과 같아요:

```
message = f"""<html><h1>Welcome to Bookr!</h1><p>
{Book.objects.count()} books and counting!</p></html>"""
return HttpResponse(message)
```

HTML을 Python 모듈 내에 하드코딩하는 것은 좋은 관행이 아닙니다. 웹 페이지에 렌더링될 콘텐츠가 증가함에 따라, 이를 위해 작성해야 하는 HTML 코드의 양도 증가하기 때문입니다. Python 코드 사이에 많은 HTML 코드가 있으면 장기적으로 코드를 읽고 유지 관리하기 어려워질 수 있습니다.

이러한 이유로, 장고 템플릿은 HTML 템플릿을 작성하고 관리하는 더 나은 방법을 제공합니다. 장고의 템플릿은 정적 HTML 콘텐츠뿐만 아니라 동적 HTML 템플릿과도 작동합니다.

장고의 템플릿 구성은 `settings.py` 파일에 있는 `TEMPLATES` 변수에서 이루어집니다. 기본 구성은 다음과 같습니다:

```
TEMPLATES = [
    {
        'BACKEND':
            'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]
```

Let's go through each keyword present in the preceding snippet:

- `'BACKEND': 'django.template.backends.django.DjangoTemplates'`: This refers to the template engine to be used. A template engine is an API used by Django to work with HTML templates. Django is built with Jinja2 and the DjangoTemplates engine. The default configuration is the DjangoTemplates engine and Django template language. However, this can be changed to use a different one if required, such as Jinja2 or any other third-party template engine. For our Bookr application, though, we will leave this configuration as it is.
- `'DIRS': []`: This refers to the list of directories where Django searches for the templates in the given order.
- `'APP_DIRS': True`: This tells the Django template engine whether it should look for templates in the installed apps defined under `INSTALLED_APPS` in the `settings.py` file. The default option for this is `True`.
- `'OPTIONS'`: This is a dictionary containing template-engine-specific settings. Inside this dictionary, there is a default list of context processors, which helps the Python code to interact with templates to create and render dynamic HTML templates.

The current default settings are mostly fine for our purposes. However, in the next exercise, we will create a new directory for our templates, and we will need to specify the location of this folder. For example, if we have a directory called `my_templates`, we need to specify its location by adding it to the `TEMPLATES` settings as follows:

```
TEMPLATES = [  
    {  
        'BACKEND':  
            'django.template.backends.django.DjangoTemplates',  
        'DIRS': [os.path.join(BASE_DIR, 'my_templates')],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
]
```

이전 스니펃에 있는 각 `idx_b4f7d8a5` 키워드를 살펴보겠습니다:

- **'BACKEND':** `'django.template.backends.django.DjangoTemplates'`: 이는 사용될 템플릿 엔진을 나타냅니다. 템플릿 엔진은 Django가 HTML 템플릿과 작업하는 데 사용하는 API입니다. Django는 Jinja2와 DjangoTemplates 엔진으로 구축되었습니다. 기본 구성은 DjangoTemplates 엔진과 Django 템플릿 언어입니다. 그러나 필요에 따라 Jinja2 또는 다른 서드파티 템플릿 엔진을 사용하도록 변경할 수 있습니다. 그러나 우리의 Bookr 애플리케이션에서는 이 구성을 그대로 두겠습니다.
- **'DIRS':** `[]`: 이는 Django가 주어진 순서로 템플릿을 검색하는 디렉토리 목록을 나타냅니다.
- **'APP_DIRS':** `True`: 이는 Django 템플릿 엔진에 `settings.py` 파일의 `INSTALLED_APPS`에 정의된 설치된 앱에서 템플릿을 찾아야 하는지를 알려줍니다. 기본 옵션은 `True`입니다.
- **'OPTIONS':** 이것은 템플릿 엔진에 특정한 설정을 포함하는 사전입니다. 이 사전 안에는 기본 컨텍스트 프로세서 목록이 있으며, 이는 Python 코드가 템플릿과 상호작용하여 동적 HTML 템플릿을 생성하고 렌더링하는 데 도움을 줍니다.

현재 기본 설정은 우리의 목적에 대부분 적합합니다. 그러나 다음 연습에서는 템플릿을 위한 새 디렉토리를 만들 것이며, 이 폴더의 위치를 지정해야 합니다. 예를 들어, `my_templates`라는 디렉토리가 있다면, 다음과 같이 `TEMPLATES` 설정에 추가하여 그 위치를 지정해야 합니다:

```
TEMPLATES = [
    {
        'BACKEND':
            'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'my_templates')],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Note

When defining the directories to collect templates from here, the Python `os` module is used, so remember to use `import os` in your settings file.

`BASE_DIR` is the directory path to the project folder. This is defined in the `settings.py` file. The `os.path.join()` method joins the project directory with the templates directory, returning the full path for the templates directory. In the following exercise, we shall use a template to display a message to the user.

Exercise 3.02 – Using templates to display a greeting message

In this exercise, we will create our first Django template, and just as we did in the previous exercise, we will display the `Welcome to Bookr!` message using the templates. Let's get started with the steps:

1. Create a directory called `templates` in the `bookr` project directory, and inside it, create a file called `base.html`. If you already have a `templates` directory, consider removing it or adapting `base.html` if you have one. The directory structure should look like the following figure:

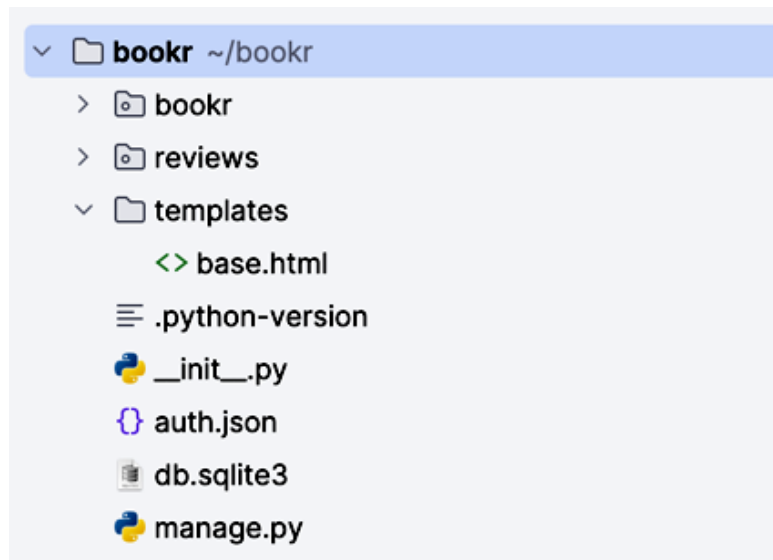


Figure 3.2 – Directory structure for `bookr`

노트

여기서 템플릿을 수집할 디렉토리를 정의할 때, Python os 모듈이 사용되므로 설정 파일에서 import os를 사용하는 것을 기억하세요.

BASE_DIR은 프로젝트 폴더에 대한 디렉토리 idx_4e8f2c88 경로입니다. 이는 settings.py 파일에서 정의됩니다. os.path.join() 메서드는 프로젝트 디렉토리와 템플릿 디렉토리를 결합하여 템플릿 디렉토리에 대한 전체 경로를 반환합니다. 다음 연습에서는 템플릿을 사용하여 사용자에게 메시지를 표시할 것입니다.

연습 3.02 – 템플릿을 사용하여 인사 메시지 표시하기

이 연습에서는 idx_eba8faf4 첫 번째 장고 템플릿을 만들 것이며, 이전 연습에서 했던 것처럼 템플릿을 사용하여 'Bookr에 오신 것을 환영합니다!' 메시지를 표시할 것입니다. 다음 단계로 시작해 봅시다:

1. bookr 프로젝트 디렉토리에 templates라는 디렉토리를 만들고 그 안에 base.html이라는 파일을 만듭니다. 이미 templates 디렉토리가 있다면 제거하거나 base.html을 조정하는 것을 고려하세요. 디렉토리 구조는 다음 그림과 같아야 합니다:

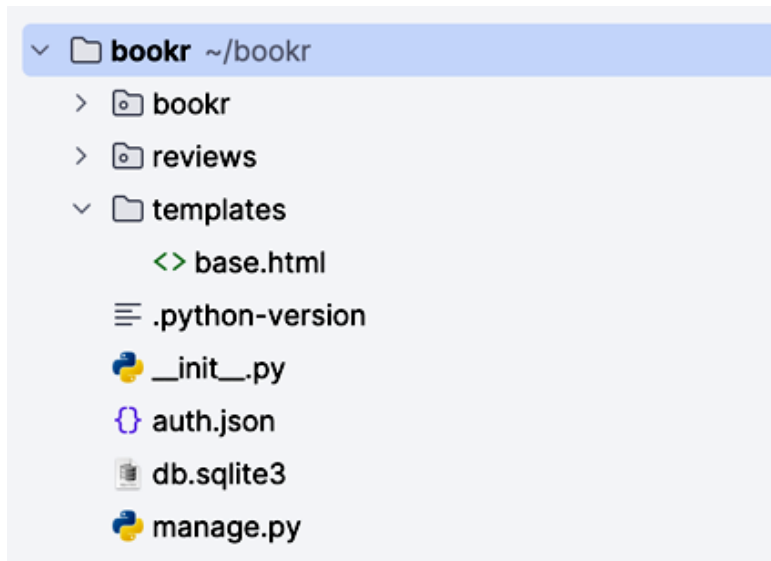


그림 3.2 - bookr의 디렉토리 구조

Note

When the default configuration is used (that is, when `DIRS` is an empty list), Django searches for templates present only in the app folders' `template` directory (the `reviews/templates` folder in the case of a book review application). Since we included the new template directory in the main project directory, Django's template engine would not be able to find the directory unless the directory is included in the `'DIRS'` list.

2. Add the folder to the `TEMPLATES` settings:

```
TEMPLATES = [  
    {  
        'BACKEND':  
            'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / 'templates'],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
]
```

3. Add the following lines of code to the `base.html` file:

```
<!doctype html>  
<html lang="en">  
<head>  
    <meta charset="utf-8">  
    <title>Home Page</title>  
</head>  
    <body>  
        <h1>Welcome to Bookr!</h1>
```

노트

기본 구성(즉, DIRS가 빈 목록일 때)을 사용할 경우, Django는 앱 폴더의 템플릿 디렉토리(책 리뷰 애플리케이션의 경우 reviews/templates 폴더)에서만 존재하는 템플릿을 검색합니다. 우리는 새로운 템플릿 디렉토리를 메인 프로젝트 디렉토리에 포함시켰기 때문에, Django의 템플릿 엔진은 'DIRS' 목록에 디렉토리가 포함되지 않는 한 해당 디렉토리를 찾을 수 없습니다.

2. idx_ec836667 폴더를 TEMPLATES 설정에 추가하세요:

```
TEMPLATES = [
    {
        'BACKEND':
            'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]
```

3. 다음 코드를 base.html 파일에 추가하세요:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>홈 페이지</title>
</head>
<body>
    <h1>북스에 오신 것을 환영합니다!</h1>
```



```
</body>  
</html>
```

This is simple HTML that displays the `Welcome to Bookr!` message in the header.

4. Modify the code inside `bookr/reviews/views.py` so that it looks as follows:

```
from django.shortcuts import render  
  
def home(request):  
    return render(request, 'base.html')
```

Since we have already configured the `'templates'` directory in the `TEMPLATES` configuration, `base.html` is available for use with the template engine. The code renders the `base.html` file using the imported `render` method from the `django.shortcuts` module.

5. Save the files, run `./manage.py runserver`, and open the `http://0.0.0.0:8000/` or `http://127.0.0.1:8000/` URL to check the newly added template loading in the browser:

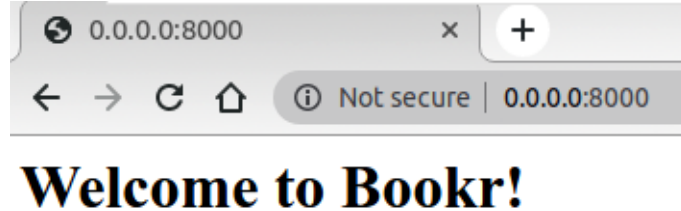


Figure 3.3 – Displaying "Welcome to Bookr!" on the home page

In this exercise, we created an HTML template and used Django templates and views to return the `Welcome to Bookr!` message.

Overall, we can say that Django templates help us to work with HTML templates and present information to the user in a desired format.

Next, we will learn about the Django template language, which can be used to render the application's data along with HTML templates.

```
</body>
</html>
```

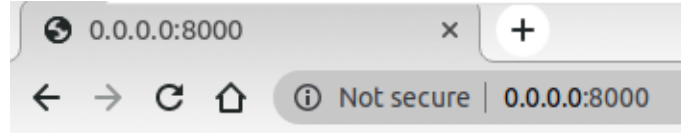
이것은 헤더에 'Bookr에 오신 것을 환영합니다!' 메시지를 표시하는 간단한 `idx_b09b57c1b11b219b.py` 내부의 코드를 다음과 같이 수정하세요:

```
from django.shortcuts import render

def home(request):
    return render(request, 'base.html')
```

우리는 이미 TEMPLATES 구성에서 'templates' 디렉토리를 설정했기 때문에 base.html을 템플릿 엔진과 함께 사용할 수 있습니다. 코드는 django.shortcuts 모듈에서 가져온 render 메서드를 사용하여 base.html 파일을 렌더링합니다.

5. `idx_b11b219b` 파일을 저장하고 `./manage.py runserver`를 실행한 후 `http://0.0.0.0:8000/` 또는 `http://127.0.0.1:8000/` URL을 열어 브라우저에서 새로 추가된 템플릿 로딩을 확인합니다:



Welcome to Bookr!

그림 3.3 - 홈페이지에 "Welcome to Bookr!" 표시하기

이번 연습에서는 HTML 템플릿을 만들고 Django 템플릿과 뷰를 사용하여 Welcome to Bookr! 메시지를 반환했습니다.

전반적으로 Django 템플릿은 HTML 템플릿으로 작업하고 사용자가 원하는 형식으로 정보를 제공하는 데 도움을 줍니다.

다음으로, HTML 템플릿과 함께 애플리케이션의 데이터를 렌더링하는 데 사용할 수 있는 Django 템플릿 언어에 대해 배울 것입니다.

Django template language

Django views not only return static HTML templates but can also add dynamic application data while generating the templates. Along with data, we can also include some programmatic elements in the templates. All of these put together form the basics of **Django's template language**. The following few sections look at some of the basic parts of the Django template language, such as template variables, template tags, comments, and filters.

Template variables

A **template variable** is represented between two curly braces, as shown here:

```
{{ variable }}
```

When this is present in the template, the value carried by the variables will be replaced in the template. Template variables help in adding the application's data into the templates. They can be added by defining context data for a view, modifying the code inside `bookr/reviews/views.py` so that it looks as follows:

```
from django.shortcuts import render

def home(request):
    context_data = {
        "template_variable": "I am a template variable."
    }
    return render(request, 'base.html', context=context_data)
```

Then, in `base.html`, the variable would be included as follows:

```
<body>
    {{ template_variable }}
</body>
```

Template tags

A **tag** is similar to a programmatic control flow, such as an `if` condition or a `for` loop. A tag is represented between two curly braces and percentage signs, as shown. Here is an example of a `for` loop iterating over a list using template tags:

```
{% for element in element_list %}

{% endfor %}
```

장고 템플릿 언어

장고 뷰는 정적 HTML 템플릿을 반환할 뿐만 아니라 템플릿을 생성하는 동안 동적 애플리케이션 데이터를 추가할 수도 있습니다. 데이터와 함께 템플릿에 일부 프로그래밍 요소를 포함할 수도 있습니다. 이 모든 것들이 함께 모여 장고의 템플릿 언어의 기초를 형성합니다. 다음 몇 섹션에서는 템플릿 변수, 템플릿 태그, 주석 및 필터와 같은 장고 템플릿 언어의 기본 부분을 살펴봅니다.

템플릿 변수

템플릿 변수는 두 개의 중괄호 사이에 표시됩니다. 다음과 같이 표시됩니다:

```
{{ variable }}
```

템플릿에 이것이 존재하면 변수에 의해 전달된 값이 템플릿에서 대체됩니다. 템플릿 변수는 애플리케이션의 데이터를 템플릿에 추가하는 데 도움이 됩니다. 뷰에 대한 컨텍스트 데이터를 정의하여 추가할 수 있으며, `bookr/reviews/views.py` 내부의 코드를 수정하여 다음과 같이 보이도록 할 수 있습니다:

```
from django.shortcuts import render

def home(request):
    context_data = {
        "template_variable": "나는 템플릿 변수입니다."
    }
    return render(request, 'base.html', context=context_data)
```

그런 다음, `base.html`에서 변수는 다음과 같이 포함됩니다:

```
<body>
    {{ template_variable }}
</body>
```

템플릿 태그

태그는 `if` 조건이나 `for` 루프와 같은 프로그래밍적 `idx_ac9710e8` 제어 흐름과 유사합니다. 태그는 두 개의 중괄호와 백분율 기호 사이에 표시됩니다. 다음은 템플릿 태그를 사용하여 목록을 반복하는 `for` 루프의 예입니다:

```
{% for element in element_list %}

{% endfor %}
```

Unlike Python programming, we also add the end of the control flow by adding the end tag, such as `{% endfor %}`. This can be used along with template variables to display the elements in the list, as shown here:

```
<ul>
  {% for element in element_list %}
    <li>{{ element.title }}</li>
  {% endfor %}
</ul>
```

Comments

Comments in the Django template language can be written as shown here; anything in between `{% comment %}` and `{% endcomment %}` will be commented out:

```
{% comment %}
  <p>This text has been commented out</p>
{% endcomment %}
```

Filters

Filters can be used to modify a variable to represent it in a different format. The syntax for a filter is a variable separated from the filter name using a pipe (|) symbol:

```
{{ variable|filter }}
```

Here are some examples of built-in filters:

- `{{ variable|lower }}`: This converts the variable string into lowercase.
- `{{ variable|title }}`: This converts the first letter of every word into uppercase.

Let's use the concepts we have learned up till now to develop the book review application.

파이썬 프로그래밍과 달리, {% endfor %}와 같은 종료 태그를 추가하여 제어 흐름의 끝을 추가합니다. 이는 템플릿 변수와 함께 사용되어 리스트의 요소를 표시할 수 있습니다, 여기에서 보여지는 것처럼:

```
<ul>
  {% for element in element_list %}
    <li>{{ element.title }}</li>
  {% endfor %}
</ul>
```

주석

장고 idx_45255a75 템플릿 언어에서 주석은 다음과 같이 작성할 수 있습니다; {% comment %}와 {% endcomment %} 사이의 모든 것은 주석 처리됩니다:

```
{% comment %}
  <p>이 텍스트는 주석 처리되었습니다</p>{%
endcomment %}
```

필터

필터는 idx_316877cd 변수를 수정하여 다른 형식으로 표현하는 데 사용할 수 있습니다. 필터의 구문은 변수와 필터 이름을 파이프 (|) 기호로 구분하는 것입니다:

```
{{ variable|filter }}
```

내장 필터의 몇 가지 예는 다음과 같습니다:

- {{ variable|lower }}: 이 필터는 변수 문자열을 소문자로 변환합니다.
- {{ variable|title }}: 이 필터는 모든 단어의 첫 글자를 대문자로 변환합니다. 지금까지 배운 개념을 사용하여 도서 리뷰 애플리케이션을 개발해 봅시다.

Exercise 3.03 – Displaying a list of books and reviews

In this exercise, we will create a web page that can display a list of all books, their ratings, and the number of reviews present on the book review application. For this, we will be using some features of the Django template language, such as template variables and tags, to pass the book review application data into the templates to display meaningful data on the web page:

1. Create a file called `utils.py` under `bookr/reviews/utils.py` and add the following code:

```
def average_rating(rating_list):
    if not rating_list:
        return 0

    return round(sum(rating_list) / len(rating_list))
```

`average_rating` is a helper method that will be used to calculate the average rating of a book.

2. Include these `import` statements at the start of `bookr/reviews/views.py`:

```
from django.shortcuts import render

from .models import Book
from .utils import average_rating
```

3. This is to import the Django `render` function, the `Book` model class, and the helper method we just added.
4. Add this function to `bookr/reviews/views.py`:

```
def book_list(request):
    books = Book.objects.all()
    book_list = []
    for book in books:
        reviews = book.review_set.all()
        if reviews:
            book_rating = average_rating([
                review.rating for review in reviews])
            number_of_reviews = len(reviews)
        else:
            book_rating = None
```

연습 3.03 – 책과 리뷰 목록 표시하기

이 연습에서는 모든 책의 목록, 평점 및 책 리뷰 애플리케이션에 있는 리뷰 수를 표시할 수 있는 `idx_c7de1ff3` 웹 페이지를 생성할 것입니다. 이를 위해 Django 템플릿 언어의 일부 기능, 즉 템플릿 변수와 태그를 사용하여 책 리뷰 애플리케이션 데이터를 템플릿에 전달하여 웹 페이지에 의미 있는 데이터를 표시합니다:

1. `bookr/reviews/utils.py` 아래에 `utils.py`라는 파일을 만들고 다음 코드를 추가합니다:

```
def average_rating(rating_list):
    if not rating_list:
        return 0

    return round(sum(rating_list) / len(rating_list))
```

`average_rating`은 책의 평균 평점을 계산하는 데 사용될 도우미 메서드입니다.

2. `bookr/reviews/views.py`의 시작 부분에 다음 `import` 문을 포함하세요:

```
from django.shortcuts import render

from .models import Book
from .utils import average_rating
```

3. 이는 Django `render` 함수, `Book` 모델 클래스 및 우리가 방금 추가한 도우미 메서드를 가져오기 위한 것입니다.
4. 이 `idx_58b69492` 함수를 `bookr/reviews/views.py`에 추가하세요:

```
def book_list(request):
    books = Book.objects.all()
    book_list = []
    for book in books:
        reviews = book.review_set.all()
        if reviews:
            book_rating = average_rating([
                review.rating for review in reviews])
            number_of_reviews = len(reviews)
        else:
            book_rating = None
```



```
        number_of_reviews = 0
    book_list.append({
        'book': book,
        'book_rating': book_rating,
        'number_of_reviews': number_of_reviews})
    context = {
        'book_list': book_list
    }
    return render(request, 'reviews/book_list.html', context)
```

5. This is a view to display the list of books for the Bookr application.

Note

This example includes a reviews folder in the template name for the render function. This folder wasn't created in the initial configuration of the templates directory with the base.html template, so the code will likely throw a `TemplateDoesNotExist` error. It is good practice to create a folder with the same name as the application for its templates. These application-specific templates can inherit from the same base template, which covers the basic setup for the whole project, such as style sheets and JavaScript libraries common between all templates.

6. Django will read all template directories and collect the files together, so without a folder structure, there is no differentiation between two templates of the same name. Here, `book_list` is the view function. In this function, we start by querying the list of all books. Next, for every book, we calculate the average rating and the number of reviews posted. All this information for each book is appended to a list called `book_list` as a list of dictionaries. This list is then added to a dictionary named `context` and is passed to the render function.

The render function takes three parameters: the first one is the request object that was passed into the view, the second is the `book_list.html` HTML template, which will display the list of books, and the third is `context`, which we pass to the template.

Since we have passed `book_list` as a part of the context, the template will use this to render the list of books using template tags and template variables.

```

        number_of_reviews = 0
    book_list.append({
        'book': book,
        'book_rating': book_rating,
        'number_of_reviews': number_of_reviews})
    context = {
        'book_list': book_list
    }
    return render(request, 'reviews/book_list.html', context)

```

5. 이것은 Bookr 애플리케이션의 책 목록을 표시하는 뷰입니다.

참고

이 예제는 render 함수의 템플릿 이름에 reviews 폴더를 포함합니다. 이 폴더는 base.html 템플릿으로 templates 디렉토리의 초기 구성에서 생성되지 않았으므로, 코드는 TemplateDoesNotExist 오류를 발생시킬 가능성이 높습니다. 애플리케이션의 템플릿과 동일한 이름의 폴더를 만드는 것이 좋은 관행입니다. 이러한 애플리케이션 전용 템플릿은 전체 프로젝트에 공통적인 스타일 시트 및 JavaScript 라이브러리와 같은 기본 설정을 포함하는 동일한 기본 템플릿에서 상속받을 수 있습니다.

6. Django는 모든 템플릿 디렉토리를 읽고 파일을 함께 수집하므로, 폴더 구조가 없으면 동일한 이름의 두 템플릿 간에 구분이 없습니다. 여기서 book_list는 idx_82413e84 뷰 함수입니다. 이 함수에서는 모든 책의 목록을 쿼리하는 것으로 시작합니다. 다음으로, 각 책에 대해 평균 평점과 게시된 리뷰 수를 계산합니다. 각 책에 대한 모든 정보는 사전 목록으로서 book_list라는 목록에 추가됩니다. 이 목록은 context라는 이름의 사전에 추가되어 render 함수에 전달됩니다.

render 함수는 세 개의 매개변수를 받습니다: 첫 번째는 뷰에 전달된 요청 객체, 두 번째는 책 목록을 표시할 book_list.html HTML 템플릿, 세 번째는 템플릿에 전달되는 context입니다.

우리는 context의 일부로 book_list를 전달했기 때문에, 템플릿은 이를 사용하여 템플릿 태그와 템플릿 변수를 사용하여 책 목록을 렌더링합니다.

7. Create the `book_list.html` file in the `bookr/reviews/templates/reviews/` `book_list.html` path and add the following HTML code to the file:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Bookr</title>
</head>
<body>
  <h1>Book Review application</h1>
  <hr>
```

You can find the complete code in the `Chapter03` folder in this book's GitHub repo.

This is a simple HTML template with template tags and variables iterating over `book_list` to display the list of books.

8. In `bookr/reviews/urls.py`, add to `urlpatterns` the following URL pattern to invoke the `book_list` view:

```
path('books/', views.book_list, name='book_list'),
```

This configures the URL for the `book_list` view function.

9. Save all the modified files and wait for the Django service to restart. Open `http://0.0.0.0:8000/books/` in the browser, and you should see something similar to the following figure:

7. bookr/reviews/templates/reviews/book_list.html 경로에 book_list.html 파일을 생성하고 다음 HTML 코드를 파일에 추가하세요:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Bookr</title>
</head>
<body>
  <h1>도서 리뷰 애플리케이션</h1>
  <hr>
```

이 책의 GitHub 저장소의 Chapter03 폴더에서 전체 코드를 찾을 수 있습니다.

이것은 template 태그와 변수를 사용하여 book_list를 반복하여 책 목록을 표시하는 간단한 idx_91db5ca7HTML 템플릿입니다.

8. bookr/reviews/urls.py에서 urlpatterns에 다음 URL 패턴을 추가하여 book_list 뷰를 호출합니다:

```
path('books/', views.book_list, name='book_list'),
```

이것은 book_list 뷰 함수에 대한 URL을 구성합니다.

9. 수정된 모든 파일을 저장하고 Django 서비스가 재시작될 때까지 기다립니다. 브라우저에서 <http://0.0.0.0:8000/books/>를 열면 다음 그림과 유사한 것을 볼 수 있어야 합니다:

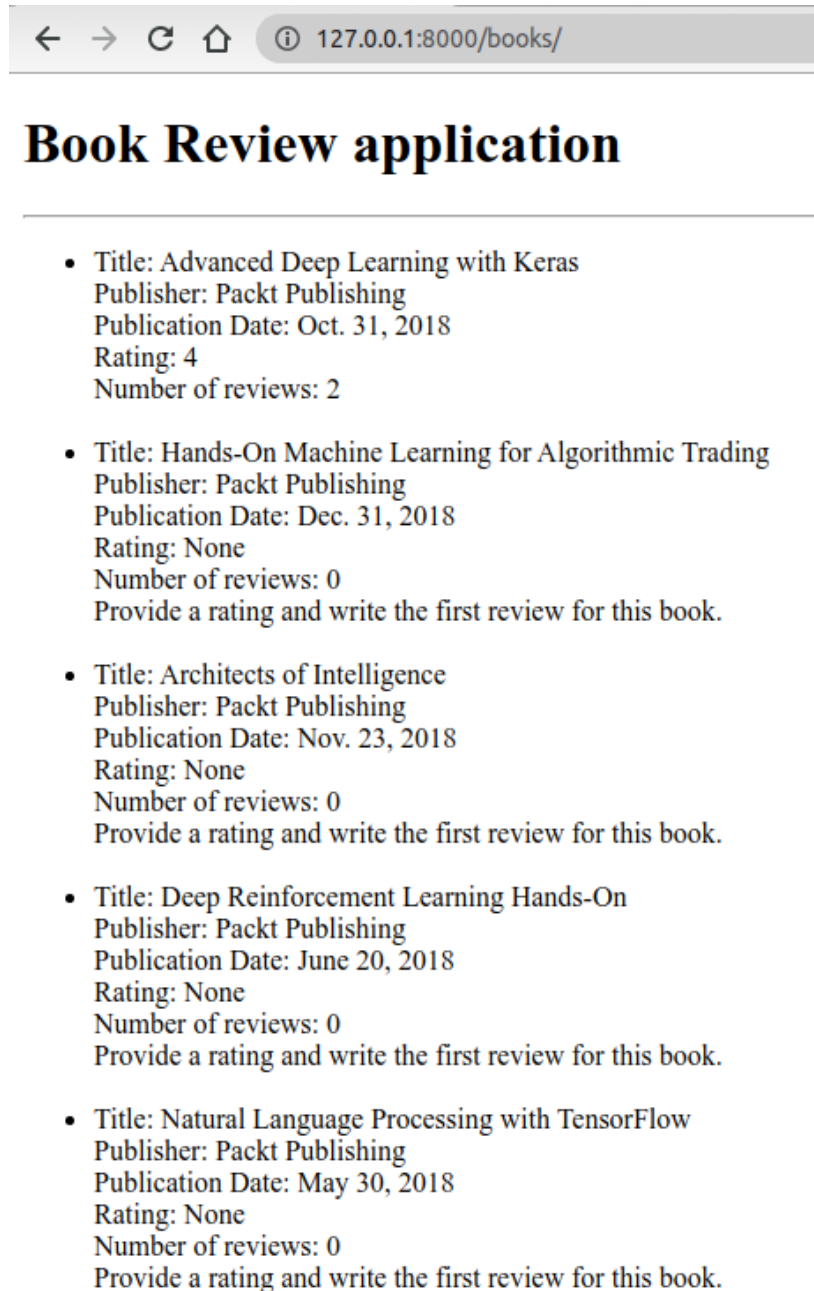
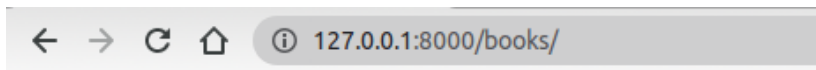


Figure 3.4 – List of books present in the book review application



Book Review application

- Title: Advanced Deep Learning with Keras
Publisher: Packt Publishing
Publication Date: Oct. 31, 2018
Rating: 4
Number of reviews: 2
- Title: Hands-On Machine Learning for Algorithmic Trading
Publisher: Packt Publishing
Publication Date: Dec. 31, 2018
Rating: None
Number of reviews: 0
Provide a rating and write the first review for this book.
- Title: Architects of Intelligence
Publisher: Packt Publishing
Publication Date: Nov. 23, 2018
Rating: None
Number of reviews: 0
Provide a rating and write the first review for this book.
- Title: Deep Reinforcement Learning Hands-On
Publisher: Packt Publishing
Publication Date: June 20, 2018
Rating: None
Number of reviews: 0
Provide a rating and write the first review for this book.
- Title: Natural Language Processing with TensorFlow
Publisher: Packt Publishing
Publication Date: May 30, 2018
Rating: None
Number of reviews: 0
Provide a rating and write the first review for this book.

그림 3.4 - 도서 리뷰 애플리케이션에 있는 책 목록

In this exercise, we created a view function, created templates, and added a URL configuration, which can display a list of all books present in the application. Although we were able to display a list of books using a single template, next, let's explore a bit about how to work with multiple templates in an application that has common or similar code.

Template inheritance

As we build the project, the number of templates will increase. It is highly probable that when we design the application, some of the pages will look similar and have common HTML code for certain features. Using template inheritance, we can inherit the common HTML code in other HTML files. This is similar to class inheritance in Python, where the parent class has all the common code, and the child class has those extras that are unique to the child's requirements.

Note

It would be good practice to update the home view to use an `home.html` that is a child of `base.html`. Having the base template as the common ancestor to templates allows for all the boilerplate, style sheets, JavaScript, and so on to be defined once and then shared between all templates.

For example, let's consider the following to be a parent template that is named `base.html`:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>{% block title %}Hello World{% endblock %}</title>
</head>
<body>
    <h1>Hello World using Django templates!</h1>
    {% block content %}
    {% endblock %}
</body>
</html>
```

The following is an example of a child template:

```
{% extends 'base.html' %}
{% block content %}
```

이 연습에서는 뷰 함수를 만들고, 템플릿을 생성하며, 애플리케이션에 있는 모든 책의 목록을 표시할 수 있는 URL 구성을 추가했습니다. 단일 템플릿을 사용하여 책 목록을 표시할 수 있었지만, 다음으로는 공통 또는 유사한 코드를 가진 애플리케이션에서 여러 템플릿을 사용하는 방법에 대해 조금 더 알아보겠습니다.

템플릿 상속

프로젝트를 진행하면서 idx_890f685b 템플릿의 수가 증가할 것입니다. 애플리케이션을 설계할 때 일부 페이지가 비슷하게 보이고 특정 기능에 대해 공통 HTML 코드를 가질 가능성이 높습니다. 템플릿 상속을 사용하면 다른 HTML 파일에서 공통 HTML 코드를 상속받을 수 있습니다. 이는 부모 클래스가 모든 공통 코드를 가지고 있고 자식 클래스가 자식의 요구 사항에 고유한 추가 코드를 가지는 Python의 클래스 상속과 유사합니다.

참고

home.html을 base.html의 자식으로 사용하도록 홈 뷰를 업데이트하는 것이 좋은 연습이 될 것입니다. 기본 템플릿을 템플릿의 공통 조상으로 두면 모든 보일러플레이트, 스타일 시트, JavaScript 등을 한 번 정의한 후 모든 템플릿 간에 공유할 수 있습니다.

예를 들어, idx_9c31f795 다음 내용을 base.html이라는 이름의 부모 템플릿으로 고려해 보겠습니다:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>{% block title %}안녕하세요, 세계!{% endblock %}</title>
</head>
  <body>
    <h1>Django 템플릿을 사용한 안녕하세요, 세계!</h1>
    {% block content %}
    {% endblock %}
  </body>
</html>
```

다음은 자식 템플릿의 예입니다:

```
{% extends 'base.html' %}
{% block content %}
```



```
<h1>How are you doing?</h1>
{% endblock %}
```

In the preceding snippet, the `{% extends 'base.html' %}` line extends the template from `base.html`, which is the parent template. After extending from the parent template, any HTML code in between the block content will be displayed along with the parent template. Once the child template is rendered, here is how it looks in the browser:

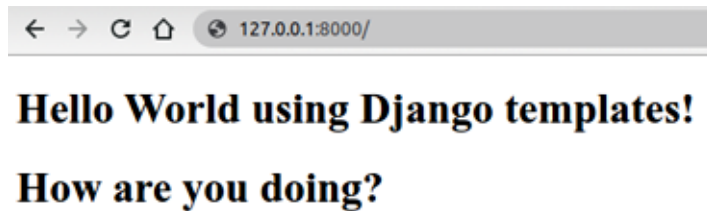


Figure 3.5 – Greeting message after extending the `base.html` template

In the next section, we will do some template styling using Bootstrap.

Template styling with Bootstrap

We have seen how to display all the books using views, URL configuration, and templates. Although we were able to display all the information in the browser, it would be even better if we could add some styling and make the web page look better. For this, we can add a few elements of **Bootstrap**. Bootstrap is an open source **Cascading Style Sheets (CSS)** framework that is particularly good for designing responsive pages that work across desktop and mobile browsers.

Using Bootstrap is simple. First, you need to add the Bootstrap CSS to your HTML. You can experiment for yourself by creating a new file called `example.html`. Populate it with the following code and open it in a browser:

```
<!doctype html>
<html lang="en">
  <head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width,
      initial-scale=1, shrink-to-fit=no">

    <!-- Bootstrap CSS -->
    <link rel="stylesheet" href=
      "https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css"
```

```
<h1>어떻게 지내고 있나요?
</h1>{% endblock %}
```

앞의 코드 조각에서 `{% extends 'base.html' %}` 줄은 부모 템플릿인 `base.html`에서 템플릿을 확장합니다. 부모 템플릿에서 확장한 후, 블록 콘텐츠 사이의 모든 HTML 코드는 부모 템플릿과 함께 표시됩니다. 자식 템플릿이 `idx_71ae3692` 렌더링되면 브라우저에서 다음과 같이 보입니다:

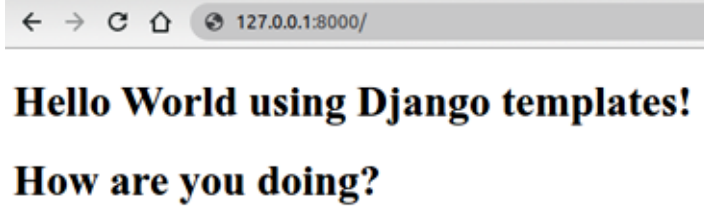


그림 3.5 - `base.html` 템플릿을 확장한 후의 인사 메시지

다음 섹션에서는 Bootstrap을 사용하여 템플릿 스타일링을 할 것입니다.

Bootstrap을 사용한 템플릿 스타일링

우리는 뷰, URL 구성 및 템플릿을 사용하여 모든 책을 표시하는 방법을 보았습니다. 브라우저에서 모든 정보를 표시할 수 있었지만, 스타일을 추가하고 웹 페이지를 더 보기 좋게 만들 수 있다면 더욱 좋을 것입니다. 이를 위해 Bootstrap의 몇 가지 요소를 추가할 수 있습니다. Bootstrap `idx_0cd80985`는 데스크톱 및 모바일 브라우저에서 작동하는 반응형 페이지 디자인에 특히 좋은 오픈 소스 Cascading Style Sheets (CSS) 프레임워크입니다.

Bootstrap을 사용하는 것은 간단합니다. 먼저, HTML에 Bootstrap CSS를 추가해야 합니다. `idx_9b1f7d99`라는 새 파일을 만들어 직접 실험해 볼 수 있습니다. 다음 코드를 사용하여 `idx_806fb2f3`를 채우고 브라우저에서 열어보세요:

```
<!doctype html>
<html lang="en">
  <head>
    <!-- 필수 메타 태그 -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width,
      initial-scale=1, shrink-to-fit=no">

    <!-- 부트스트랩 CSS -->
    <link rel="stylesheet" href=
      "https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css"
```

```
integrity="sha384-Vkoo8x4CGs03+Hhxv8T/
Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh" crossorigin="anonymous">

</head>
<body>
    Content goes here
</body>
</html>
```

The Bootstrap CSS link in the preceding code adds the Bootstrap CSS library to your page. This means that certain HTML element types and classes will inherit their styles from Bootstrap. For example, if you add the `btn-primary` class to the class of a button, the button will be rendered blue with white text. Try adding the following between `<body>` and `</body>`:

```
<h1>Welcome to my Site</h1>
<button type="button" class="btn btn-primary">
    Checkout my Blog!</button>
```

You will see that the title and button are both styled nicely, using Bootstrap's default styles:

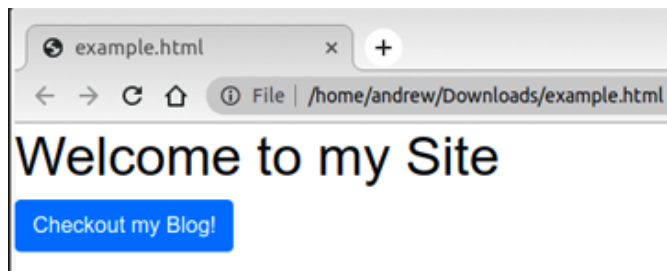


Figure 3.6 – Display after applying Bootstrap

This is because in the Bootstrap CSS code, they specify the color of the `btn-primary` class with the following code:

```
.btn-primary {
    color: #fff;
    background-color: #007bff;
    border-color: #007bff
}
```

```

integrity="sha384-Vkoo8x4CGs03+
Hhxv8T/Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh" crossorigin="anonymous">

</head>
<body>
  내용이 여기에 들어
  갑니다</body>

</html>

```

앞서 언급한 코드의 Bootstrap idx_f3114fccCSS 링크는 Bootstrap CSSidx_5c15f01e 라이브러리를 페이지에 추가합니다. 이는 특정 HTML 요소 유형과 클래스가 Bootstrap에서 스타일을 상속받는다는 것을 의미합니다. 예를 들어, 버튼의 클래스에 btn-primary 클래스를 추가하면 버튼이 흰색 텍스트와 함께 파란색으로 렌더링됩니다. <body>와 </body> 사이에 다음을 추가해 보세요:

```

<h1>내 사이트에 오신 것을 환영합니다</h1>

<button type="button" class="btn btn-primary">
  내 블로그 확인하기!</button>

```

제목과 버튼이 모두 부트스트랩의 기본 스타일을 사용하여 멋지게 스타일링된 것을 보게 될 것입니다:

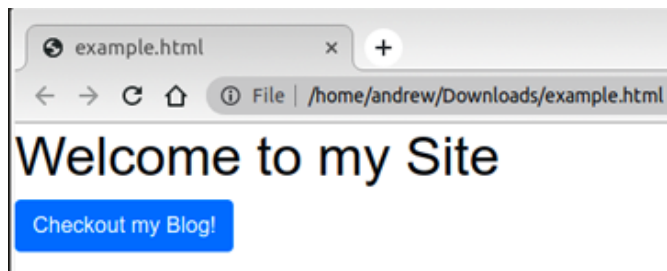


그림 3.6 - 부트스트랩을 적용한 후의 표시

이는 부트스트랩 CSS 코드에서 btn-primary 클래스의 색상을 다음 코드로 지정하기 때 문입니다:

```

.btn-primary {
  color: #fff;
  background-color: #007bff;
  border-color: #007bff
}

```

You can see that using third-party CSS libraries such as Bootstrap allows you to quickly create nicely styled components without needing to write too much CSS. Next, we shall use template inheritance from a base template and also add some styling elements, such as a navigation bar.

Note

We recommend that you explore Bootstrap further with its tutorial here: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>.

Exercise 3.04 – Adding template inheritance and a Bootstrap navigation bar

In this exercise, we will use template inheritance to inherit the template elements from a base template and reuse them in the `book_list` template to display the list of books. We will also use certain elements of Bootstrap in the base HTML file to add a navigation bar to the top of our page. The Bootstrap code for `base.html` was taken from <https://getbootstrap.com/docs/5.3/getting-started/introduction/> and <https://getbootstrap.com/docs/5.3/components/navbar/>. Let's get started with the steps:

1. Open the `base.html` file from the `bookr/templates/base.html` location. Remove any existing code and replace it with the following code:

```
<!doctype html>
{% load static %}
<html lang="en">
<head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width,
        initial-scale=1, shrink-to-fit=no">
    <!-- Bootstrap CSS -->
```

You can view the entire code for this file in the `Chapter03` folder in this book's GitHub repo. This is a `base.html` file with all the Bootstrap elements for styling and the navigation bar.

부트스트랩과 같은 서드파티 CSS 라이브러리를 사용하면 너무 많은 CSS를 작성하지 않고도 빠르게 멋지게 스타일링된 구성 요소를 만들 수 있다는 것을 알 수 있습니다. 다음으로, 기본 템플릿에서 템플릿 상속을 사용하고 탐색 막대와 같은 스타일 요소를 추가할 것입니다.

참고

부트스트랩에 대한 튜토리얼을 통해 더 자세히 알아보시기 바랍니다:
<https://getbootstrap.com/docs/5.3/getting-started/introduction/>.

연습 3.04 – 템플릿 상속 및 부트스트랩 탐색 막대 추가

이 연습에서는 idx_69efac04 템플릿 상속을 사용하여 기본 템플릿의 템플릿 요소를 상속받고 이를 book_list 템플릿에서 재사용하여 책 목록을 표시할 것입니다. 우리는 idx_da16616a 또한 기본 HTML 파일에서 부트스트랩의 특정 요소를 사용하여 페이지 상단에 탐색 막대를 추가할 것입니다. base.html의 부트스트랩 코드는 <https://getbootstrap.com/docs/5.3/getting-started/introduction/> 및 <https://getbootstrap.com/docs/5.3/components/navbar/>에서 가져왔습니다. 단계별로 시작해 보겠습니다:

1. bookr/templates/base.html 위치에서 base.html 파일을 엽니다. 기존 코드를 모두 제거하고 다음 코드로 교체합니다:

```
<!doctype html>
{% load static %}
<html lang="en">
<head>
    <!-- 필수 메타 태그 -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width,
        initial-scale=1, shrink-to-fit=no">
    <!-- 부트스트랩 CSS -->
```

이 파일의 전체 코드는 이 책의 GitHub 저장소의 Chapter03 폴더에서 확인할 수 있습니다. 이것은 스타일링과 내비게이션 바를 위한 모든 부트스트랩 요소가 포함된 base.html 파일입니다.

2. Next, open the template at `bookr/reviews/templates/reviews/book_list.html`, remove all the existing code, and replace it with the following code:

```
{% extends 'base.html' %}

{% block content %}
<ul class="list-group">
  {% for item in book_list %}
    <li class="list-group-item">
      <span class="text-info">Title: </span> <span>{{
        item.book.title }}</span>
      <br>
      <span class="text-info">Publisher:
        </span><span>{{
          item.book.publisher }}</span>
    </li>
  {% endfor %}
</ul>
```

You can view the complete code for this file in the `Chapter03` folder in this book's GitHub repo. This template has been configured to inherit the `base.html` file, and it has also been added with a few styling elements to display the list of books. The part of the template that helps in inheriting the `base.html` file is as follows:

```
{% extends 'base.html' %}

{% block content %}
{% endblock %}
```

3. After adding the two new templates, open either of the URLs (`http://0.0.0.0:8000/books/` or `http://127.0.0.1:8000/books/`) in your web browser to see the book list page, which should now look neatly formatted:

2. 다음으로, `bookr/reviews/templates/reviews/book_list.html`에서 템플릿을 열고, 기존 코드를 모두 제거한 후 다음 코드를 추가하세요:

```
{% extends 'base.html' %}

{% block content %}
<ul class="list-group">
  {% for item in book_list %}
  <li class="list-group-item">
    <span class="text-info">제목: </span> <span>{{
      item.book.title }}</span>
    <br>
    <span class="text-info">출판사:
      </span><span>{{
        item.book.publisher }}</span>
  </li>
  </ul>
{% endblock %}
```

이 파일의 전체 코드는 이 책의 GitHub 리포지토리의 Chapter03 폴더에서 확인할 수 있습니다. 이 템플릿은 `base.html` 파일을 상속하도록 구성되었으며, 책 목록을 표시하기 위해 몇 가지 스타일 요소가 추가되었습니다. `base.html` 파일을 상속하는 데 도움이 되는 템플릿 부분은 다음과 같습니다:

```
{% extends 'base.html' %}

{% block content %}
{% endblock %}
```

3. 두 개의 새로운 템플릿을 추가한 후, 웹 브라우저에서 `idx_9f69b23f` URL 중 하나 (<http://0.0.0.0:8000/books/> 또는 <http://127.0.0.1:8000/books/>)를 열어 책 목록 페이지를 확인하세요. 이제 페이지가 깔끔하게 포맷되어 있어야 합니다:

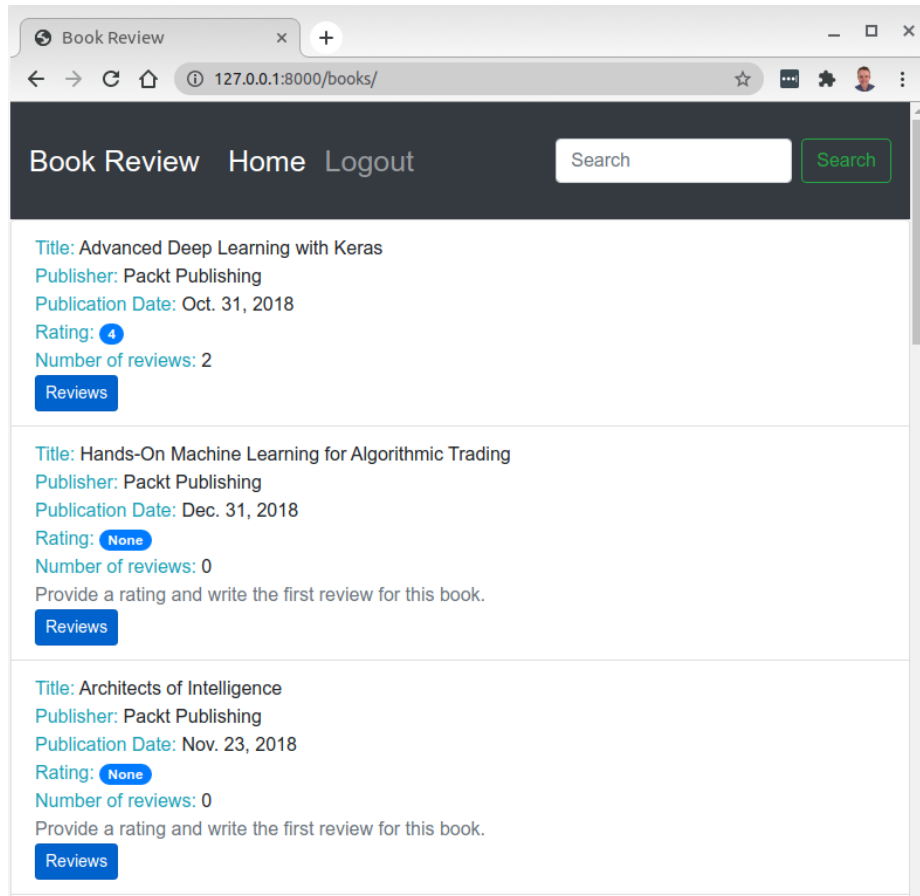


Figure 3.7 – Neatly formatted book list page

In this exercise, we added some styling to the application using Bootstrap, and we also used template inheritance while we displayed the list of books from the book review application. So far, we have worked extensively on displaying all the books present in the application. In the next activity, you will display the details and reviews of an individual book.

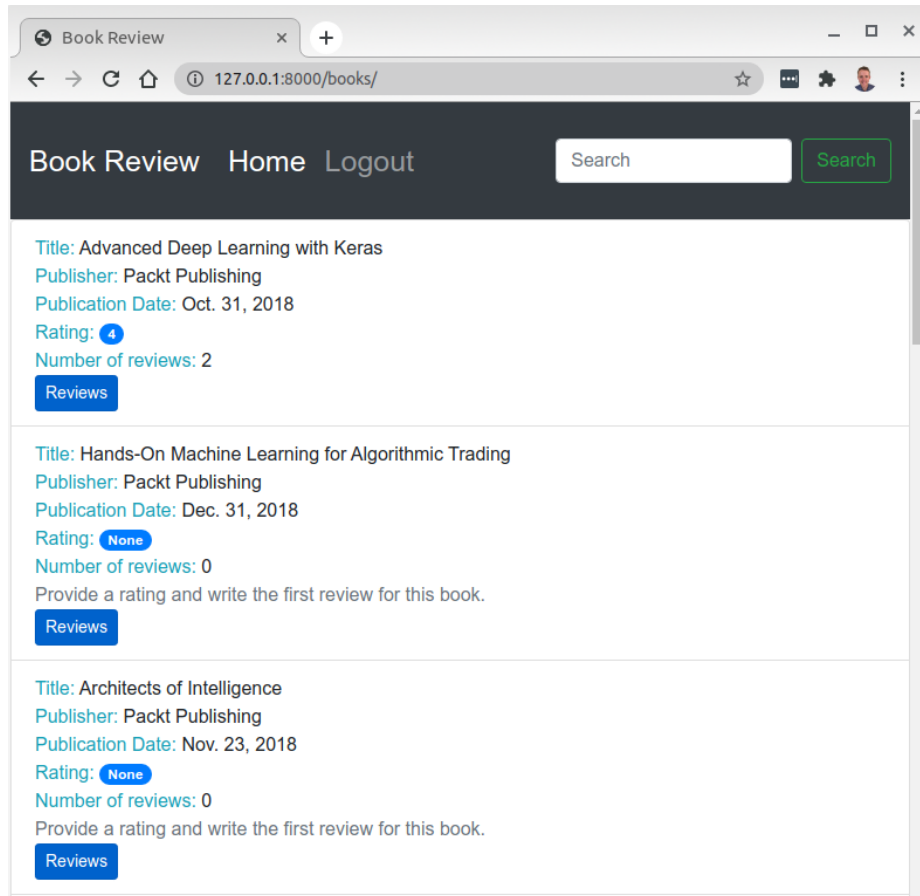


그림 3.7 - 깔끔하게 포맷된 도서 목록 페이지

이번 연습에서는 Bootstrap을 사용하여 애플리케이션에 스타일을 추가했으며, 도서 리뷰 애플리케이션에서 도서 목록을 표시할 때 템플릿 상속도 사용했습니다. 지금까지 우리는 애플리케이션에 있는 모든 도서를 표시하는 데 많은 작업을 했습니다. 다음 활동에서는 개별 도서의 세부정보와 리뷰를 표시할 것입니다.

Activity 3.01 – Implementing the book details view

In this activity, you will implement a new view, template, and URL configuration to display these details of a book: title, publisher, publication date, and overall rating. In addition to these details, the page should also display all the review comments, specifying the name of the commenter and the dates on which the comments were written and (if applicable) modified. The following steps will help you complete this activity:

1. Create a book details endpoint that extends the base template.
2. Create a book details view that takes a specific book's primary key as the argument and returns an HTML page listing the book's details and any associated reviews.
3. Add the required URL configuration in `urls.py`. The book details view URL should be `http://0.0.0.0:8000/books/1/` (where 1 will represent the ID of the book being accessed). You can use the `get_object_or_404` method to retrieve the book with the given primary key.

Note

The `get_object_or_404` function is a useful shortcut for retrieving an instance based on its primary key. You could also do this using the `.get()` method described in *Chapter 2*: `Book.objects.get(pk=pk)`. However, `get_object_or_404` has the added advantage of returning an HTTP 404 Not Found response if the object does not exist. If we simply use `get()` and someone attempts to access an object that does not exist, our Python code will hit an exception and return an HTTP 500 Server Error response. This is undesirable because it looks as though our server has failed to handle the request correctly.

4. At the end of the activity, you should be able to click the **Reviews** button on the book list page and get a detailed view of the book. The detailed view should have all the details displayed, as shown in the following screenshot:

활동 3.01 – 책 세부 정보 보기 구현하기

이 활동에서는 책의 세부 정보인 제목, 출판사, 출판 날짜 및 전체 평점을 표시하기 위해 새로운 보기, 템플릿 및 URL 구성을 구현합니다. 이러한 세부 정보 외에도 페이지에는 모든 리뷰 댓글이 표시되어야 하며, 댓글 작성자의 이름과 댓글이 작성된 날짜 및 (해당되는 경우) 수정된 날짜를 지정해야 합니다. 다음 단계는 이 활동을 완료하는 데 도움이 될 것입니다:

1. 기본 템플릿을 확장하는 책 세부 정보 엔드포인트를 만듭니다.
2. 특정 책의 기본 키를 인수로 받아 책의 세부 정보와 관련된 리뷰를 나열하는 HTML 페이지를 반환하는 책 세부 정보 보기를 만듭니다.
3. `urls.py`에 필요한 URL 구성을 추가합니다. 책 세부 정보 보기 URL은 `http://0.0.0.0:8000/books/1/` 여야 합니다 (여기서 1은 접근하는 책의 ID를 나타냅니다). 주어진 기본 키로 책을 검색하기 위해 `get_object_or_404` 메서드를 사용할 수 있습니다.

메모

`get_object_or_404` 함수는 기본 키를 기반으로 인스턴스를 검색하는 데 유용한 단축키입니다. Chapter 2에서 설명한 `.get()` 메서드를 사용하여 이 작업을 수행할 수도 있습니다: `Book.objects.get(pk=pk)`. 그러나 `get_object_or_404`는 객체가 존재하지 않을 경우 HTTP 404 Not Found 응답을 반환하는 추가 이점이 있습니다. 단순히 `get()`을 사용하고 누군가 존재하지 않는 객체에 접근하려고 하면, 우리의 Python 코드는 예외를 발생시키고 HTTP 500 Server Error 응답을 반환하게 됩니다. 이는 요청을 올바르게 처리하지 못한 것처럼 보이기 때문에 바람직하지 않습니다.

4. `idx_4366a7fc` 활동이 끝나면, 책 목록 페이지에서 리뷰 버튼을 클릭하여 책의 자세한 보기를 얻을 수 있어야 합니다. 자세한 보기에는 다음 스크린샷에 표시된 모든 세부 정보가 표시되어야 합니다:

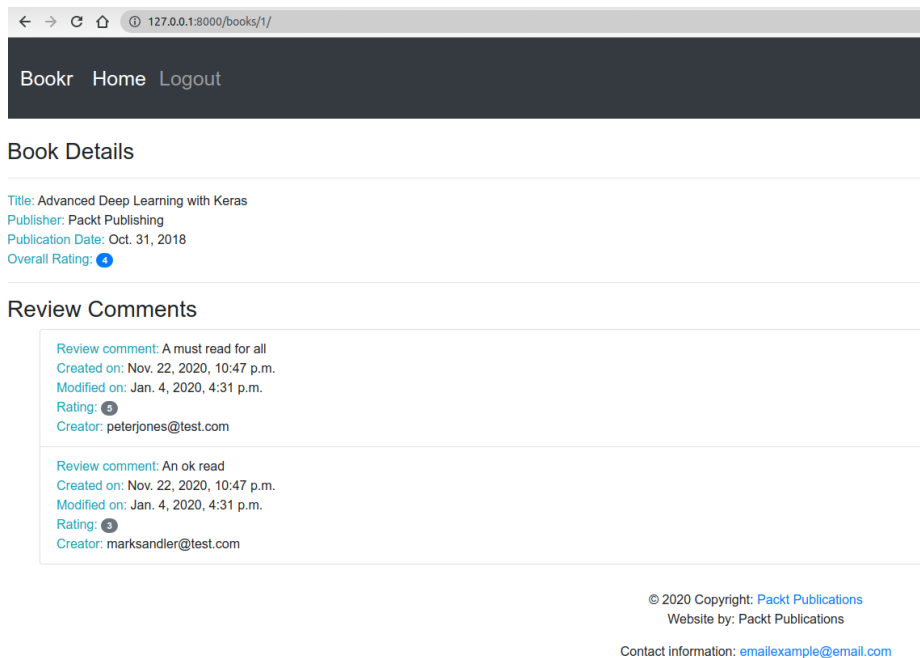


Figure 3.8 – Page displaying the book details

So far in this section, we have learned how to use Django templates to present static information to the user, template language to display the application's dynamic data, and Bootstrap to add styling to the information and make it presentable to users.

Summary

This chapter covered the core infrastructure required to handle an HTTP request to our website. The request is first mapped via URL patterns to an appropriate view. Parameters from the URL are also passed into the view to specify the object displayed on the page.

The view is responsible for compiling any necessary information to display on the website, and then passes this dictionary through to a template, which renders the information as HTML code that can be returned as a response to the user. We covered both class- and function-based views and learned about the Django template language and template inheritance. We created two new pages for the book review application, one displaying all the books present and the other being the book details view page.

In the next chapter, we will learn about Django admin and superuser, registering models, and performing **create, read, update, and delete (CRUD)** operations using the admin site.

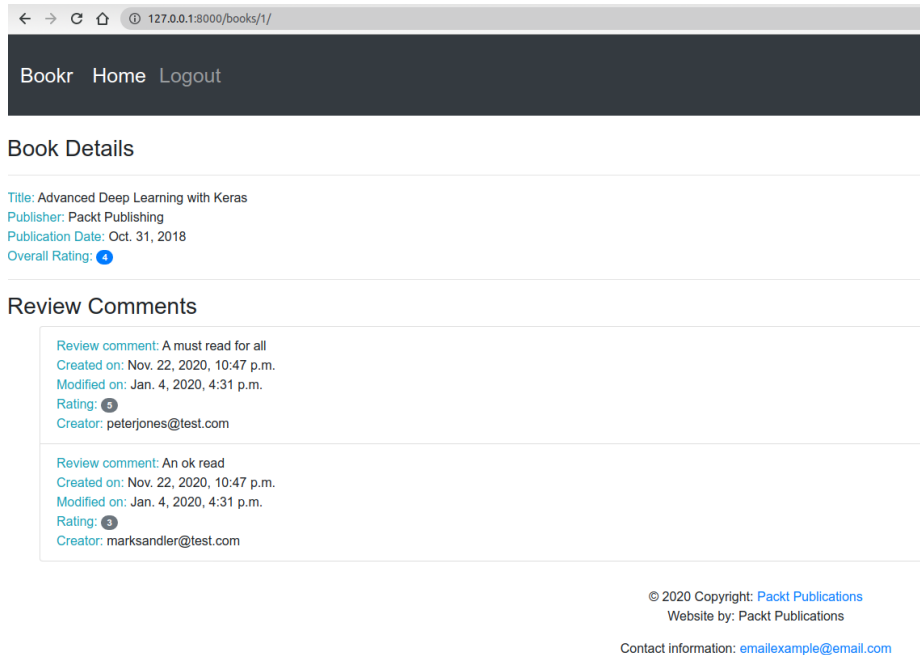


그림 3.8 - 책 세부 정보를 표시하는 페이지

지금까지 이 섹션에서는 장고 템플릿을 사용하여 사용자에게 정적 정보를 제공하는 방법, 애플리케이션의 동적 데이터를 표시하기 위한 템플릿 언어, 그리고 정보를 스타일링하고 사용자에게 보기 좋게 만들기 위한 부트스트랩을 사용하는 방법을 배웠습니다.

요약

이 장에서는 우리 웹사이트에 대한 HTTP 요청을 처리하는 데 필요한 핵심 인프라를 다루었습니다. 요청은 먼저 URL 패턴을 통해 적절한 뷰에 매핑됩니다. URL의 매개변수는 페이지에 표시되는 객체를 지정하기 위해 뷰로 전달됩니다.

뷰는 웹사이트에 표시할 필요한 정보를 컴파일하는 책임이 있으며, 그런 다음 이 사전을 템플릿으로 전달하여 정보를 HTML 코드로 렌더링하고 사용자에게 응답으로 반환할 수 있습니다. 우리는 클래스 기반 및 함수 기반 뷰를 모두 다루었고 장고 템플릿 언어와 템플릿 상속에 대해 배웠습니다. 우리는 책 리뷰 애플리케이션을 위해 두 개의 새로운 페이지를 만들었으며, 하나는 현재 있는 모든 책을 표시하고 다른 하나는 책 세부 정보 뷰 페이지입니다.

다음 장에서는 장고 관리자 및 슈퍼유저, 모델 등록, 그리고 관리자 사이트를 사용하여 생성, 읽기, 업데이트 및 삭제(CRUD) 작업을 수행하는 방법에 대해 배울 것입니다.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

이 책의 PDF 사본, 코드 번들 등을 받으세요

QR 코드를 스캔하세요 (또는 packtpub.com/unlock으로 가세요). 이 책의 이름으로 검색하고, 판을 확인한 후 페이지의 단계를 따르세요.



UNLOCK NOW

참고: 인보이스를 준비하세요. Packt 웹사이트에서 직접 구매한 경우 인보이스가 필요하지 않습니다.

4

An Introduction to Django Admin

This chapter introduces you to the basic functionality of the Django admin app. You will start by creating superuser accounts for the Bookr app, before moving on to executing **Create, Read, Update, and Delete (CRUD)** operations with the admin app. You will learn how to integrate your Django app with the admin app, and you'll also look at the behavior of `ForeignKey` in the admin app. At the end of this chapter, you will see how you can customize the admin app according to a unique set of preferences by subclassing the `AdminSite` and `ModelAdmin` classes to make its interface more intuitive and user-friendly.

In this chapter, we will cover the following topics:

- Introduce the Django admin app
- Creating a superuser account
- CRUD operations using the Django admin app
- Managing Django users and groups
- Registering models with the admin app
- Customizing the admin interface

By the end of this chapter, you will be able to create superuser accounts, user accounts, and groups, and assign permissions to them. You will be able to register your models with the admin app, customize the functionality of change list pages and forms for administering your models, as well as customize the global properties of the admin app.

Technical requirements

Find the solution in the `Chapter04` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

4

장고 관리자 소개

이 장에서는 장고 관리자 앱의 기본 기능을 소개합니다. Bookr 앱을 위한 슈퍼유저 계정을 생성한 후, `idx_0f475055admin` 앱을 사용하여 생성(Create), 읽기(Read), 업데이트(Update), 삭제>Delete) (CRUD) 작업을 실행하는 방법으로 넘어갑니다. 장고 앱을 관리자 앱과 통합하는 방법을 배우고, 관리자 앱에서 ForeignKey의 동작을 살펴볼 것입니다. 이 장의 끝에서는 AdminSite와 ModelAdmin 클래스를 서브클래싱하여 관리자 앱을 고유한 선호 세트에 따라 사용자 정의하여 인터페이스를 더 직관적이고 사용자 친화적으로 만드는 방법을 볼 수 있습니다.

이 장에서는 다음 주제를 다룰 것입니다: 장고

- 관리자 앱 소개 슈퍼유저 계정 생성
- 장고 관리자 앱을 사용한 CRUD 작업 장고
- 사용자 및 그룹 관리 관리자 앱에 모델 등록
- 관리 인터페이스 사용자 정의

이 장이 끝날 무렵, 여러분은 슈퍼유저 계정, 사용자 계정 및 그룹을 생성하고, 이들에게 권한을 부여할 수 있게 될 것입니다. 여러분은 모델을 admin 앱에 등록하고, 모델 관리를 위한 변경 목록 페이지와 양식의 기능을 사용자 정의하며, admin 앱의 전역 속성을 사용자 정의할 수 있게 될 것입니다.

기술 요구 사항

이 책의 GitHub 리포지토리의 Chapter04 폴더에서 솔루션을 찾으세요. 리포지토리 링크에 접근하려면, 서문에 있는 예제 코드 파일 다운로드 섹션의 단계를 따르세요.

Introducing the Django admin app

When developing an app, there is often a need to populate it with data and then alter that data. We have already seen, in *Chapter 2*, how this can be done in the command line using the Python `manage.py` shell. In *Chapter 3*, we learned how to develop a web form interface for our model using Django's views and templates. But neither of these approaches is ideal for administering the data from the classes in `reviews/models.py`. Using the shell to manage data is too technical for non-programmers, and building individual web pages would be a laborious process, as it would see us repeating the same view logic and very similar template features for each table in the model. Fortunately, a solution to this problem was devised in the early days of Django when it was still being developed.

Django admin is actually written as a Django app. It offers an intuitively rendered web interface to give administrative access to the model data. The admin interface is designed to be used by the website administrators. It is not intended to be used by non-privileged users who interact with the site. In our case of a book review system, the general population of book reviewers will never encounter the admin app. They will see the app pages, like those we built with views and templates in *Chapter 3*, and will write their reviews on the pages.

Also, while developers put a lot of effort into creating a simple and inviting web interface for general users, the admin interface, aimed at administrative users, maintains a utilitarian feel that typically displays the intricacies of the models that make up the application and are registered with the admin. It may have escaped your attention, but you already have an admin app in your Bookr project. Look at the list of installed apps in `bookr/bookr/settings.py`:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    ...  
]
```

Now, look at the URL patterns in `bookr/bookr/urls.py`:

```
urlpatterns = [  
    path("admin/", admin.site.urls),  
    ...  
]
```

If we put this path into our browser, we can see that the link to the admin app on the development server is `http://127.0.0.1:8000/admin/`. Before we make use of it, though, we need to create a superuser through the command line.

장고 관리 앱 소개

앱을 개발할 때, 데이터로 채우고 그 데이터를 변경해야 할 필요가 종종 있습니다. 우리는 이미 2장에서 Python manage.py shell을 사용하여 명령줄에서 이를 수행하는 방법을 보았습니다. 3장에서는 Django의 뷰와 템플릿을 사용하여 모델을 위한 웹 폼 인터페이스를 개발하는 방법을 배웠습니다. 그러나 이러한 접근 방식은 reviews/models.py의 클래스에서 데이터를 관리하는 데 이상적이지 않습니다. 비프로그래머에게 데이터 관리를 위해 셸을 사용하는 것은 너무 기술적이며, 개별 웹 페이지를 만드는 것은 힘든 과정이 될 것입니다. 왜냐하면 모델의 각 테이블에 대해 동일한 뷰 로직과 매우 유사한 템플릿 기능을 반복해야 하기 때문입니다. 다행히도, Django가 아직 개발 중이던 초기 시절에 이 문제에 대한 해결책이 고안되었습니다.

장고 관리자는 실제로 장고 앱으로 작성되었습니다. 이는 모델 데이터에 대한 관리 액세스를 제공하기 위해 직관적으로 렌더링된 웹 인터페이스를 제공합니다. 관리 인터페이스는 웹사이트 관리자가 사용하도록 설계되었습니다. 비특권 사용자가 사이트와 상호 작용하는 데 사용되도록 의도되지 않았습니다. 책 리뷰 시스템의 경우, 일반적인 책 리뷰어들은 관리 앱을 결코 접하지 않을 것입니다. 그들은 3장에서 뷰와 템플릿으로 구축한 것과 같은 앱 페이지를 보고 그 페이지에 리뷰를 작성할 것입니다.

또한, 개발자들은 일반 사용자를 위한 간단하고 매력적인 웹 인터페이스를 만드는 데 많은 노력을 기울였지만, 관리 사용자를 대상으로 하는 관리 인터페이스는 일반적으로 애플리케이션을 구성하고 관리에 등록된 모델의 복잡성을 표시하는 실용적인 느낌을 유지합니다. 당신의 주목을 받지 못했을 수도 있지만, 당신은 이미 Bookr 프로젝트에 관리 앱을 가지고 있습니다. bookr/bookr/settings.py에서 설치된 앱 목록을 확인해 보세요:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    ...
]
```

이제, bookr/bookr/urls.py의 URL 패턴을 살펴보세요:

```
urlpatterns = [
    path("admin/", admin.site.urls),
    ...
]
```

이 경로를 브라우저에 입력하면 개발 서버의 관리자 앱 링크가 <http://127.0.0.1:8000/admin/>임을 확인할 수 있습니다. 하지만 사용하기 전에 명령줄을 통해 슈퍼유저를 생성해야 합니다.

Creating a superuser account

Our Bookr application has just found a new user. Her name is Alice, and she wants to start adding her reviews right away. Bob, who is already using Bookr, has just informed us that his profile seems incomplete and needs to be updated. Eve no longer wants to use the application and wants their account to be deleted. For security reasons, we do not want just any user performing these tasks for us. That's why we need to create a **superuser** with elevated privileges. Let's start by doing just that.

In Django's authorization model, a superuser is one with all permissions. A user with the staff flag set is able to log in to the admin site, but then their specific permissions dictate what they can see and do with the models registered. We will examine this later in the chapter and learn more about this authorization model in *Chapter 9*.

We can create a superuser by using the `manage.py` script, which we explored in earlier chapters. Again, we need to be in the project directory when we enter the command. We will use the `createsuperuser` subcommand by entering the following command in the command line:

```
python manage.py createsuperuser
```

Note

In this chapter, we will use email addresses that fall under the `example.com` domain. This follows an established convention to use this reserved domain for testing and documentation. You could use your own email addresses if you prefer.

Let's go ahead and create our superuser.

Exercise 4.01 – Creating a superuser account

In this exercise, you will create a superuser account that lets the user log in to the admin site. This functionality will also be used in the upcoming exercises to implement changes that only a superuser can. The following steps will help you complete this exercise:

1. Enter the following command to create a superuser:

```
python manage.py createsuperuser
```

슈퍼유저 계정 생성

우리의 Bookr 애플리케이션 idx_e79ec83ahas 새로운 사용자를 찾았습니다. 그녀의 이름은 엘리스이며, 그녀는 즉시 리뷰를 추가하고 싶어합니다. 이미 Bookr를 사용하고 있는 밥은 그의 프로필이 불완전해 보이며 업데이트가 필요하다고 방금 알려주었습니다. 이브는 더 이상 애플리케이션을 사용하고 싶지 않으며, 그녀의 계정을 삭제하길 원합니다. 보안상의 이유로, 우리는 아무 사용자에게 이러한 작업을 수행하게 하고 싶지 않습니다. 그래서 우리는 권한이 상승된 슈퍼유저를 생성해야 합니다. 바로 그것부터 시작해 봅시다.

장고의 권한 모델에서 슈퍼유저는 모든 권한을 가진 사용자입니다. 직원 플래그가 설정된 사용자는 관리자 사이트에 로그인할 수 있지만, 그들의 특정 권한이 등록된 모델로 무엇을 보고 할 수 있는지를 결정합니다. 우리는 이 장에서 이를 나중에 살펴보고 9장에서 이 권한 모델에 대해 더 배울 것입니다.

우리는 이전 장에서 살펴본 `manage.py` 스크립트를 사용하여 슈퍼유저를 생성할 수 있습니다. 다시 말해, 명령어를 입력할 때 프로젝트 디렉토리에 있어야 합니다. 우리는 다음 명령어를 입력하여 `createsuperuser` 하위 명령을 사용할 것입니다:

```
python manage.py createsuperuser
```

참고

이 장에서는 `example.com` 도메인에 해당하는 이메일 주소를 사용할 것입니다. 이는 테스트 및 문서화를 위해 이 예약된 도메인을 사용하는 확립된 관례를 따릅니다. 원하신다면 자신의 이메일 주소를 사용할 수 있습니다.

슈퍼유저를 생성해 봅시다.

연습 4.01 – 슈퍼유저 계정 만들기

이 연습에서는 사용자가 관리자 사이트에 로그인할 수 있는 슈퍼유저 계정을 생성합니다. 이 기능은 슈퍼유저만이 수행할 수 있는 변경 사항을 구현하기 위해 다가오는 연습에서도 사용될 것입니다. 다음 단계는 이 연습을 완료하는 데 도움이 될 것입니다:

1. 슈퍼유저를 생성하기 위해 다음 명령어를 입력하세요:

```
python manage.py createsuperuser
```

On executing the preceding command, you will be prompted to create a superuser. This command will prompt you for a superuser name, an optional email address, and a password.

2. Add the username and email for the superuser as follows. Here, we enter `bookradmin` (highlighted) at the prompt and press the *Enter* key. Similarly, at the next prompt, which asks you to enter your email address, you can add `bookradmin@example.com` (highlighted). Press the *Enter* key to continue:

```
Username (leave blank to use 'django'): bookradmin
Email address: bookradmin@example.com
Password:
```

This will assign the name `bookradmin` to the superuser. Note that you won't see any output immediately.

3. The next prompt in the shell is for your password. Add a strong password and press the *Enter* key to confirm it once again:

```
Password:
Password (again):
```

You should see the following message on your screen:

```
Superuser created successfully.
```

Note that the password is validated as per the following criteria:

- It cannot be among the 20 thousand most common passwords
- It should have a minimum of eight characters
- It cannot be only numerical characters
- It cannot be derived from the username, first name, last name, or email address of the user

These criteria are determined by `AUTH_PASSWORD_VALIDATORS` in the `settings.py` file. It is possible to change the required password strength by configuring these validators or writing custom validators.

With this, you have created a superuser named `bookradmin` who can log in to the admin app. The following screenshot shows how this looks in the shell:

이전 명령을 실행하면 슈퍼유저를 생성하라는 메시지가 표시됩니다. 이 명령은 슈퍼유저 이름, 선택적 이메일 주소 및 비밀번호를 입력하라는 메시지를 표시합니다.

- 다음과 같이 슈퍼유저의 사용자 이름과 이메일을 추가합니다. 여기서 우리는 프롬프트에서 bookradmin(강조됨)을 입력하고 Enter 키를 누릅니다. 마찬가지로, 다음 프롬프트에서 이메일 주소를 입력하라는 idx_fdb2b38b의 요청에 대해 bookradmin@example.com(강조됨)을 추가할 수 있습니다. 계속하려면 Enter 키를 누르세요:

```
사용자 이름 ('django'를 사용하려면 비워 두세요):
bookradmin이메일 주소: bookradmin@example.com

비밀번호:
```

이렇게 하면 슈퍼유저에게 bookradmin이라는 이름이 할당됩니다. 즉시 출력이 표시되지 않는다는 점에 유의하세요.

- 셸의 다음 프롬프트는 비밀번호를 입력하라는 것입니다. 강력한 비밀번호를 추가하고 Enter 키를 눌러 다시 확인하세요:

```
비밀번호:
비밀번호 (다시 입력):
```

다음 메시지가 화면에 표시되어야 합니다:

```
슈퍼유저가 성공적으로 생성되었습니다.
```

비밀번호는 다음 기준에 따라 검증됩니다: 20,000개 가장 일반적인 비밀번호 중 하나일 수 없습니다. 최

- 소 8자 이상이어야 합니다.

- 숫자만으로 구성될 수 없습니다.
- 사용자의 사용자 이름, 이름, 성 또는 이메일 주소에서 파생될 수 없습니다.

이 기준은 settings.py 파일의 AUTH_PASSWORD_VALIDATORS에 의해 결정됩니다. 이러한 검증기를 구성하거나 사용자 정의 검증기를 작성하여 필요한 비밀번호 강도를 변경할 수 있습니다.

이로써, youidx_28d34edb는 admin 앱에 로그인할 수 있는 superuser인 bookradmin을 생성했습니다. 다음 스크린샷은 셸에서 어떻게 보이는지를 보여줍니다:


```
> python manage.py createsuperuser
Username (leave blank to use 'django'): bookradmin
Email address: bookradmin@example.com
Password:
Password (again):
Superuser created successfully.
>
```

Figure 4.1 – Creating a superuser

4. Visit the admin app at <http://127.0.0.1:8000/admin> and log in with the superuser account that you have created:

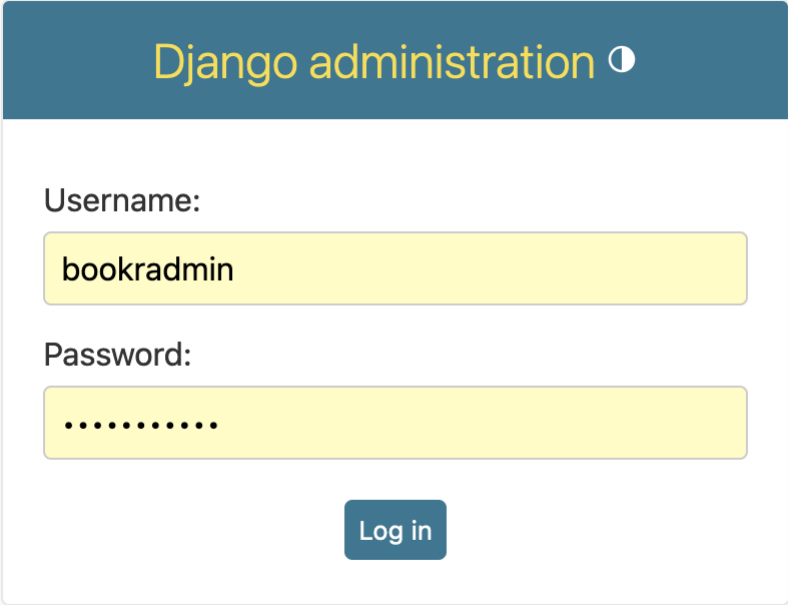
The image shows a web browser window displaying the Django administration login page. At the top, there is a dark blue header with the text "Django administration" in yellow, followed by a small circular icon. Below the header, the page has a white background. There are two input fields: "Username:" with the text "bookradmin" entered, and "Password:" with a series of dots indicating a masked password. Below these fields is a blue button with the text "Log in" in white.

Figure 4.2 – The Django administration login form

In this exercise, you created a superuser account that we will be using for the rest of this chapter to assign or remove privileges as needed.

With a superuser account created, we are now able to log into the Django admin app and learn about the CRUD facilities that it provides.

```
> python manage.py createsuperuser
Username (leave blank to use 'django'): bookradmin
Email address: bookradmin@example.com
Password:
Password (again):
Superuser created successfully.
>
```

그림 4.1 - 슈퍼유저 생성

4. <http://127.0.0.1:8000/admin>에서 관리자 앱을 방문하고 생성한 슈퍼유저 계정으로 로그인하세요:

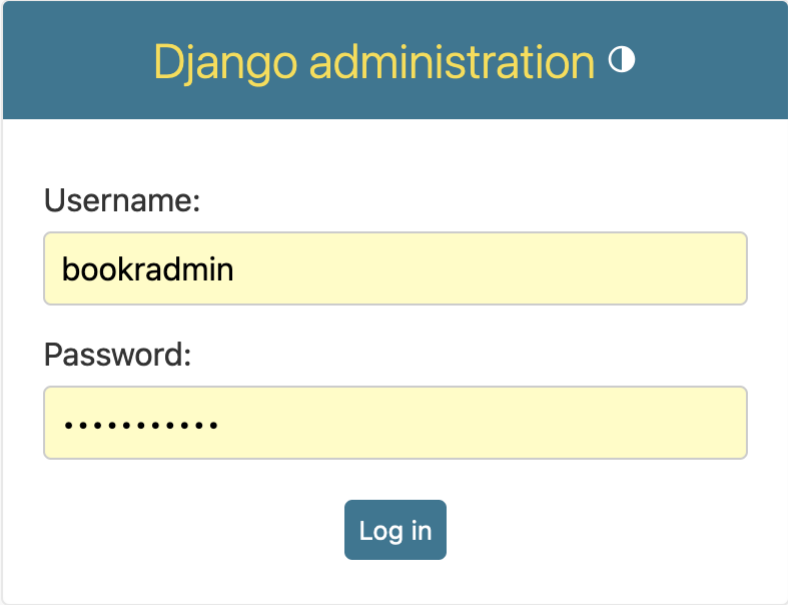


그림 4.2 - 장고 관리 로그인 양식

이 연습에서는 `idx_e3adfcdd` 권한을 할당하거나 필요에 따라 제거하기 위해 이 장의 나머지 부분에서 사용할 슈퍼유저 계정을 생성했습니다.

슈퍼유저 계정이 생성되었으므로 이제 장고 관리자 앱에 로그인하고 제공하는 CRUD 기능에 대해 배울 수 있습니다.

CRUD operations using the Django admin app

Let's get back to the requests we got from Bob, Alice, and Eve. As a superuser, your tasks will involve creating, updating, retrieving, and deleting various user accounts, reviews, and title names. This set of activities is collectively termed CRUD. CRUD operations are central to the behavior of the admin app. It turns out that the admin app is already aware of the models from another Django app, Authentication and Authorization – referenced in `INSTALLED_APPS` as `'django.contrib.auth'`. When logging in to `http://127.0.0.1:8000/admin/`, we are presented with the models from the authorization application, as shown in *Figure 4.3*:

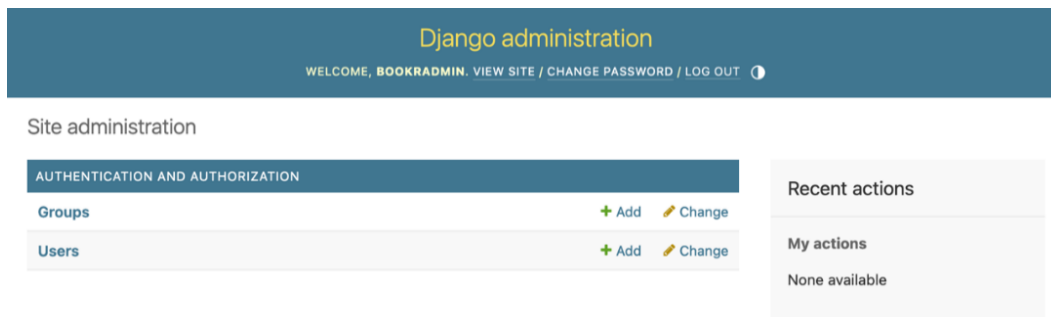


Figure 4.3 – The Django administration window

When the admin app is initialized, it calls its `autodiscover()` method to detect whether any other installed apps contain an admin module. If so, these admin models are imported. In our case, it has discovered `'django.contrib.auth.admin'`. Now that the modules are imported and our superuser account is ready, let's start working on the requests from Bob, Alice, and Eve.

Create

Before Alice starts writing her reviews, we need to create an account for her through the admin app. Once that is done, we can look at the administration access levels we can assign to her. To create a user, we need to click the **+ Add** option next to **Users** (refer to *Figure 4.3*) and fill out the form, as shown in *Figure 4.4*.

Note

We don't want any random user to have access to the Bookr users' accounts. Therefore, it is imperative that we choose strong, secure passwords.

Django 관리 앱을 사용한 CRUD 작업

Bob, Alice, Eve로부터 받은 요청으로 돌아가 보겠습니다. 슈퍼유저로서 귀하의 작업은 다양한 사용자 계정, 리뷰 및 제목 이름을 생성, 업데이트, 검색 및 삭제하는 것입니다. 이 활동 세트를 CRUD라고 합니다. CRUD 작업은 관리 앱의 동작에 중심적입니다. 관리 앱은 이미 다른 Django 앱인 인증 및 권한 부여의 모델을 인식하고 있습니다. 이는 INSTALLED_APPS에서 'django.contrib.auth'로 참조됩니다. <http://127.0.0.1:8000/admin/>에 로그인하면 권한 부여 애플리케이션의 모델이 표시됩니다. 이는 그림 4.3에 나와 있습니다:

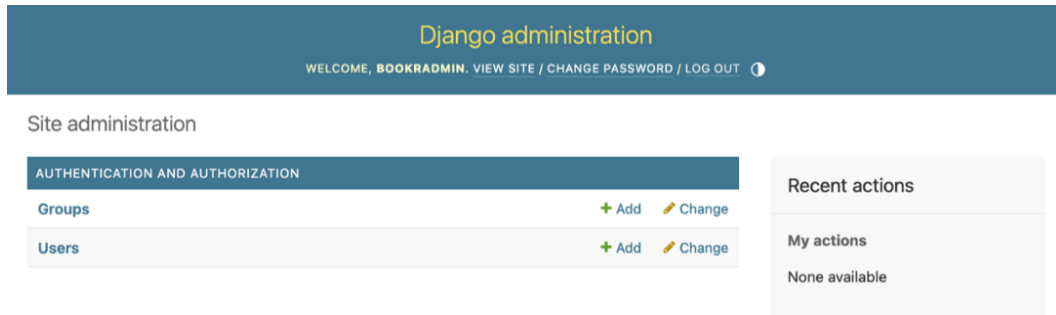


그림 4.3 - Django 관리 창

관리 앱이 초기화되면, 다른 설치된 앱에 관리 모듈이 포함되어 있는지 감지하기 위해 autodiscover() 메서드를 호출합니다. 그렇다면 이러한 관리 모델이 가져옵니다. 우리의 경우, 'django.contrib.auth.admin'을 발견했습니다. 이제 모듈이 가져와졌고 우리의 idx_f0f461a5 슈퍼유저 계정이 준비되었으므로, Bob, Alice, Eve의 요청 작업을 시작해 보겠습니다.

생성

Alice가 리뷰를 작성하기 시작하기 전에, 우리는 관리 앱을 통해 그녀의 계정을 생성해야 합니다. 그것이 완료되면, 그녀에게 할당할 수 있는 관리 액세스 수준을 살펴볼 수 있습니다. 사용자를 생성하려면, 사용자 옆의 + 추가 옵션을 클릭하고(그림 4.3 참조) 그림 4.4에 나와 있는 대로 양식을 작성해야 합니다.

참고

무작위 사용자가 Bookr 사용자 계정에 접근하는 것을 원하지 않습니다. 따라서 강력하고 안전한 비밀번호를 선택하는 것이 중요합니다.

Home > Authentication and Authorization > Users > Add user

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Users [+ Add](#)

Add user

First, enter a username and password. Then, you'll be able to edit more user options.

Username:
 Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password-based authentication: ☒ Enabled ☐ Disabled
 Whether the user will be able to authenticate using a password or not. If disabled, they may still be able to authenticate using other backends, such as Single Sign-On or LDAP.

Password:
 Your password can't be too similar to your other personal information.
 Your password must contain at least 8 characters.
 Your password can't be a commonly used password.
 Your password can't be entirely numeric.

Password confirmation:
 Enter the same password as before, for verification.

[SAVE](#) [Save and add another](#) [Save and continue editing](#)

Figure 4.4 – The Add user page

There are three buttons at the bottom of the form:

- **Save and add another:** This creates the user and renders the same **Add user** page again, with blank fields.
- **Save and continue editing:** This creates the user and loads the **Change user** page. The **Change user** page lets you add additional information that wasn't present on the **Add user** page, such as **First name**, **Last name**, and more (see *Figure 4.5*). Note that **Password** does not have an editable field on the form. Instead, it shows information about the hashing technique that it is stored with, in addition to a link to a separate **Reset password** form:

Home > Authentication and Authorization > Users > Add user

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Users [+ Add](#)

Add user

First, enter a username and password. Then, you'll be able to edit more user options.

Username:
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password-based authentication: ☒ Enabled ☐ Disabled
Whether the user will be able to authenticate using a password or not. If disabled, they may still be able to authenticate using other backends, such as Single Sign-On or LDAP.

Password:
Your password can't be too similar to your other personal information.
 Your password must contain at least 8 characters.
 Your password can't be a commonly used password.
 Your password can't be entirely numeric.

Password confirmation:
Enter the same password as before, for verification.

[SAVE](#) [Save and add another](#) [Save and continue editing](#)

그림 4.4 - 사용자 추가 페이지

양식 하단에 세 개의 `idx_7c94f834` 버튼이 있습니다:

- 저장하고 다른 추가: 이것은 사용자를 생성하고 빈 필드가 있는 동일한 사용자 추가 페이지를 다시 렌더링합니다.
- 저장하고 계속 편집: 이것은 사용자를 생성하고 사용자 변경 페이지를 로드합니다. 사용자 변경 페이지에서는 사용자 추가 페이지에 없었던 추가 정보를 추가할 수 있습니다. 예를 들어 이름, 성 및 기타 정보(그림 4.5 참조). 비밀번호는 양식에서 편집 가능한 필드가 없음을 유의하십시오. 대신, 비밀번호는 저장된 해싱 기술에 대한 정보와 별도의 비밀번호 재설정 양식에 대한 링크를 보여줍니다:

✓ The user "alice" was added successfully. You may edit it again below.

Change user

HISTORY

alice

Username:

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password: algorithm: pbkdf2_sha256 iterations: 870000 salt: 0QXJFj*****
hash: HM/vXt*****

Reset password

Raw passwords are not stored, so there is no way to see the user's password.

Personal info

First name:

Last name:

Email address:

Permissions

☒ **Active**
Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☐ **Staff status**
Designates whether the user can log into this admin site.

☐ **Superuser status**
Designates that this user has all permissions without explicitly assigning them.

Figure 4.5 – The Change user page presented after clicking Save and continuing editing

- **SAVE:** This creates the user and lets the user navigate to the **Select user to change** list page, as depicted in Figure 4.6.

Activity 4.01 – Creating a user account

We have now seen how to create user accounts through the Django admin interface.

It is time to put your knowledge into practice to create an account for a new user, David Green, using the email address `david.green@example.com`.

As was the case with Alice White's account, just set this one as an active user without ticking the **Staff status** or **Superuser status** boxes.

Now that you have created a user through the Django admin interface, we can examine the other three CRUD operations: retrieve, update, and delete.

✓ The user "alice" was added successfully. You may edit it again below.

Change user

alice

HISTORY

Username:

alice

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

algorithm: pbkdf2_sha256 iterations: 870000 salt: 0QXJFj*****
hash: HM/vXt*****

Reset password

Raw passwords are not stored, so there is no way to see the user's password.

Personal info

First name:

Alice

Last name:

White

Email address:

alice.white@example.com

Permissions

☒ Active

Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☐ Staff status

Designates whether the user can log into this admin site.

☐ Superuser status

Designates that this user has all permissions without explicitly assigning them.

그림 4.5 - 저장 후 편집을 계속하기 위해 클릭한 후 표시되는 사용자 변경 페이지

- 저장: 이것은 사용자를 생성하고 사용자가 그림 4.6에 묘사된 대로 변경할 사용자를 선택하는 목록 페이지로 이동할 수 있게 합니다.

활동 4.01 – 사용자 계정 만들기

이제 Django 관리 인터페이스를 통해 사용자 계정을 만드는 방법을 보았습니다.

이제 지식을 실천에 옮겨 새로운 사용자 David Green의 계정을 생성할 차례입니다. 이메일 주소는 david.green@example.com입니다.

idx_6dbab7feAlice White의 계정과 마찬가지로, 이 계정도 Staff 상태 또는 Superuser 상태 상자를 선택하지 않고 활성 사용자로 설정하십시오.

Django 관리 인터페이스를 통해 사용자를 생성했으므로, 이제 나머지 세 가지 CRUD 작업인 검색, 업데이트 및 삭제를 살펴볼 수 있습니다.

Retrieve

The administrative tasks need to be divided among some users, and for this, the admin (the person with the superuser account) would like to view those users whose email addresses end with `n@example.com` and assign the tasks to these users. This is where the **retrieve** functionality can come in handy. After we have clicked the **SAVE** button on the **Add user** page (refer to *Figure 4.4*), we are taken to the **Select user to change** list page (as shown in *Figure 4.6*), which carries out the retrieve operation. Note that the **Create** form is also reachable by clicking on the **ADD USER +** button on the **Select user to change** list page. So, after we have added a few more users, the change list will look something like this:

Select user to change

ADD USER +

Q Search

Action: Go 0 of 6 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	alice	alice.white@example.com	Alice	White	✗
☐	bob	bob.black@example.com	Bob	Black	✗
☐	bookradmin	bookradmin@example.com	-	-	✓
☐	carol	carol.brown@example.com	Carol	Brown	✗
☐	david	david.green@example.com	David	Green	✗
☐	eve	eve.pink@example.com	Eve	Pink	✗

6 users

FILTER

Show counts

By staff status

All
Yes
No

By superuser status

All
Yes
No

By active

Figure 4.6 – The Select user to change page

At the top of the form is a **Search** bar that searches the content, such as username, email address, and first and last names of users. On the right-hand side is a **FILTER** panel that narrows down the selection based on the values of **By staff status**, **By superuser status**, and **By active**. In *Figure 4.7*, you can see what happens when we search the `n@example.com` string. This will return only the names of the users whose email addresses consist of a username ending with an `n` and a domain starting with `example.com`. We can only see three users with email addresses matching this requirement – `bookradmin@example.com`, `carol.brown@example.com`, and `david.green@example.com`:

가져오기

관리 작업은 일부 사용자 간에 나누어져야 하며, 이를 위해 관리자는(슈퍼유저 계정을 가진 사람) n@example.com으로 끝나는 이메일 주소를 가진 사용자를 보고 이 사용자에게 작업을 할당하고자 합니다. 여기서 retrieve 기능이 유용하게 사용될 수 있습니다. 사용자 추가 페이지에서 SAVE 버튼을 클릭한 후(그림 4.4 참조), 우리는 변경할 사용자를 선택하는 목록 페이지(그림 4.6에 표시됨)로 이동하게 되며, 여기서 retrieve 작업이 수행됩니다. 선택한 사용자 변경 목록 페이지에서 ADD USER + 버튼을 클릭하면 Create 양식에도 접근할 수 있습니다. 따라서 몇 명의 사용자를 추가한 후 변경 목록은 다음과 같이 보일 것입니다:

Select user to change

ADD USER +

Search

Action: ----- Go 0 of 6 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	alice	alice.white@example.com	Alice	White	✗
☐	bob	bob.black@example.com	Bob	Black	✗
☐	bookradmin	bookradmin@example.com	-	-	✓
☐	carol	carol.brown@example.com	Carol	Brown	✗
☐	david	david.green@example.com	David	Green	✗
☐	eve	eve.pink@example.com	Eve	Pink	✗

6 users

FILTER

Show counts

By staff status

All
Yes
No

By superuser status

All
Yes
No

By active

그림 4.6 - 변경할 사용자 선택 페이지

양식 상단에는 사용자 이름, 이메일 주소, 이름 및 성과 같은 내용을 검색하는 검색창이 있습니다. 오른쪽에는 직원 상태, 슈퍼유저 상태 및 활성 상태의 값에 따라 선택 범위를 좁히는 필터 패널이 있습니다. 그림 4.7에서 n@example.com 문자열을 검색할 때 발생하는 일을 볼 수 있습니다. 이는 이메일 주소가 n으로 끝나는 사용자 이름과 example.com으로 시작하는 도메인으로 구성된 사용자 이름만 반환합니다. 이 요구 사항에 맞는 이메일 주소를 가진 사용자는 세 명만 볼 수 있습니다 – bookradmin@example.com, carol.brown@example.com, david.green@example.com:

Select user to change

3 results (6 total)

Action:

 0 of 3 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	bookradmin	bookradmin@example.com	-	-	✓
☐	carol	carol.brown@example.com	Carol	Brown	✗
☐	david	david.green@example.com	David	Green	✗

3 users

FILTER

↓ By staff status

All

Yes

No

↓ By superuser status

All

Yes

No

→ By active

Figure 4.7 – Searching for users by a portion of their email address

The retrieve functionality, such as the list page and search bar, allows us to obtain records for update and delete operations.

Update

Remember that Bob wanted his profile to be updated. Let's update Bob's unfinished profile by clicking the **bob** username link in the **Select user to change** list:

Select user to change

0 of 6 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	alice	alice.white@example.com	Alice	White	✗
☒	bob	-	-	-	✗
☐	bookradmin	bookradmin@example.com	-	-	✓
☐	carol	carol.brown@example.com	Carol	Brown	✗
☐	david	david.green@example.com	David	Green	✗
☐	eve	eve.pink@example.com	Eve	Pink	✗

6 users

FILTER

↓ By staff status

All

Yes

No

↓ By superuser status

All

Yes

No

→ By active

→ By groups

Figure 4.8 – Selecting bob from the Select user to change list

This will take us back to the **Change user** form, where the values for **First name**, **Last name**, and **Email address** can be entered:

Select user to change ADD USER +

Q Search 3 results (6 total)

Action: Go 0 of 3 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	bookradmin	bookradmin@example.com	-	-	✓
☐	carol	carol.brown@example.com	Carol	Brown	✗
☐	david	david.green@example.com	David	Green	✗

3 users

FILTER

Show counts

↓ By staff status

All
Yes
No

↓ By superuser status

All
Yes
No

→ By active

그림 4.7 - 이메일 주소의 일부로 사용자를 검색하기

검색 기능, 예를 들어 목록 페이지와 검색 바는 idx_367bff66us가 업데이트 및 삭제 작업을 위한 기록을 얻을 수 있게 해줍니다.

업데이트

밥이 자신의 프로필 idx_4746211b를 업데이트하기를 원했다는 것을 기억하세요. 선택한 사용자 목록에서 밥의 사용자 이름 링크를 클릭하여 밥의 미완성 프로필을 업데이트합니다:

Select user to change ADD USER +

Q Search

Action: Go 0 of 6 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	alice	alice.white@example.com	Alice	White	✗
☐	bob	-	-	-	✗
☐	bookradmin	bookradmin@example.com	-	-	✓
☐	carol	carol.brown@example.com	Carol	Brown	✗
☐	david	david.green@example.com	David	Green	✗
☐	eve	eve.pink@example.com	Eve	Pink	✗

6 users

FILTER

Show counts

↓ By staff status

All
Yes
No

↓ By superuser status

All
Yes
No

→ By active

→ By groups

그림 4.8 - 변경할 사용자를 선택하는 목록에서 밥 선택하기

이것은 우리를 사용자 변경 양식으로 되돌려 보낼 것이며, 여기서 이름, 성 및 이메일 주소의 값을 입력할 수 있습니다:

The screenshot shows the Django Admin interface. At the top, there's a header with 'Django administration' and navigation links: 'WELCOME, BOOKRADMIN. VIEW SITE / CHANGE PASSWORD / LOG OUT'. Below this is a breadcrumb trail: 'Home > Authentication and Authorization > Users > bob'. On the left, there's a sidebar with a search bar and a menu for 'AUTHENTICATION AND AUTHORIZATION' containing 'Groups' and 'Users' (both with '+ Add' links). The main content area is titled 'Change user' and shows the user 'bob'. The 'Username' field is filled with 'bob'. The 'Password' field has an error message: 'Invalid password format or unknown hashing algorithm.' with a 'Reset password' button. Below this is the 'Personal info' section, which is highlighted in blue. It contains three fields: 'First name' (filled with 'Bob'), 'Last name' (filled with 'Black'), and 'Email address' (filled with 'bob.black@example.com').

Figure 4.9 – Adding personal info

As can be seen from *Figure 4.9*, we are adding personal information about Bob here – his name, surname, and email address, specifically.

Another type of update operation is *soft deleting*. The **Active** Boolean property allows us to deactivate a user rather than deleting the entire record and losing all the data that has dependencies on the account. This practice of using a Boolean flag to denote a record as inactive or removed (and subsequently filtering these flagged records out of queries) is referred to as a **soft delete**. Similarly, we can upgrade the user to **Staff status** or **Superuser status** by ticking the respective checkboxes for those:

The screenshot shows the Django administration interface. At the top, there's a blue header with 'Django administration' and links for 'WELCOME, BOOKADMIN', 'VIEW SITE', 'CHANGE PASSWORD', and 'LOG OUT'. Below this is a breadcrumb trail: 'Home > Authentication and Authorization > Users > bob'. The left sidebar has a search bar and a menu with 'AUTHENTICATION AND AUTHORIZATION' and 'Users' (with a '+ Add' link). The main content area is titled 'Change user' and shows details for user 'bob'. It includes a 'Username' field with the value 'bob', a 'Password' field with an error message 'Invalid password format or unknown hashing algorithm.' and a 'Reset password' button, and a 'Personal info' section with fields for 'First name' (Bob), 'Last name' (Black), and 'Email address' (bob.black@example.com). A 'HISTORY' button is also visible.

그림 4.9 - 개인 정보 추가

그림 4.9에서 볼 수 있듯이, 우리는 여기에서 Bob에 대한 개인 정보 – 그의 이름, 성, 이메일 주소를 추가하고 있습니다.

또 다른 유형의 업데이트 작업은 소프트 삭제입니다. Active Boolean 속성은 전체 레코드를 삭제하고 계정에 의존하는 모든 데이터를 잃는 대신 사용자를 비활성화할 수 있게 해줍니다. 레코드를 비활성 상태 또는 제거된 상태로 표시하기 위해 Boolean 플래그를 사용하는 이 관행(그리고 이후 이러한 플래그가 지정된 레코드를 쿼리에서 필터링하는 것)은 소프트 삭제라고 합니다. 마찬가지로, 해당 체크박스를 선택하여 사용자를 직원 상태 또는 슈퍼유저 상태로 업그레이드할 수 있습니다:

Permissions

☒ Active
Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☐ Staff status
Designates whether the user can log into this admin site.

☐ Superuser status
Designates that this user has all permissions without explicitly assigning them.

Figure 4.10 – The Active, Staff status, and Superuser status permissions

Delete

Eve no longer wants to use the Bookr application and has requested that we delete their account. The auth admin caters to this, too. Select a user or user records on the **Select user to change** list page and choose the **Delete selected users** option from the **Action** drop-down menu. Then, hit the **Go** button (Figure 4.11):

Select user to change

ADD USER +

Q

Search

Action:

✓ Delete selected users

1 of 6 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	alice	alice.white@example.com	Alice	White	✗
☐	bob	bob.black@example.com	Bob	Black	✗
☐	bookradmin	bookradmin@example.com	-	-	✓
☐	carol	carol.brown@example.com	Carol	Brown	✗
☐	david	david.green@example.com	David	Green	✗
☒	eve	eve.pink@example.com	Eve	Pink	✗

6 users

FILTER

Show counts

↓ By staff status

All

Yes

No

↓ By superuser status

All

Yes

No

→ By active

→ By groups

Figure 4.11 – Deleting from the Select user to change list page

You will be presented with a confirmation screen and taken back to the **Select user to change** list once you have deleted the object:

Permissions

☒ Active
 Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☐ Staff status
 Designates whether the user can log into this admin site.

☐ Superuser status
 Designates that this user has all permissions without explicitly assigning them.

그림 4.10 - 활성, 직원 상태 및 슈퍼유저 상태 권한

삭제

Eve는 더 이상 Bookr 애플리케이션을 사용하고 싶지 않으며, idx_4e1daaact에 그들의 계정을 삭제해 달라고 요청했습니다. 인증 관리자는 이것도 처리합니다. 사용자 변경 목록 페이지에서 사용자 또는 사용자 기록을 선택하고 작업 드롭다운 메뉴에서 선택한 사용자 삭제 옵션을 선택합니다. 그런 다음, 이동 버튼을 클릭합니다 (그림 4.11):

Select user to change ADD USER +

Search

Action: ✓ Delete selected users 1 of 6 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	alice	alice.white@example.com	Alice	White	✗
☐	bob	bob.black@example.com	Bob	Black	✗
☐	bookradmin	bookradmin@example.com	-	-	✓
☐	carol	carol.brown@example.com	Carol	Brown	✗
☐	david	david.green@example.com	David	Green	✗
☒	eve	eve.pink@example.com	Eve	Pink	✗

6 users

FILTER
 Show counts
 By staff status
 All
 Yes
 No
 By superuser status
 All
 Yes
 No
 By active
 By groups

그림 4.11 - 사용자 변경 목록 페이지에서 삭제하기

객체를 삭제한 후 확인 화면이 표시되고 사용자 변경 목록으로 돌아갑니다:

Are you sure?

Are you sure you want to delete the selected user? All of the following objects and their related items will be deleted:

Summary

- Users: 1
- Reviews: 1

Objects

- User: eve
 - Review: Review object (3)

Yes, I'm sure No, take me back

Figure 4.12 – User deletion confirmation

You will see the following message once the user is deleted:

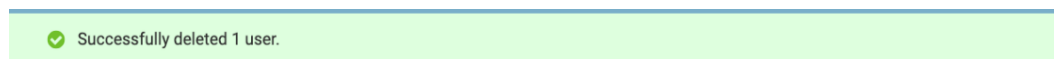


Figure 4.13 – User deletion notification

After that confirmation, you will find that Eve's account no longer exists.

So far, we have learned how we can add a new user, get the details of another user, make changes to the data for a user, and delete a user. These skills helped us cater to Alice, Bob, and Eve's requests. As the number of users of our app grows, managing requests from hundreds of users will eventually become quite difficult. One way around this problem would be to delegate some of the administrative responsibilities to a selected set of users. We'll learn how to do that in the section that follows.

Managing Django users and groups

Django's authentication model consists of users, groups, and permissions. Users can belong to many groups, which is a way of categorizing users. It also streamlines the implementation of permissions by allowing permissions to be assigned to collections of users as well as individuals.

In the *CRUD operations using the Django admin app* section, we saw how a superuser account could cater to Alice, David, and Bob's requests to make modifications to their profiles. It was quite easy to do, and our application seems well-equipped to handle their requests.

What will happen when the number of users grows? Will the admin user be able to manage 100 or 150 users at once? As you can imagine, this can be quite a complicated task. To overcome

Are you sure?

Are you sure you want to delete the selected user? All of the following objects and their related items will be deleted:

Summary

- Users: 1
- Reviews: 1

Objects

- User: eve
- Review: Review object (3)

Yes, I'm sure

No, take me back

그림 4.12 - 사용자 삭제 확인

사용자가 삭제되면 다음 메시지가 표시됩니다:

✓ Successfully deleted 1 user.

그림 4.13 - 사용자 삭제 알림

그 확인 후, Eve의 계정이 더 이상 존재하지 않음을 알 수 있습니다.

지금까지 우리는 새로운 사용자를 추가하고, 다른 사용자의 세부 정보를 가져오고, 사용자 데이터에 변경을 가하고, 사용자를 삭제하는 방법을 배웠습니다. 이러한 기술은 Alice, Bob, Eve의 요청을 처리하는 데 도움이 되었습니다. 우리의 앱 사용자 수가 증가함에 따라, 수백 명의 사용자로부터 요청을 관리하는 것은 결국 꽤 어려워질 것입니다. 이 문제를 해결하는 한 가지 방법은 선택된 사용자 집합에 일부 관리 책임을 위임하는 것입니다. 우리는 다음 섹션에서 이를 수행하는 방법을 배울 것입니다.

장고 사용자 및 그룹 관리

장고의 인증 모델은 사용자, 그룹 및 권한으로 구성됩니다. 사용자 `idx_b3ad4f6acan`은 여러 그룹에 속할 수 있으며, 이는 사용자를 분류하는 방법입니다. 또한 `idx_b0e60864`는 권한을 개인뿐만 아니라 사용자 집합에 할당할 수 있도록 하여 권한 구현을 간소화합니다.

장고 관리자 앱 섹션에서 CRUD 작업을 사용할 때, 슈퍼유저 계정이 Alice, David, Bob의 프로필 수정을 요청하는 데 어떻게 도움이 되었는지 보았습니다. 이는 꽤 쉬웠고, 우리의 애플리케이션은 그들의 요청을 처리하는 데 잘 갖추어져 있는 것 같습니다.

사용자 수가 증가하면 어떤 일이 발생할까요? 관리자가 한 번에 100명 또는 150명의 사용자를 관리할 수 있을까요? 상상할 수 있듯이, 이는 꽤 복잡한 작업이 될 수 있습니다. 극복하기 위해

this, we can give elevated permissions to a certain set of users, and they can help ease the admin's tasks. And that's where groups come in handy. Though we'll learn more about users, groups, and permissions in *Chapter 9*, we can start understanding groups and their functionality by creating a **Help Desk User group** that contains accounts that have access to the admin interface but lack many powerful features, such as the ability to add, edit, or delete groups or to add or delete users.

Exercise 4.02 – Adding and modifying users and groups through the admin app

In this exercise, we will grant a certain level of administrative access to one of our Bookr users, Carol. First, we will define the level of access for a group, and then we will add Carol to the group. This will allow Carol to update user profiles and check user logs. The following steps will help you implement this exercise:

1. Visit the admin interface at `http://127.0.0.1:8000/admin/` and log in as bookradmin using the account set up with the superuser command.
2. In the admin interface, follow the links to **Home | AUTHENTICATION AND AUTHORIZATION | Groups:**

Authentication and Authorization administration

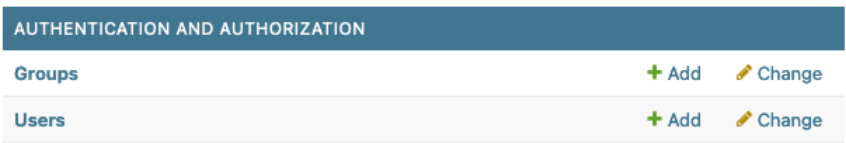


Figure 4.14 – The Groups and Users options on the AUTHENTICATION AND AUTHORIZATION page

3. Use **ADD GROUP +** in the top right-hand corner to add a new group:



Figure 4.15 – Adding a new group

4. Name the group **Help Desk User** and give it the following permissions, as shown in *Figure 4.16*:

- Can view log entry

이를 통해 특정 사용자 집합에 높은 권한을 부여할 수 있으며, 이들이 관리자의 작업을 수월하게 도와줄 수 있습니다. 그리고 여기서 그룹이 유용하게 사용됩니다. 사용자, 그룹 및 권한에 대해 9장에서 더 배우겠지만, 관리 인터페이스에 접근할 수 있지만 그룹을 추가, 수정 또는 삭제하거나 사용자를 추가하거나 삭제할 수 있는 많은 강력한 기능이 부족한 계정을 포함하는 헬프 데스크 사용자 그룹을 생성함으로써 그룹과 그 기능을 이해하기 시작할 수 있습니다.

연습 4.02 – 관리 앱을 통해 사용자 및 그룹 추가 및 수정하기

이 연습에서는 Bookr 사용자 중 한 명인 Carol에게 특정 수준의 관리 접근 권한을 부여할 것입니다. 먼저, 그룹에 대한 접근 수준을 정의한 다음 Carol을 그룹에 추가할 것입니다. 이렇게 하면 Carol이 사용자 프로필을 업데이트하고 사용자 로그를 확인할 수 있습니다. 다음 단계는 이 연습을 구현하는 데 도움이 될 것입니다:

1. <http://127.0.0.1:8000/admin/>에서 관리 인터페이스를 방문하고 superuser 명령으로 설정된 계정을 사용하여 bookradmin으로 로그인합니다.
2. 관리 인터페이스에서 홈 | 인증 및 권한 부여 | 그룹으로 이동합니다:

Authentication and Authorization administration

AUTHENTICATION AND AUTHORIZATION		
Groups	+ Add	Change
Users	+ Add	Change

그림 4.14 - 인증 및 권한 부여 페이지의 그룹 및 사용자 옵션

3. 오른쪽 상단 모서리의 ADD GROUP +를 사용하여 새 그룹을 추가합니다:

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add

Select group to change

Search

0 groups

ADD GROUP +

그림 4.15 - 새 그룹 추가

4. 그룹 이름을 Help Desk User로 지정하고 그림 4.16에 표시된 대로 다음 권한을 부여합니다:

◦ 로그 항목 보기 가능

- Can view permission
- Can change user
- Can view user

Add group

Name:

Permissions:

Available permissions ⓘ

Filter

- Authentication and Authorization | group | Can delete group
- Authentication and Authorization | group | Can view group
- Authentication and Authorization | permission | Can add permission
- Authentication and Authorization | permission | Can change permission
- Authentication and Authorization | permission | Can delete permission
- Authentication and Authorization | permission | Can view permission
- Authentication and Authorization | user | Can add user
- Authentication and Authorization | user | Can change user
- Authentication and Authorization | user | Can delete user
- Authentication and Authorization | user | Can view user

Choose all ⓘ

Hold down "Control", or "Command" on a Mac, to select more than one.

Chosen permissions ⓘ

Filter

Remove all

Figure 4.16 – Selecting permissions

This can be done by selecting the permissions from **Available permissions** and clicking the right arrow in the middle so that they appear under **Chosen permissions**. Note that to add multiple permissions at a time, you can hold down the *Ctrl* key (or *Command* for Macintosh) to select more than one:

Help Desk User

Name:

Permissions:

Available permissions ⓘ

Filter

- Authentication and Authorization | group | Can delete group
- Authentication and Authorization | group | Can view group
- Authentication and Authorization | permission | Can add permission
- Authentication and Authorization | permission | Can change permission
- Authentication and Authorization | permission | Can delete permission
- Authentication and Authorization | user | Can add user
- Authentication and Authorization | user | Can delete user
- Content Types | content type | Can add content type
- Content Types | content type | Can change content type
- Content Types | content type | Can delete content type
- Content Types | content type | Can view content type

Choose all ⓘ

Hold down "Control", or "Command" on a Mac, to select more than one.

Chosen permissions ⓘ

Filter

- Administration | log entry | Can view log entry
- Authentication and Authorization | permission | Can view permission
- Authentication and Authorization | user | Can change user
- Authentication and Authorization | user | Can view user

Remove all

Figure 4.17 – Adding selected permissions to Chosen permissions

Once you click the **SAVE** button, you will see a confirmation message stating **The group "Help Desk User" was added successfully:**

- 사용 권한 보기
- 용자가 변경 가능
- 사용자 보기

Add group

Name:

Permissions:

Available permissions ⓘ

Q Filter

- Authentication and Authorization | group | Can delete group
- Authentication and Authorization | group | Can view group
- Authentication and Authorization | permission | Can add permission
- Authentication and Authorization | permission | Can change permission
- Authentication and Authorization | permission | Can delete permission
- Authentication and Authorization | permission | Can view permission
- Authentication and Authorization | user | Can add user
- Authentication and Authorization | user | Can change user
- Authentication and Authorization | user | Can delete user
- Authentication and Authorization | user | Can view user

Choose all ⓘ

Hold down "Control", or "Command" on a Mac, to select more than one.

Chosen permissions ⓘ

Q Filter

Remove all

그림 4.16 - 권한 선택

이것은 사용 가능한 권한에서 권한을 선택하고 중간의 오른쪽 화살표를 클릭하여 선택된 권한 아래에 나타나게 함으로써 수행할 수 있습니다. 한 번에 여러 권한을 추가하려면 Ctrl 키(또는 Macintosh의 경우 Command 키)를 누른 채로 여러 개를 선택할 수 있습니다:

Help Desk User

Name:

Permissions:

Available permissions ⓘ

Q Filter

- Authentication and Authorization | group | Can delete group
- Authentication and Authorization | group | Can view group
- Authentication and Authorization | permission | Can add permission
- Authentication and Authorization | permission | Can change permission
- Authentication and Authorization | permission | Can delete permission
- Authentication and Authorization | user | Can add user
- Authentication and Authorization | user | Can delete user
- Content Types | content type | Can add content type
- Content Types | content type | Can change content type
- Content Types | content type | Can delete content type
- Content Types | content type | Can view content type

Choose all ⓘ

Hold down "Control", or "Command" on a Mac, to select more than one.

Chosen permissions ⓘ

Q Filter

- Administration | log entry | Can view log entry
- Authentication and Authorization | permission | Can view permission
- Authentication and Authorization | user | Can change user
- Authentication and Authorization | user | Can view user

Remove all

그림 4.17 - 선택한 권한을 선택된 권한에 추가

SAVE 버튼을 클릭하면 "도움 요청 사용자" 그룹이 성공적으로 추가되었다는 확인 메시지가 표시됩니다:

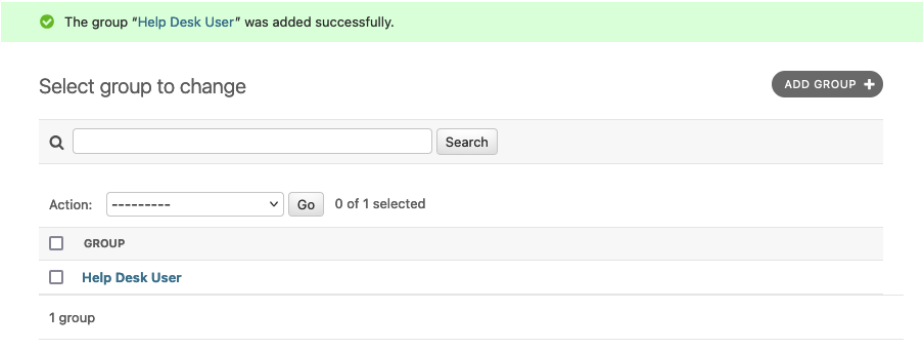


Figure 4.18 – The group Help Desk User was added successfully

5. Now, navigate to **Home | AUTHENTICATION AND AUTHORIZATION | Users** and click the link of the user with the first name **carol**:

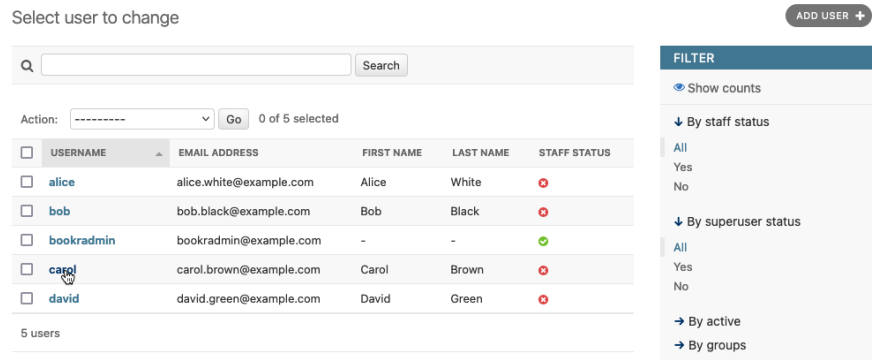


Figure 4.19 – Clicking on the carol username

6. Scroll down to the **Permissions** fields set and select the **Staff status** checkbox. This is required for Carol to be able to log in to the admin app:

✓ The group "Help Desk User" was added successfully.

Select group to change ADD GROUP +

Q Search

Action: Go 0 of 1 selected

☐ GROUP

☐ Help Desk User

1 group

그림 4.18 - 그룹 Help Desk User가 성공적으로 추가되었습니다

5. 이제 Home | 인증 및 권한 부여 | 사용자로 이동하여 이름이 carol인 사용자의 링크를 클릭합니다:

Select user to change ADD USER +

Q Search

Action: Go 0 of 5 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	alice	alice.white@example.com	Alice	White	⊘
☐	bob	bob.black@example.com	Bob	Black	⊘
☐	bookradmin	bookradmin@example.com	-	-	⊙
☐	carol	carol.brown@example.com	Carol	Brown	⊘
☐	david	david.green@example.com	David	Green	⊘

5 users

FILTER

Show counts

↓ By staff status

All
Yes
No

↓ By superuser status

All
Yes
No

→ By active

→ By groups

그림 4.19 - carol 사용자 이름 클릭하기

6. Permissions 필드 세트로 스크롤하여 Staff 상태 체크박스를 선택합니다. 이는 Carol이 admin 앱에 로그인할 수 있도록 하기 위해 필요합니다:

Permissions

☒ **Active**
Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☒ **Staff status**
Designates whether the user can log into this admin site.

☐ **Superuser status**
Designates that this user has all permissions without explicitly assigning them.

Groups:

Available groups

Filter

Help Desk User

Chosen groups

Filter

Choose all **Remove all**

The groups this user belongs to. A user will get all permissions granted to each of their groups. Hold down "Control", or "Command" on a Mac, to select more than one.

Figure 4.20 – Clicking the Staff status checkbox

7. Add Carol to the **Help Desk User** group that we created in the previous steps by selecting it from the **Available groups** selection box (refer to *Figure 4.20*) and clicking the right arrow to shift it into her list of **Chosen groups** (as shown in *Figure 4.21*). Note that unless you do this, Carol won't be able to log in to the admin interface using her credentials:

Permissions

☒ Active
Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☒ Staff status
Designates whether the user can log into this admin site.

☐ Superuser status
Designates that this user has all permissions without explicitly assigning them.

Groups:

Available groups ⓘ

Q Filter

Help Desk User

Choose all ⓘ

Chosen groups ⓘ

Q Filter

Remove all

The groups this user belongs to. A user will get all permissions granted to each of their groups. Hold down "Control", or "Command" on a Mac, to select more than one.

그림 4.20 - 직원 상태 체크박스 클릭하기

7. 이전 단계에서 생성한 도움말 데스크 사용자 그룹에 캐롤을 추가하려면 사용 가능한 그룹 선택 상자에서 선택하고(그림 4.20 참조) 오른쪽 화살표를 클릭하여 그녀의 선택된 그룹 목록으로 이동시킵니다(그림 4.21 참조). 이 작업을 수행하지 않으면 캐롤은 자신의 자격 증명을 사용하여 관리자 인터페이스에 로그인할 수 없음을 유의하세요:

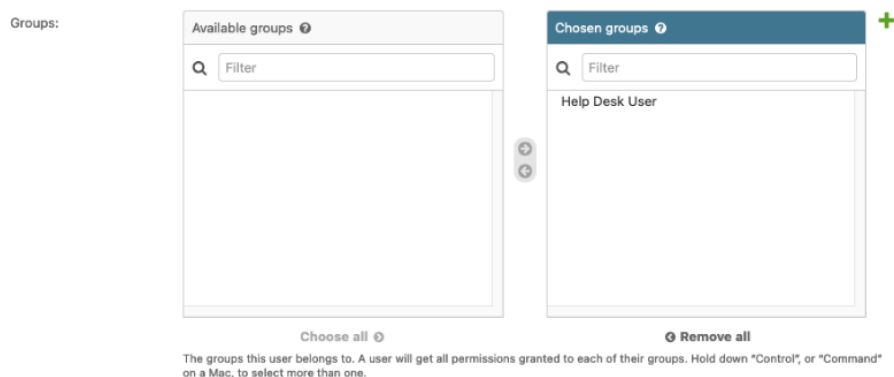


Figure 4.21 – Shifting the Help Desk User group into the Chosen groups list for Carol

8. Let's test whether what we've done up till now has yielded the right outcome. To do this, log out of the admin site and log in again as Carol. Upon logging out, you should see the following on your screen:

Logged out

Thanks for spending some quality time with the web site today.

[Log in again](#)

Figure 4.22 – The Logged out screen

Note

If you don't recall the password that you initially gave her, you can change the password in the command line by typing `python manage.py changepassword carol`.

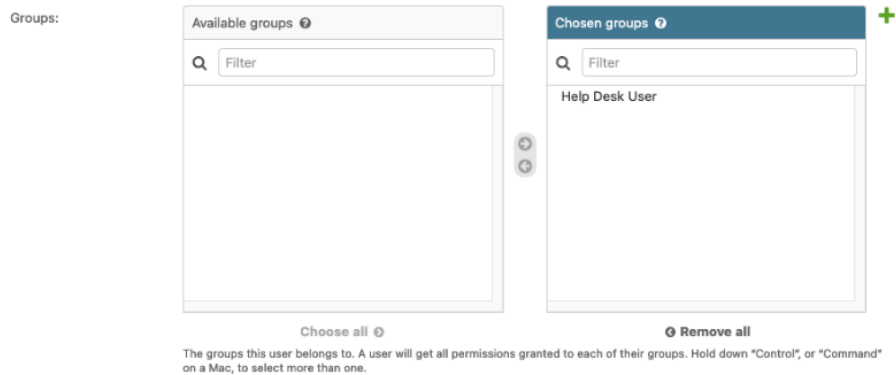


그림 4.21 - Carol을 위한 선택된 그룹 목록으로 도움말 데스크 사용자 그룹 이동

8. 지금까지 우리가 한 일이 올바른 결과를 가져왔는지 테스트해 보겠습니다. 이를 위해 관리자 사이트에서 로그아웃하고 Carol로 다시 로그인하세요. 로그아웃하면 화면에 다음과 같은 내용이 표시되어야 합니다:

Logged out

Thanks for spending some quality time with the web site today.

[Log in again](#)

그림 4.22 - 로그아웃 화면

참고

처음에 그녀에게 제공한 비밀번호를 기억하지 못한다면, 명령줄에서 `python manage.py changepasswordcarol`을 입력하여 비밀번호를 변경할 수 있습니다.

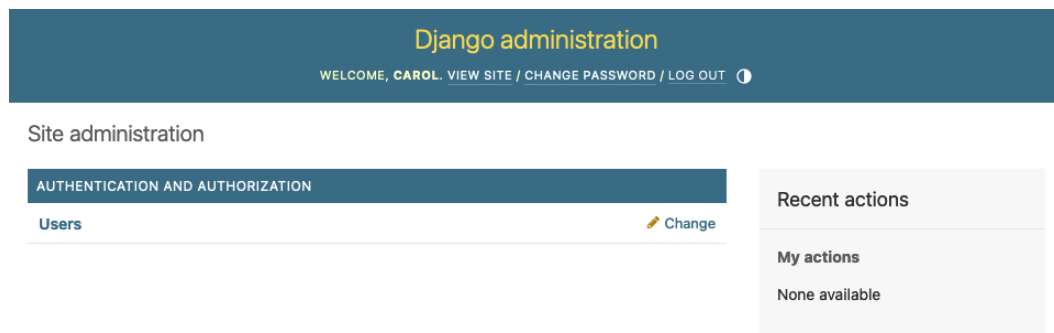


Figure 4.23 – The admin dashboard

As we did not assign any group permissions, not even **auth | group | Can view group**, to the **Help Desk User** group, when Carol logs in, the **Groups** admin interface is not available to her. Similarly, when you navigate to **Home | AUTHENTICATION AND AUTHORIZATION | Users** and click a user link, you will see that there are no options to edit or delete the user. This is because of the permissions that were granted to the **Help Desk User** group, of which Carol is a member. The members of the group can view and edit users, but cannot add or delete any user.

In this exercise, we learned how to grant a certain number of administrative privileges to users of our Django app.

Now that we have an understanding of how the admin app works with the **AUTHENTICATION AND AUTHORIZATION** models for **Users** and **Groups**, we can turn our attention to registering the models that we have developed in the reviews app.

Registering models with the admin app

The Django admin app produces workable CRUD interfaces to Django objects based on the characteristics of the models, with a minimal amount of coding. In this section, we will first look at how to register the models with the admin app and how elements of the interface, such as change lists and change pages, are derived from the model properties. We will conclude with an important exercise that demonstrates how foreign key settings in the model determine the behavior of the object deletion process.

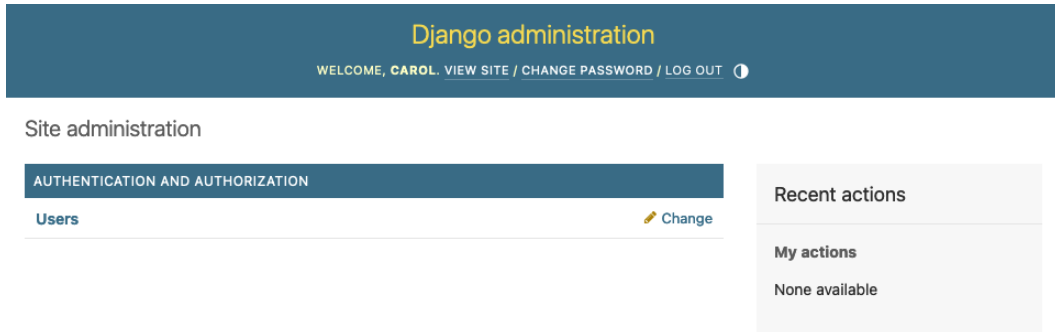


그림 4.23 - 관리자 대시보드

우리가 `idx_ecf59c0eany` 그룹 권한을 할당하지 않았기 때문에, `auth | group |` 그룹 보기 권한조차도 `Help Desk User` 그룹에 부여되지 않았습니다. Carol이 로그인할 때, 그룹 관리자 인터페이스 `idx_bd357842`는 그녀에게 제공되지 않습니다. 마찬가지로, `Home | 인증 및 권한 부여 |` 사용자로 이동하고 사용자 링크를 클릭하면 사용자를 편집하거나 삭제할 수 있는 옵션이 없음을 알 수 있습니다. 이는 Carol이 소속된 `Help Desk User` 그룹에 부여된 권한 때문입니다. 그룹의 구성원은 사용자를 보고 편집할 수 있지만, `idx_7732f2e6`은 `idx_aac813f2`를 추가하거나 삭제할 수 없습니다.

이번 연습에서는 장고 앱의 사용자에게 특정 수의 관리 권한을 부여하는 방법을 배웠습니다.

이제 관리 앱이 사용자 및 그룹에 대한 인증 및 권한 부여 모델과 어떻게 작동하는지 이해했으므로, 리뷰 앱에서 개발한 모델을 등록하는 데 주의를 기울일 수 있습니다.

관리 앱에 모델 등록하기

장고 관리 앱은 최소한의 코딩으로 모델의 특성에 따라 장고 `idx_d33f5ec2` 객체에 대한 작동 가능한 CRUD 인터페이스를 생성합니다. 이 섹션에서는 먼저 관리 앱에 모델을 등록하는 방법과 변경 목록 및 변경 페이지와 같은 인터페이스 요소가 모델 속성에서 어떻게 파생되는지를 살펴보겠습니다. 우리는 모델의 외래 키 설정이 객체 삭제 프로세스의 동작을 어떻게 결정하는지를 보여주는 중요한 연습으로 결론을 맺을 것입니다.

Registering the reviews model

Let's say that Carol is tasked with improving the **Reviews** section in Bookr; that is, only the most relevant and comprehensive reviews should be shown, and duplicate or spam entries should be removed. For this, she will need access to the reviews model. As we have seen previously with our investigation of groups and users, the admin app already contains admin pages for the models from the *Authentication and Authorization* app, but it does not yet reference the models in our *Reviews* app. To make the admin app aware of the models, we need to explicitly register them with the admin app. Fortunately, we don't need to modify the admin app's code to do so, as we can instead import the admin app into our project and use its API to register our models. This has already been done in the *Authentication and Authorization* app, so let's try it with our *Reviews* app. Our aim is to be able to use the admin app to edit the data in our reviews model.

Take a look at the `reviews/admin.py` file. It is a placeholder file that was generated with the `startapp` subcommand that we used in *Chapter 1*, and currently contains these lines:

```
from django.contrib import admin

# Register your models here.
```

Now, we can try to expand this. To make the admin app aware of our models, we can modify the `reviews/admin.py` file and import the models. Then, we can register the models with the `AdminSite` object, `admin.site`. The `AdminSite` object contains the instance of the Django admin application (later, we will learn how to subclass this `AdminSite` object and override many of its properties). Then, `reviews/admin.py` will look as follows:

```
from django.contrib import admin
from reviews.models import (Publisher, Contributor,
                             Book, BookContributor, Review)

admin.site.register(Publisher)
admin.site.register(Contributor)
admin.site.register(Book)
admin.site.register(BookContributor)
admin.site.register(Review)
```

The `admin.site.register` method makes the models available to the admin app by adding it to a registry of classes contained in `admin.site._registry`. If we chose not to make a model accessible through the admin interface, we would simply not register it. When you reload

리뷰 모델 등록하기

캐롤이 Bookr의 리뷰 섹션을 개선하는 임무를 맡았다고 가정해 보자. 즉, 가장 관련성이 높고 포괄적인 리뷰만 표시되고, 중복되거나 스팸인 항목은 제거되어야 한다. 이를 위해 그녀는 리뷰 모델에 접근할 필요가 있다. 이전에 그룹과 사용자에 대한 조사에서 보았듯이, 관리 앱에는 인증 및 권한 부여 앱의 모델에 대한 관리 페이지가 이미 포함되어 있지만, 아직 리뷰 앱의 모델을 참조하고 있지 않다. 관리 앱이 모델을 인식할 수 있도록 하려면, 명시적으로 관리 앱에 등록해야 한다. 다행히도, 이를 위해 관리 앱의 코드를 수정할 필요는 없으며, 대신 관리 앱을 프로젝트에 가져오고 그 API를 사용하여 모델을 등록할 수 있다. 이는 인증 및 권한 부여 앱에서 이미 수행되었으므로, 리뷰 앱으로 시도해 보자. 우리의 목표는 관리 앱을 사용하여 리뷰 모델의 데이터를 편집할 수 있는 것이다.

리뷰/admin.py 파일을 살펴보자. 이 파일은 1장에서 사용한 startapp 하위 명령으로 생성된 자리 표시자 파일이며, 현재 다음과 같은 줄이 포함되어 있다:

```
from django.contrib import admin

# 여기에 모델을 등록하세요.
```

이제 이를 확장해 볼 수 있다. 관리 앱이 우리의 모델을 인식할 수 있도록 하기 위해, 리뷰/admin.py 파일을 수정하고 모델을 가져올 수 있다. 그런 다음, AdminSite 객체인 admin.site에 모델을 등록할 수 있다. AdminSite 객체는 Django 관리 애플리케이션의 인스턴스를 포함하고 있다(나중에 이 AdminSite 객체를 서브클래싱하고 많은 속성을 재정의하는 방법을 배울 것이다). 그러면 리뷰/admin.py는 다음과 같이 보일 것이다:

```
from django.contrib import admin
from reviews.models import (Publisher, Contributor,
                             도서, 도서 기여자, 리뷰)

admin.site.register(출판사)
admin.site.register(기여자)
admin.site.register(도서)
admin.site.register(도서 기여자)
admin.site.register(리뷰)
```

admin.site.register 메서드는 admin.site._registry에 포함된 클래스의 레지스트리에 추가하여 모델을 관리 앱에서 사용할 수 있도록 만듭니다. 관리 인터페이스를 통해 모델에 접근할 수 없도록 선택하면 단순히 등록하지 않으면 됩니다. 다시 로드할 때

`http://127.0.0.1:8000/admin/` in your browser, you will see the following on the admin app landing page:

Site administration

AUTHENTICATION AND AUTHORIZATION		
Groups	+ Add	Change
Users	+ Add	Change
REVIEWS		
Book contributors	+ Add	Change
Books	+ Add	Change
Contributors	+ Add	Change
Publishers	+ Add	Change
Reviews	+ Add	Change

Figure 4.24 – Admin app landing page

Note the change in the appearance of the admin page after the reviews model has been imported.

With the five models of the *Reviews* app registered with the admin app, we can examine some of the default interfaces that it has created for us.

Change lists

We have seen several examples of change list pages so far, as we took a tour of the Django admin app.

A change list is a page that displays a list of links to objects that are retrieved by a query, and also options to perform actions on multiple objects (such as deletion), add new ones, filter, or sort the objects being displayed.

Now that we have registered our models with the admin app, we can take a look at the default change lists that have been created. If we click the **Publishers** link, we will be taken to `http://127.0.0.1:8000/admin/reviews/publisher` and see a change list containing links to the publishers. These links are designated by the `id` field of the `Publisher` objects. If your database

브라우저에서 <http://127.0.0.1:8000/admin/>를 열면, 관리 애플리케이션 랜딩 페이지에서 다음을 볼 수 있습니다:

Site administration

AUTHENTICATION AND AUTHORIZATION		
Groups	+ Add	Change
Users	+ Add	Change
REVIEWS		
Book contributors	+ Add	Change
Books	+ Add	Change
Contributors	+ Add	Change
Publishers	+ Add	Change
Reviews	+ Add	Change

그림 4.24 - 관리자 앱 랜딩 페이지

리뷰 모델이 가져온 후 관리자 페이지의 외관이 변경된 것을 주목하세요.

리뷰 앱의 다섯 개 모델이 관리자 앱에 등록되었으므로, 우리가 위해 생성된 기본 인터페이스를 살펴볼 수 있습니다.

변경 목록

지금까지 장고 관리자 앱을 둘러보면서 여러 변경 목록 페이지의 예를 보았습니다.

변경 목록 `idx_36bfbde4`는 쿼리에 의해 검색된 객체에 대한 링크 목록을 표시하는 페이지이며, 여러 객체(예: 삭제)에 대한 작업을 수행할 수 있는 옵션, 새 객체 추가, 필터링 또는 표시되는 객체 정렬을 제공합니다.

모델을 관리자 앱에 등록했으므로, 생성된 기본 변경 목록을 살펴볼 수 있습니다. 게시자 링크를 클릭하면 <http://127.0.0.1:8000/admin/reviews/publisher>로 이동하게 되며, 게시자에 대한 링크가 포함된 변경 목록을 볼 수 있습니다. 이러한 링크는 게시자 객체의 id 필드로 지정됩니다. 데이터베이스가 만약 당신의 데이터베이스

has been populated with the script in *Chapter 3*, you will see a list with seven publishers, as shown in *Figure 4.25*.

Note

Depending on the state of your database and based on the activities you have completed, the object IDs, URLs, and links in your session may be ordered and numbered differently from the following screenshots.

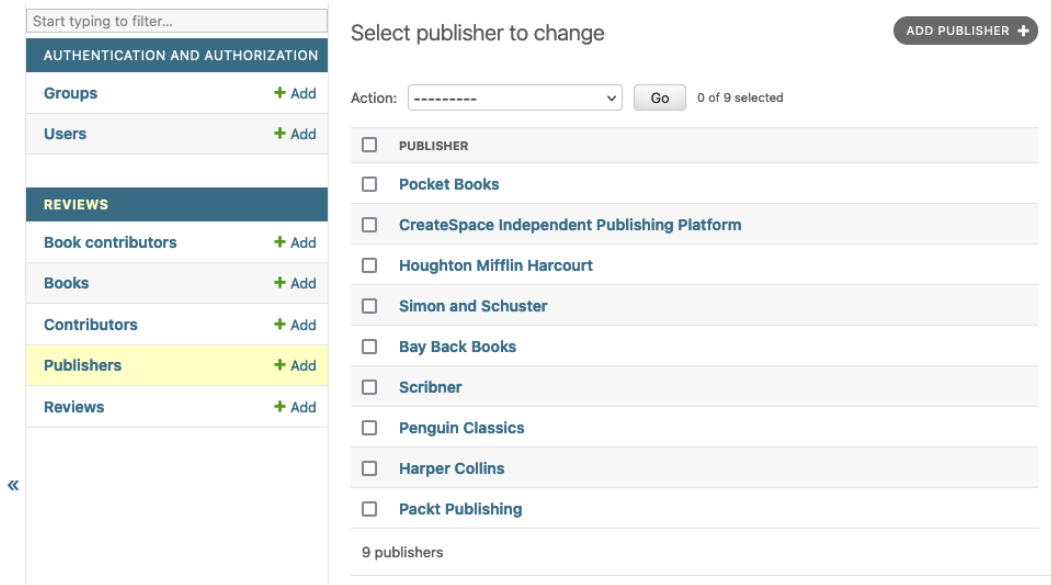


Figure 4.25 – The Select publisher to change list

We can follow one of the links in the change list and examine the **Change publisher** page.

The Change publisher page

The **Change publisher** page at <http://127.0.0.1:8000/admin/reviews/publisher/1> contains what we might expect (see *Figure 4.26*). There is a form for editing the publisher's details. These details have been derived from the `reviews.models.Publisher` class:

3장에서 스크립트로 채워진 idx_89fb9952를 사용하면 그림 4.25에 표시된 대로 7개의 출판사 목록이 표시됩니다.

참고

데이터베이스의 상태와 완료한 활동에 따라 세션의 객체 ID, URL 및 링크는 다음 스크린샷과 다르게 정렬되고 번호가 매겨질 수 있습니다.

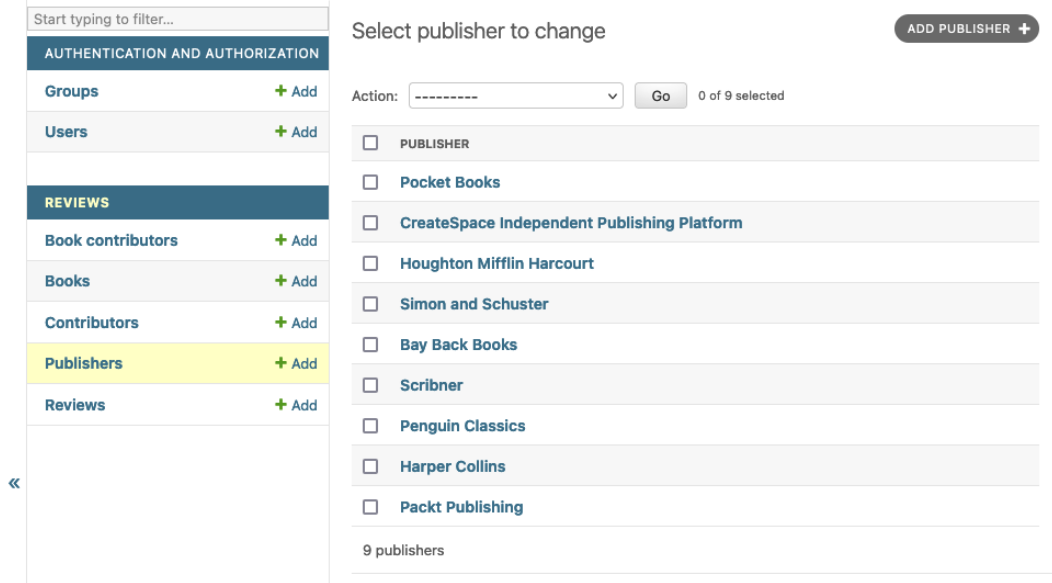


그림 4.25 - 변경할 출판사를 선택하는 목록

변경 목록의 링크 중 하나를 따라가서 출판사 변경 페이지를 살펴볼 수 있습니다.

출판사 변경 페이지

<http://127.0.0.1:8000/admin/reviews/publisher/1>의 출판사 변경 페이지에는 우리가 idx_039e9466에서 기대할 수 있는 내용이 포함되어 있습니다(그림 4.26 참조). 출판사 세부정보를 편집하기 위한 양식이 있습니다. 이 세부정보는 reviews.models.Publisher 클래스에서 파생되었습니다.

Change publisher

HISTORY

Packt Publishing

Name:

The name of the Publisher.

Website: **Currently:** <https://www.packtpub.com/>
Change:

The Publisher's website.

Email:

The Publisher's email address.

Figure 4.26 – The Change publisher page

If we had clicked the **ADD PUBLISHER +** button, the admin app would have returned a similar form for adding a publisher. The beauty of the admin app is that it gives us all of this CRUD functionality with just one line of coding – `admin.site.register(Publisher)` – using the definition of the `reviews.models.Publisher` attributes as a schema for the page content:

```
class Publisher(models.Model):
    """A company that publishes books."""
    Name = models.CharField(max_length=50,
                           help_text="The name of the
                           Publisher.")
    website = models.URLField(
        help_text="The Publisher's website.")
    email = models.EmailField(
        help_text="The Publisher's email address.")
```

The publisher **Name** field is constrained to 50 characters as specified in the model. The help text that appears in gray below each field is derived from the `help_text` attributes specified on

Change publisher

Packt Publishing

HISTORY

Name:

Packt Publishing

The name of the Publisher.

Website:

Currently: <https://www.packtpub.com/>

Change:

<https://www.packtpub.com/>

The Publisher's website.

Email:

info@packtpub.com

The Publisher's email address.

SAVE

Save and add another

Save and continue editing

Delete

그림 4.26 - 출판사 변경 페이지

우리가 ADD PUBLISHER + 버튼을 클릭했다면, 관리자 앱은 출판사를 추가하기 위한 유사한 formidx_9d778bfe를 반환했을 것입니다. 관리자 앱의 장점은 단 한 줄의 코드로 모든 CRUD 기능을 제공한다는 것입니다 – `admin.site.register(Publisher)` – 페이지 콘텐츠의 스키마로 `reviews.models.Publisher` 속성의 정의를 사용합니다:

```
class Publisher(models.Model):
    """책을 출판
    하는 회사입니다."""
    name = models.CharField(max_length=50,
                            help_text="출판사의 이름입니다",
                            Publisher.)
    website = models.URLField(
        help_text="출판사의 웹사이트입니다.")
    email = models.EmailField(
        help_text="발행자의 이메일 주소.")
```

발행자 이름 필드는 모델에 지정된 대로 50자로 제한됩니다. 각 필드 아래에 회색으로 나타나는 도움말 텍스트는 지정된 `help_text` 속성에서 파생됩니다.

the model. We can see that `models.CharField`, `models.URLField`, and `models.EmailField` are rendered in HTML as input elements of the text, url, and email types, respectively.

The fields in the form come with validation where appropriate. Unless model fields are set to `blank=True` or `null=True`, the form will throw an error if the field is left blank, as is the case for the `Publisher.name` field. Similarly, as `Publisher.website` and `Publisher.email` are respectively defined as instances of `models.URLField` and `models.EmailField`, they are validated accordingly. In *Figure 4.27*, we can see the validation of **Name** as a required field, **Website** as a URL, and **Email** as an email address:

Add publisher

Please correct the errors below.

Name:

This field is required.

The name of the Publisher.

Website:

Enter a valid URL.

The Publisher's website.

Email:

Enter a valid email address.

The Publisher's email address.

SAVE

Save and add another

Save and continue editing

Figure 4.27 – Field validation

It is useful to examine how the admin app renders elements of the models to understand how it functions. In your browser, right-click **View Page Source** and examine the HTML that has been rendered for this form. You will see a browser tab displaying HTML code similar to that in *Figure 4.28*.

모델. 우리는 `models.CharField`, `models.URLField`, 및 `models.EmailField`가 각각 텍스트, URL 및 이메일 유형의 입력 요소로 HTML에 렌더링되는 것을 볼 수 있습니다.

양식의 필드는 적절한 경우 유효성 검사를 제공합니다. 모델 필드가 `blank=True` 또는 `null=True`로 설정되지 않는 한, 필드가 비어 있으면 양식이 오류를 발생시킵니다. 이는 `Publisher.name` 필드의 경우와 같습니다. 마찬가지로, `Publisher.website` 및 `Publisher.email`은 각각 `models.URLField` 및 `models.EmailField`의 인스턴스로 정의되므로, 그에 따라 유효성 검사가 수행됩니다. 그림 4.27에서는 이름이 필수 필드로서의 유효성 검사, 웹사이트가 `idx_cb97da4fa` URL로서의 유효성 검사, 이메일이 이메일 주소로서의 유효성 검사를 받는 것을 볼 수 있습니다:

Add publisher

Please correct the errors below.

Name:

This field is required.

The name of the Publisher.

Website:

Enter a valid URL.

packtcom

The Publisher's website.

Email:

Enter a valid email address.

infoatpackt.com

The Publisher's email address.

SAVE

Save and add another

Save and continue editing

그림 4.27 - 필드 유효성 검사

관리자 앱이 모델의 요소를 렌더링하는 방식을 살펴보는 것이 유용합니다. 브라우저에서 마우스 오른쪽 버튼을 클릭하고 페이지 소스 보기 (View Page Source)를 선택하여 이 양식에 대해 렌더링된 HTML을 확인하십시오. 그러면 그림 4.28과 유사한 `idx_b2af63ee` HTML 코드가 표시되는 브라우저 탭을 볼 수 있습니다.


```

<fieldset class="module aligned ">
  <div class="form-row errors field-name">
    <ul class="errorlist"><li>This field is
      required.</li></ul>
    <div>
      <label class="required" for="id_name">Name:</label>
      <input type="text" name="name" class="vTextField"
        maxlength="50" required="" id="id_name">
      <div class="help">The name of the Publisher.</div>
    </div>
  </div>
  <div class="form-row errors field-website">
    <ul class="errorlist"><li>Enter a valid URL.</li></ul>
    <div>
      <label class="required"
        for="id_website">Website:</label>
      <input type="url" name="website" value="packtcom"
        class="vURLField" maxlength="200" required=""
        id="id_website">
      <div class="help">The Publisher's website.</div>
    </div>
  </div>
  <div class="form-row errors field-email">
    <ul class="errorlist"><li>Enter a valid email
      address.</li></ul>
    <div>
      <label class="required" for="id_email">Email:</label>
      <input type="email" name="email"
        value="infoatpackt.com"
        class="vTextField" maxlength="254"
        required="" id="id_email">
      <div class="help">The Publisher's email
        address.</div>
    </div>
  </div>
</fieldset>

```

Figure 4.28 – Browser displaying HTML code

```
<fieldset class="module aligned ">
  <div class="form-row errors field-name">
    <ul class="errorlist"><li>This field is
      required.</li></ul>
    <div>
      <label class="required" for="id_name">Name:</label>
      <input type="text" name="name" class="vTextField"
        maxlength="50" required="" id="id_name">
      <div class="help">The name of the Publisher.</div>
    </div>
  </div>
  <div class="form-row errors field-website">
    <ul class="errorlist"><li>Enter a valid URL.</li></ul>
    <div>
      <label class="required"
        for="id_website">Website:</label>
      <input type="url" name="website" value="packtcom"
        class="vURLField" maxlength="200" required=""
        id="id_website">
      <div class="help">The Publisher's website.</div>
    </div>
  </div>
  <div class="form-row errors field-email">
    <ul class="errorlist"><li>Enter a valid email
      address.</li></ul>
    <div>
      <label class="required" for="id_email">Email:</label>
      <input type="email" name="email"
        value="infoatpackt.com"
        class="vTextField" maxlength="254"
        required="" id="id_email">
      <div class="help">The Publisher's email
        address.</div>
    </div>
  </div>
</fieldset>
```

그림 4.28 - HTML 코드를 표시하는 브라우저

The form has a `publisher_form` ID, and it contains a field set with HTML elements corresponding to the data structure of the `Publisher` model in `reviews/models.py`, shown as follows:

```
class Publisher(models.Model):
    """A company that publishes books."""
    name = models.CharField(max_length=50,
                           help_text="The name of the
                           Publisher.")
    website = models.URLField(
        help_text="The Publisher's website.")
    email = models.EmailField(
        help_text="The Publisher's email address.")
```

Note that for the name, the input field is rendered like this:

```
<input type="text" name="name" value="Packt Publishing"
class="vTextField" maxlength="50"
required="" id="id_name">
```

It is a required field, and it has a text type and `maxlength` of 50, as defined by the `max_length` parameter in the model definition:

```
name = models.CharField(max_length=50,
                        help_text="The name of the
                        Publisher.")
```

Similarly, we can see the website and email being defined in the model, as `URLField` and `EmailField` are rendered in HTML as input elements of the `url` and `email` types, respectively:

```
<input type="url" name="website"
value="https://www.packtpub.com/"
class="vURLField" maxlength="200" required=""
id="id_website">
<input type="email" name="email" value="info@packtpub.com"
class="vTextField" maxlength="254"
required="" id="id_email">
```

We have learned that the Django admin app derives sensible HTML representations of Django models based on the model definitions that we have provided.

폼 idx_f4512fe6에는 publisher_form ID가 있으며, reviews/models.py의 Publisher 모델 데이터 구조에 해당하는 HTML 요소가 포함된 필드 세트가 다음과 같이 포함되어 있습니다:

```
class Publisher(models.Model):
    ...
    책을 출판하는 회사.
    name = models.CharField(max_length=50,
                             help_text="이름"
                             출판사.")
    website = models.URLField(
        help_text="출판사의 웹사이트입니다.")
    email = models.EmailField(
        help_text="출판사의 이메일 주소입니다.")
```

이름의 경우, 입력 필드는 다음과 같이 렌더링됩니다:

```
<input type="text" name="name" value="Packt Publishing"
class="vTextField" maxlength="50"
required="" id="id_name">
```

이것은 필수 필드이며, 텍스트 유형과 최대 길이 50을 가지며, 이는 모델 정의의 max_length 매개변수에 의해 정의됩니다:

```
name = models.CharField
        (max_lengt
         h=50,
         help_text=
         "출판사의
         이름입니다")
```

유사하게, 우리는 모델에서 웹사이트와 이메일이 정의되는 것을 볼 수 있으며, URLField와 EmailField는 각각 url 및 email 유형의 입력 요소로 HTML에 렌더링됩니다:

```
<input type="url" name="website"
value="https://www.packtpub.com/"
class="vURLField" maxlength="200" required=""
id="id_website">
<input type="email" name="email" value="info@packtpub.com"
class="vTextField" maxlength="254"
required="" id="id_email">
```

우리는 Django 관리 앱이 제공한 모델 정의를 기반으로 Django 모델의 합리적인 HTML 표현을 유도한다는 것을 배웠습니다.

Now let's examine the book change page.

The book change page

Similarly, there is a change page that can be reached by selecting **Books** from the **Site administration** page and then selecting a specific book in the change list:

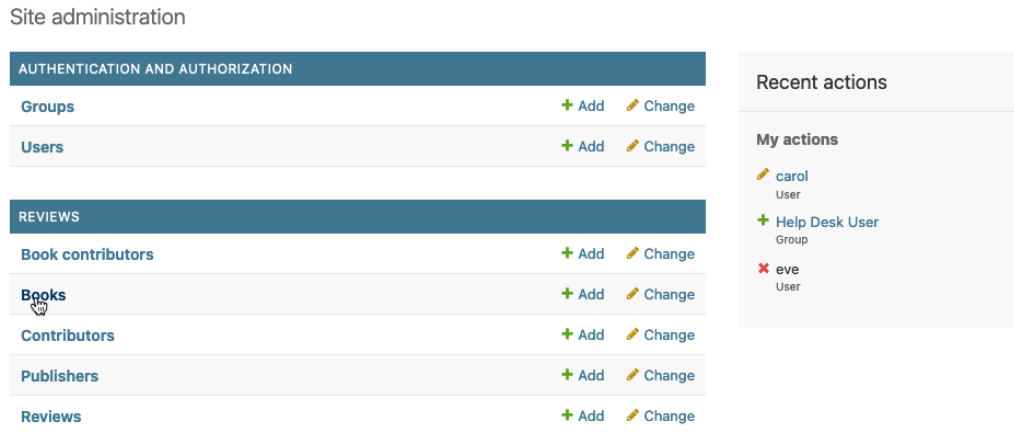


Figure 4.29 – Selecting Books from the Site administration page

After clicking **Books**, as shown in the preceding screenshot, you will see the following on your screen:

이제 책 변경 페이지를 살펴보겠습니다.

책 변경 페이지

유사하게, 사이트 관리 페이지에서 책을 선택한 다음 변경 목록에서 특정 책을 선택하여 접근할 수 있는 변경 페이지가 있습니다:

Site administration

The screenshot shows the Django Site Administration interface. On the left, under the 'AUTHENTICATION AND AUTHORIZATION' section, there are links for 'Groups' and 'Users'. Below that, under the 'REVIEWS' section, there are links for 'Book contributors', 'Books', 'Contributors', 'Publishers', and 'Reviews'. The 'Books' link is highlighted with a mouse cursor. To the right, there is a 'Recent actions' sidebar showing a list of actions: 'carol User', 'Help Desk User Group', and 'eve User'.

AUTHENTICATION AND AUTHORIZATION	
Groups	+ Add Change
Users	+ Add Change

REVIEWS	
Book contributors	+ Add Change
Books	+ Add Change
Contributors	+ Add Change
Publishers	+ Add Change
Reviews	+ Add Change

Recent actions

My actions

- carol User
- + Help Desk User Group
- x eve User

그림 4.29 - 사이트 관리 페이지에서 책 선택하기

이전 스크린샷에 표시된 대로 책을 클릭한 후, 화면에서 다음을 보게 될 것입니다:

Select book to change ADD BOOK +

Action: 0 of 18 selected

☐	BOOK
☐	The Talisman
☐	Paul Clifford
☐	Animal Farm: A Fairy Story
☐	1984
☐	Pride and Prejudice
☐	Fahrenheit 451
☐	The Catcher in the Rye
☐	The Great Gatsby
☐	To Kill A Mocking Bird
☐	For Whom The Bell Tolls
☐	The Grapes of Wrath
☐	Brave New World
☐	Hands-On Reinforcement Learning with Python
☐	Natural Language Processing with TensorFlow
☐	Deep Reinforcement Learning Hands-On
☐	Architects of Intelligence
☐	Hands-On Machine Learning for Algorithmic Trading
☐	Advanced Deep Learning with Keras

18 books

Figure 4.30 – The Select book to change page

In this instance, selecting the book *Architects of Intelligence* will take us to `http://127.0.0.1:8000/admin/reviews/book/3/change/`. In the previous example, all the model

Select book to change
ADD BOOK +

Action: ----- Go 0 of 18 selected

☐ BOOK

☐ The Talisman

☐ Paul Clifford

☐ Animal Farm: A Fairy Story

☐ 1984

☐ Pride and Prejudice

☐ Farenheit 451

☐ The Catcher in the Rye

☐ The Great Gatsby

☐ To Kill A Mocking Bird

☐ For Whom The Bell Tolls

☐ The Grapes of Wrath

☐ Brave New World

☐ Hands-On Reinforcement Learning with Python

☐ Natural Language Processing with TensorFlow

☐ Deep Reinforcement Learning Hands-On

☐ Architects of Intelligence

☐ Hands-On Machine Learning for Algorithmic Trading

☐ Advanced Deep Learning with Keras

18 books

그림 4.30 - 페이지를 변경할 책 선택하기

이 경우, idx_20e86e75 책 '지능의 건축가'를 선택하면 http://로 이동합니다.
127.0.0.1:8000/admin/reviews/book/3/change/. 이전 예제에서는 모든 모델

fields were rendered as simple HTML text widgets. The rendering of some other subclasses of `django.db.models.Field` used in `models.Book` are worthy of closer examination:

The screenshot shows the Django Admin interface for changing a book. The header is dark blue with 'Django administration' in yellow. Below it, a navigation bar shows 'Home > Reviews > Books > Architects of Intelligence'. The main content area is white. At the top, 'Change book' is displayed. To the right is a 'HISTORY' button. Below this, the book title 'Architects of Intelligence' is shown in a text box, with a small note 'The title of the book.' underneath. The next section is 'Date the book was published.', showing '2018-11-23' in a date picker, with a 'Today' button and a calendar icon. A note below says 'Note: You are 10 hours ahead of server time.' The 'ISBN number of the book.' is shown in a text box with the value '9781789954531'. The 'Publisher:' is shown in a dropdown menu with 'Packt Publishing' selected, and icons for edit, add, and view are to the right. At the bottom, there are four buttons: 'SAVE', 'Save and add another', 'Save and continue editing', and 'Delete'.

Figure 4.31 – The Change book page

Here, `publication_date` is defined using `models.DateField`. It is rendered using a date selection widget. The visual representation of the widgets will vary among operating systems and the choice of browser:

필드는 간단한 HTML 텍스트 위젯으로 렌더링되었습니다. `models.Book`에서 사용되는 `django.db.models.Field`의 다른 하위 클래스의 렌더링은 더 가까이 살펴볼 가치가 있습니다:

The screenshot shows the Django administration interface for a book titled "Architects of Intelligence". The page is titled "Change book" and includes a "HISTORY" button. The form contains the following fields:

- Title:** A text input field containing "Architects of Intelligence". Below the field is the label "The title of the book."
- Date the book was published:** A date input field containing "2018-11-23". To the right of the field is a "Today" button with a calendar icon. Below the field is the note "Note: You are 10 hours ahead of server time."
- ISBN number of the book:** A text input field containing "9781789954531".
- Publisher:** A dropdown menu showing "Packt Publishing". To the right of the dropdown are icons for edit (pencil), add (plus), and view (eye).

At the bottom of the form, there are four buttons: "SAVE", "Save and add another", "Save and continue editing", and "Delete".

그림 4.31 - 책 변경 페이지

여기서 `publication_date`는 `models.DateField`를 사용하여 정의됩니다. 날짜 선택 위젯을 사용하여 렌더링됩니다. 위젯의 시각적 표현은 운영 체제와 브라우저 선택에 따라 다를 수 있습니다:

By setting `on_delete=CASCADE` on a foreign key, the model specifies the database behavior required when a record is deleted; the deletion is cascaded to other objects that are referenced by the foreign key.

It is important to understand the impact of foreign key settings on the flow of the deletion process, and the following exercise examines this.

Exercise 4.03 – Foreign keys and deletion behavior in the admin app

At present, all the `ForeignKey` relations in the reviews models are defined with an `on_delete=CASCADE` behavior. For instance, think of a case wherein an admin deletes one of the publishers. This would delete all the books that are associated with the publisher. We do not want that to happen, and that is precisely the behavior that we will be changing in this exercise:

1. Visit the **Contributors** change list at `http://127.0.0.1:8000/admin/reviews/contributor/` and select a contributor to delete. Make sure that the contributor is the author of a book.
2. Click the **Delete** button, but don't click **Yes, I'm sure** on the confirmation dialog. You will see a message like the one in *Figure 4.34*:

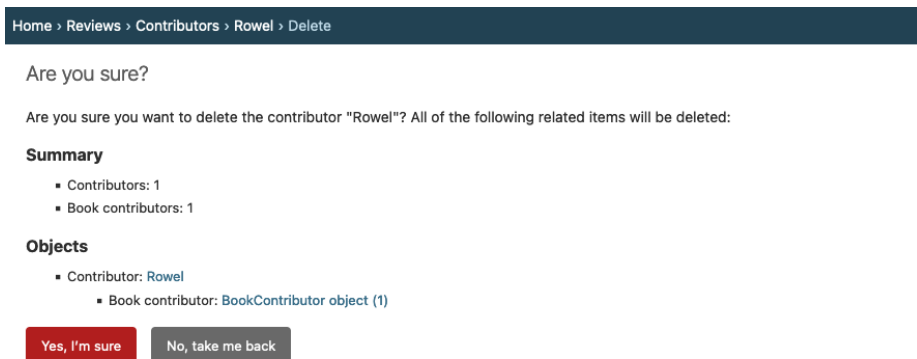


Figure 4.34 – Cascading delete confirmation dialog

In accordance with the `on_delete=CASCADE` argument to the foreign key, we are warned that deleting this `Contributor` object will have a cascading effect on a `BookContributor` object.

aidx_43ff10f1 외래 키에 on_delete=CASCADE를 설정함으로써, 모델은 레코드가 삭제될 때 데이터베이스의 동작을 지정합니다; 삭제는 외래 키에 의해 참조되는 다른 객체로 전파됩니다.

외래 키 설정이 삭제 프로세스의 흐름에 미치는 영향을 이해하는 것이 중요하며, 다음 연습에서 이를 살펴봅니다.

연습 4.03 – 관리 앱의 외래 키 및 삭제 동작

현재 리뷰 모델의 모든 ForeignKey 관계는 on_delete=CASCADE 동작으로 정의되어 있습니다. 예를 들어, 관리자가 출판사 중 하나를 삭제하는 경우를 생각해 보십시오. 이 idx_d1c1912f는 출판사와 관련된 모든 책을 삭제합니다. 우리는 그것이 발생하는 것을 원하지 않으며, 그것이 바로 우리가 이 idx_c183f950 연습에서 변경할 동작입니다:

1. <http://127.0.0.1:8000/admin/reviews/contributor/>에서 기여자 변경 목록을 방문하고 삭제할 기여자를 선택합니다. 기여자가 책의 저자인지 확인하십시오.
2. 삭제 버튼을 클릭하지만 확인 대화 상자에서 '예, 확실합니다'를 클릭하지 마십시오. 다음과 같은 메시지를 보게 될 것입니다:

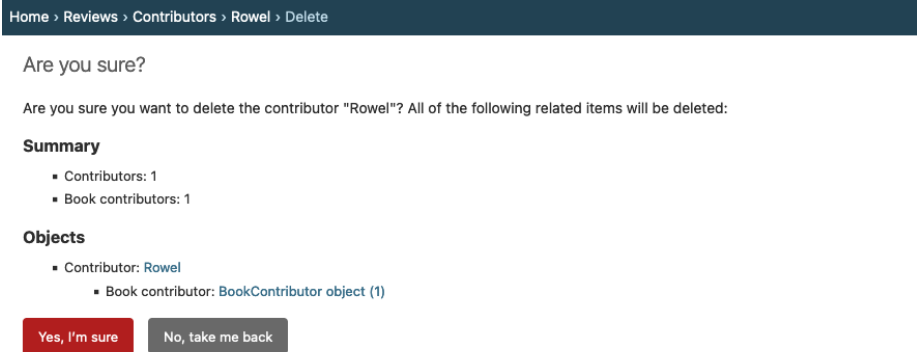


그림 4.34 – 삭제 확인 대화 상자

on_delete=CASCADE 인수에 따라 외래 키에 대해, 이 Contributor 객체를 삭제하면 aBookContributor 객체에 cascading 효과가 있을 것이라는 경고를 받습니다.

3. In the `reviews/models.py` file, modify the `Contributor` attribute of `BookContributor` to the following and save the file:

```
contributor = models.ForeignKey(Contributor,  
                                on_delete=models.PROTECT)
```

4. Now, try deleting the `Contributor` object again. You will see a message similar to the one in *Figure 4.35*:

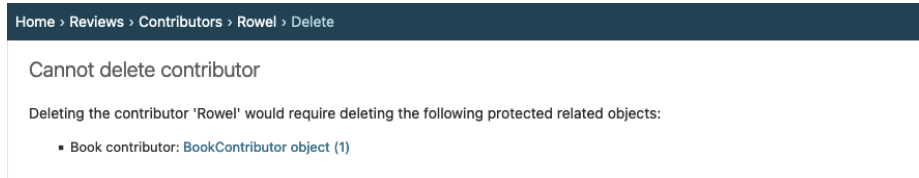


Figure 4.35 – Foreign key protection error

Because the `on_delete` argument is `PROTECT`, our attempt to delete the object with dependencies will throw an error. If we used this approach in our model, we would need to delete objects in the `ForeignKey` relation before we deleted the original object. In this case, it would mean deleting the `BookContributor` object before deleting the `Contributor` object.

5. Now that we have learned about how the admin app handles the `ForeignKey` relations, let's revert the `ForeignKey` definition in the `BookContributor` class to the following:

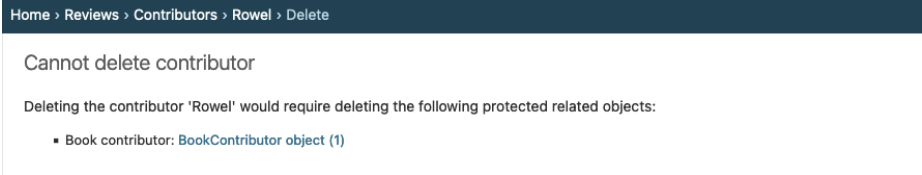
```
contributor = models.ForeignKey(Contributor,  
                                on_delete=models.CASCADE)
```

We have examined how the admin app's behavior adapts to the `ForeignKey` constraints that are expressed in model definitions. If the `on_delete` behavior is set to `models.PROTECT`, the admin app returns an error explaining why a protected object is blocking the deletion. This functionality can come in handy while building real-world apps, as there is often a chance of a manual error inadvertently leading to the deletion of important records. In the next section, we will look at how we can customize our admin app interface for a smoother user experience.

3. reviews/models.py 파일에서 BookContributor의 Contributor 속성을 다음과 같이 수정하고 파일을 저장하세요:

```
contributor = models.ForeignKey(Contributor,
                                on_delete=models.PROTECT)
```

4. 이제 Contributor 객체를 다시 삭제해 보세요. 그림 4.35와 유사한 메시지가 표시됩니다:



```
Home > Reviews > Contributors > Rowel > Delete
```

Cannot delete contributor

Deleting the contributor 'Rowel' would require deleting the following protected related objects:

- Book contributor: BookContributor object (1)

그림 4.35 - 외래 키 보호 오류

on_delete 인수가 PROTECT이기 때문에, 종속성이 있는 객체를 삭제하려고 하면 오류가 발생합니다. 모델에서 이 접근 방식을 사용했다면, 원래 객체를 삭제하기 전에 ForeignKey 관계의 객체를 삭제해야 합니다. 이 경우, Contributor 객체를 삭제하기 전에 BookContributor 객체를 삭제해야 한다는 의미입니다.

5. 이제 admin 앱이 ForeignKey 관계를 처리하는 방법에 대해 배웠으므로, BookContributor 클래스의 ForeignKey 정의를 다음과 같이 되돌리겠습니다:

```
contributor = models.ForeignKey(Contributor,
                                on_delete=models.CASCADE)
```

admin 앱의 동작이 모델 정의에서 표현된 ForeignKey 제약 조건에 어떻게 적응하는지 idx_b3c2ae8f 살펴보았습니다. on_delete 동작이 models.PROTECT로 설정되면, admin 앱은 보호된 객체가 삭제를 차단하는 이유를 설명하는 오류를 반환합니다. 이 기능은 실제 앱을 구축할 때 유용할 수 있습니다. 수동 idx_967a1038 오류로 인해 중요한 기록이 삭제될 가능성이 종종 있기 때문입니다. 다음 섹션에서는 사용자 경험을 개선하기 위해 admin 앱 인터페이스를 어떻게 사용자 정의할 수 있는지 살펴보겠습니다.

Customizing the admin interface

When first developing an application, the convenience of the default admin interface is excellent for building a rapid prototype of the app. Indeed, for many simpler applications or projects that require minimal data maintenance, this default admin interface may be entirely adequate. However, as the application matures to the point of release, the admin interface will generally need to be customized to facilitate more intuitive use and to robustly control data, subject to user permissions. You might want to retain certain aspects of the default admin interface and, at the same time, make some tweaks to certain features to better suit your purposes. For example, you would want the **Publisher** list to show the complete names of the publishing houses instead of `Publisher(1)`, `Publisher(2)`, and so on. In addition to the aesthetic appeal, this makes it easier to use and navigate through the app.

In this section, we will be examining the `AdminSite` object and learning about some of its important properties that can be used to modify the global appearance of the admin app. Then, we will examine approaches to subclassing `AdminSite` within our project so that we can customize it from its default appearance. We will end by testing our knowledge of these techniques by creating a new project and customizing its admin interface.

Site-wide Django admin customizations

We saw in *Figure 4.2* a page titled **Log in | Django site admin** containing a **Django Administration** form. However, an administrative user of the Bookr application may be somewhat perplexed by all this Django jargon, and it would be very confusing and a recipe for error if they had to deal with multiple Django apps that all had identical admin apps. As a developer of an intuitive and user-friendly application, you should customize this. Global properties such as these are specified as attributes of the `AdminSite` object. The following table details some of the simplest customizations to improve the usability of your app's admin interface:

AdminSite Attribute	Base Value	Description
<code>site_title</code>	"Django site admin"	Populates the <code><title></code> tag on each page of the admin interface.
<code>site_header</code>	"Django administration"	Sets the header on the login form.

관리자 인터페이스 사용자 정의

애플리케이션을 처음 개발할 때, 기본 관리자 인터페이스의 편리함은 앱의 빠른 프로토타입을 구축하는 데 매우 유용합니다. 실제로, 많은 간단한 애플리케이션이나 최소한의 데이터 유지 관리가 필요한 프로젝트의 경우, 이 기본 관리자 인터페이스가 완전히 적합할 수 있습니다. 그러나 애플리케이션이 출시할 정도로 성숙해짐에 따라, 관리자 인터페이스는 일반적으로 더 직관적인 사용을 촉진하고 사용자 권한에 따라 데이터를 강력하게 제어하기 위해 사용자 정의해야 합니다. 기본 관리자 인터페이스의 특정 측면을 유지하면서 동시에 특정 기능을 조정하여 귀하의 목적에 더 잘 맞게 만들고 싶을 수 있습니다. 예를 들어, 출판사 목록이 Publisher(1), Publisher(2) 대신 출판사의 전체 이름을 표시하도록 하고 싶을 것입니다. 미적 매력 외에도, 이는 앱을 사용하고 탐색하는 데 더 쉽게 만듭니다.

이 섹션에서는 AdminSite 객체를 살펴보고 관리자 앱의 전반적인 모양을 수정하는 데 사용할 수 있는 몇 가지 중요한 속성에 대해 배울 것입니다. 그런 다음, 기본 모양에서 사용자 정의를 할 수 있도록 프로젝트 내에서 AdminSite를 서브클래싱하는 접근 방식을 살펴볼 것입니다. 마지막으로, 새로운 프로젝트를 만들고 그 관리자 인터페이스를 사용자 정의하여 이러한 기술에 대한 우리의 지식을 테스트할 것입니다.

사이트 전역 장고 관리자 사용자 정의

우리는 그림 4.2에서 Log in | Django site admin이라는 제목의 페이지를 보았고, 그 페이지에는 Django Administration 양식이 포함되어 있습니다. 그러나 Bookr 애플리케이션의 관리자는 이러한 모든 장고 전문 용어에 다소 혼란스러울 수 있으며, 동일한 관리자 앱을 가진 여러 장고 앱을 처리해야 한다면 매우 혼란스럽고 오류를 일으킬 수 있습니다. 직관적이고 사용자 친화적인 애플리케이션의 개발자로서, 이를 사용자 정의해야 합니다. 이러한 전역 속성은 AdminSite 객체의 속성으로 지정됩니다. 다음 표는 앱의 관리자 인터페이스의 사용성을 개선하기 위한 가장 간단한 사용자 정의 중 일부를 자세히 설명합니다:

AdminSite 속성	기본 값	설명
site_title	"장고 사이트 관리자"	<title> 태그를 채웁니다 관리 인터페이스의 각 페이지에서.
사이트 헤더	"장고 관리"	로그인 폼의 헤더를 설정 합니다.

AdminSite Attribute	Base Value	Description
<code>index_title</code>	"Site administration"	Sets the heading on the admin index page (where the models are listed).
<code>index_template</code>	None	Provides the path to find the admin index template. If unset, the <code>admin/index.html</code> template is used.
<code>app_index_template</code>	None	Provides the path to find the app admin index template. If unset, the <code>admin/app_index.html</code> template is used.
<code>login_template</code>	None	Provides the path to find the login template. If unset, the <code>admin/login.html</code> template is used.
<code>logout_template</code>	None	Provides the path to find the logout template. If unset, the <code>registration/logged_out.html</code> template is used.
<code>password_change_template</code>	None	Provides the path to find the password change template. If unset, the <code>registration/password_change_form.html</code> template is used.

AdminSite 속성	기본 값	설명
index_title	"사이트 관리"	모델이 나열되는 관리자 인덱스 페이지의 제목을 설정합니다.
index_template	없음	관리자 인덱스 템플릿을 찾기 위한 경로를 제공합니다. 설정되지 않으면, <code>admin/index.htmltemplate</code> 이 사용됩니다.
app_index_template	없음	앱 관리자 인덱스 템플릿을 찾기 위한 경로를 제공합니다. 설정되지 않으면 <code>admin/app_index.htmltemplate</code> 이 사용됩니다.
login_template	없음	로그인 템플릿을 찾기 위한 경로를 제공합니다. 설정되지 않으면, <code>admin/login.html</code> 템플릿이 사용됩니다.
logout_template	없음	로그아웃 템플릿을 찾기 위한 경로를 제공합니다. 만약 없으면— 설정, 그 <code>registration/logged_out.htmltemplate</code> 가 사용됩니다.
비밀번호_변경_템플릿 e	없음	비밀번호 변경 템플릿을 찾기 위한 경로를 제공합니다. 설정하지 않으면 <code>registration/password_change_form.html</code> 템플릿이 사용됩니다.

AdminSite Attribute	Base Value	Description
password_change_done_template	None	Provides the path to find the password change done template. If unset, the registration/password_change_done.html template is used.

Figure 4.36 – Important AdminSite attributes

If this seems a little abstract, it might help if we examine the properties in this table using the Django shell.

Examining the AdminSite object from the Python shell

Let's take a deeper look at the AdminSite class. We have already encountered an object of the AdminSite class. It is the `admin.site` object that we used in the *Registering the reviews model* section. If the development server is not running, start it now with the `runserver` subcommand as follows :

```
python manage.py runserver
```

We can examine the `admin.site` object by importing the `admin` app in the Django shell, using the `manage.py` script again:

```
python manage.py shell
>>> from django.contrib import admin
```

We can interactively examine the default values of `site_title`, `site_header`, and `index_title` and see that they match the expected values of 'Django site admin', 'Django administration', and 'Site administration' that we have already observed on the rendered web pages of the Django admin app:

```
>>> admin.site.site_title
'Django site admin'
>>> admin.site.site_header
'Django administration'
>>> admin.site.index_title
'Site administration'
```

AdminSite 속성	기본 값	설명
비밀번호_변경_완료_template	없음	비밀번호 변경 완료 템플릿을 찾기 위한 경로를 제공합니다. 설정하지 않으면 registration/password_change_done.htmltemplate가 사용됩니다.

그림 4.36 - 중요한 AdminSite 속성

이것이 다소 추상적으로 보인다면, Django 셸을 사용하여 이 표의 속성을 살펴보면 도움이 될 수 있습니다.

Python 셸에서 AdminSite 객체 조사하기

AdminSite 클래스를 좀 더 깊이 살펴보겠습니다. 우리는 이미 AdminSite 클래스의 객체를 만났습니다. 그것은 우리가 리뷰 모델 등록 섹션에서 사용한 admin.site 객체입니다. 개발 서버가 실행되고 있지 않다면, 다음과 같이 runserver 하위 명령어로 지금 시작하세요:

```
python manage.py runserver
```

manage.py 스크립트를 다시 사용하여 Django 셸에서 admin 앱을 가져와 admin.site 객체를 조사할 수 있습니다:

```
python manage.py shell
>>> from django.contrib import admin
```

site_title, site_header 및 index_title의 기본값을 대화식으로 조사하고, 이 값들이 우리가 이미 Django admin 앱의 렌더링된 웹 페이지에서 관찰한 'Django site admin', 'Django administration', 'Site administration'의 예상 값과 일치하는지 확인할 수 있습니다:

```
>>> admin.site.site_title
'장고 사이트 관리자'
>>> admin.site.site_header
'장고 관리'
>>> admin.site.index_title
'사이트 관리'
```

The `AdminSite` class also specifies which forms and views are used to render the admin interface and determine its global behavior.

Now that we have examined the object's properties, we will attempt to subclass it so that we can customize them in our Django project.

Subclassing `AdminSite` — a first approach

We can make some modifications to the `reviews/admin.py` file. Instead of importing the `django.contrib.admin` module and using its `site` object, we will import `AdminSite`, subclass it, and instantiate our customized `admin_site` object. Consider the following code snippet. Here, `BookrAdminSite` is a subclass of `AdminSite` that contains custom values for `site_title`, `site_header`, and `index_title`; `admin_site` is an instance of `BookrAdminSite`, and we can use this instead of the default `admin.site` object to register our models. The `reviews/admin.py` file will look as follows:

```
from django.contrib.admin import AdminSite
from reviews.models import (Publisher, Contributor, Book,
                             BookContributor, Review)

class BookrAdminSite(AdminSite):
    title_header = 'Bookr Admin'
    site_header = 'Bookr administration'
    index_title = 'Bookr site admin'

admin_site = BookrAdminSite(name='bookr')

admin_site.register(Publisher)
admin_site.register(Contributor)
admin_site.register(Book)
admin_site.register(BookContributor)
admin_site.register(Review)
```

Now that we have created our own `admin_site` object that overrides the behavior of the `admin.site` object, we need to remove the existing references in our code to the `admin.site` object. In `bookr/bookr/urls.py`, we need to point `admin` to the new `admin_site` object and update our URL patterns. Otherwise, we would still be using the default admin site, and our customizations would be ignored. The change will look as follows:

```
from reviews.admin import admin_site
import reviews.views
```

AdminSite 클래스는 관리 인터페이스를 렌더링하는 데 사용되는 양식과 뷰를 지정하고 전반적인 동작을 결정합니다.

이제 객체의 속성을 살펴보았으므로, Django 프로젝트에서 이를 사용자 정의할 수 있도록 서브클래싱을 시도해 보겠습니다.

AdminSite 서브클래싱 – 첫 번째 접근 방식

reviews/admin.py 파일에 몇 가지 수정을 할 수 있습니다. django.contrib.admin 모듈을 가져오고 그 사이트 객체를 사용하는 대신, AdminSite를 가져오고 서브클래싱한 후 사용자 정의 admin_site 객체를 인스턴스화할 것입니다. 다음 코드 스니펫을 고려해 보세요. 여기서 BookrAdminSite는 site_title, site_header 및 index_title에 대한 사용자 정의 값을 포함하는 AdminSite의 서브클래스이며, admin_site는 BookrAdminSite의 인스턴스입니다. 우리는 이를 사용하여 기본 admin.site 객체 대신 모델을 등록할 수 있습니다. reviews/admin.py 파일은 다음과 같이 보일 것입니다:

```
from django.contrib.admin import AdminSite
from reviews.models import (Publisher, Contributor, Book,
                             BookContributor, Review)

class BookrAdminSite(AdminSite):
    title_header = 'Bookr Admin'
    site_header = 'Bookr administration'
    index_title = 'Bookr site admin'

admin_site = BookrAdminSite(name='bookr')

admin_site.register(Publisher)
admin_site.register(Contributor)
admin_site.register(Book)
admin_site.register(BookContributor)
admin_site.register(Review)
```

이제 admin.site 객체의 동작을 재정의하는 자체 admin_site 객체를 생성했으므로, 코드에서 admin.site 객체에 대한 기존 참조를 제거해야 합니다. bookr/bookr/urls.py에서 admin을 새로운 admin_site 객체로 가리키고 URL 패턴을 업데이트해야 합니다. 그렇지 않으면 여전히 기본 관리 사이트를 사용하게 되고, 우리의 idx_d47239ed 사용자 정의는 무시될 것입니다. 변경 사항은 다음과 같이 보일 것입니다:

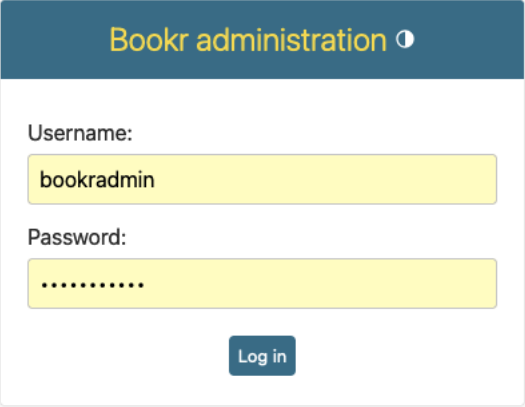
```
from reviews.admin import admin_site
import reviews.views
```



```
from django.urls import include, path

urlpatterns = [
    path("admin/", admin_site.urls),
    path("book-search", reviews.views.book_search),
    path("", include("reviews.urls")),
]
```

This produces the expected results on the login screen:



Bookr administration 🌙

Username:

Password:

Figure 4.37 – Customizing the login screen

However, now there is a problem; we have lost the interface for auth objects. Previously, the admin app discovered the models registered in `reviews/admin.py` and `django.contrib.auth.admin` through the auto-discovery process, but now we have overridden this behavior by creating a new `AdminSite`.

```
from django.urls import include, path

urlpatterns = [
    path("admin/", admin_site.urls),
    path("book-search", reviews.views.book_search),
    path("", include("reviews.urls")),
]
```

이것은 로그인 화면에서 예상되는 idx_882a64ec 결과를 생성합니다:

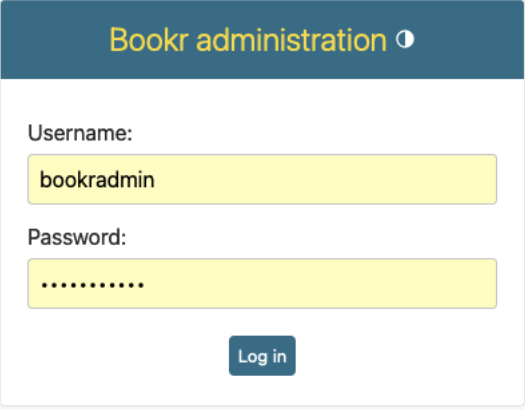


그림 4.37 - 로그인 화면 사용자 정의

하지만 이제 문제가 있습니다; 우리는 auth 객체에 대한 인터페이스를 잃어버렸습니다. 이전에, admin 앱은 reviews/admin.py와 django.contrib.auth.admin에 등록된 모델을 자동 발견 프로세스를 통해 발견했지만, 이제 우리는 새로운 AdminSite를 생성하여 이 동작을 재정의했습니다.

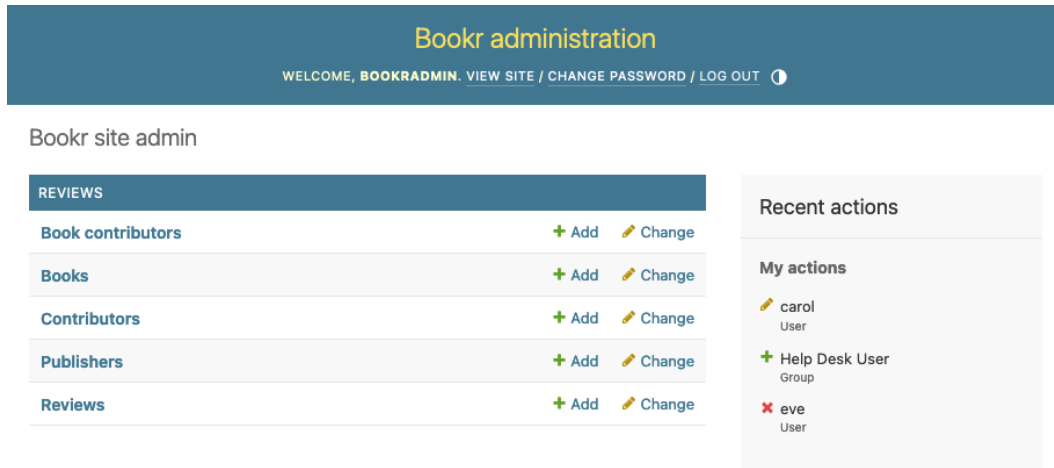


Figure 4.38 – Customized AdminSite is missing authentication and authorization

We could go down the path of referencing both the AdminSite objects to URL patterns in `bookr/bookr/urls.py`, but this approach would mean that we would end up with two separate admin apps for authentication and reviews. So, the URL `http://127.0.0.1:8000/admin` will take you to the original admin app derived from the `admin.site` object, while `http://127.0.0.1:8000/bookradmin` will take you to our `BookrAdminSite`, `admin_site`. This is not what we want to do, as we are still left with the admin app without the customizations that we added when we subclassed `BookrAdminSite`:

```
from django.contrib import admin
from reviews.admin import admin_site
from django.urls import path
urlpatterns = [path('admin/', admin.site.urls),
               path('bookradmin/', admin_site.urls),]
```

To avoid this problem, let's see what we can do in the next subsection.

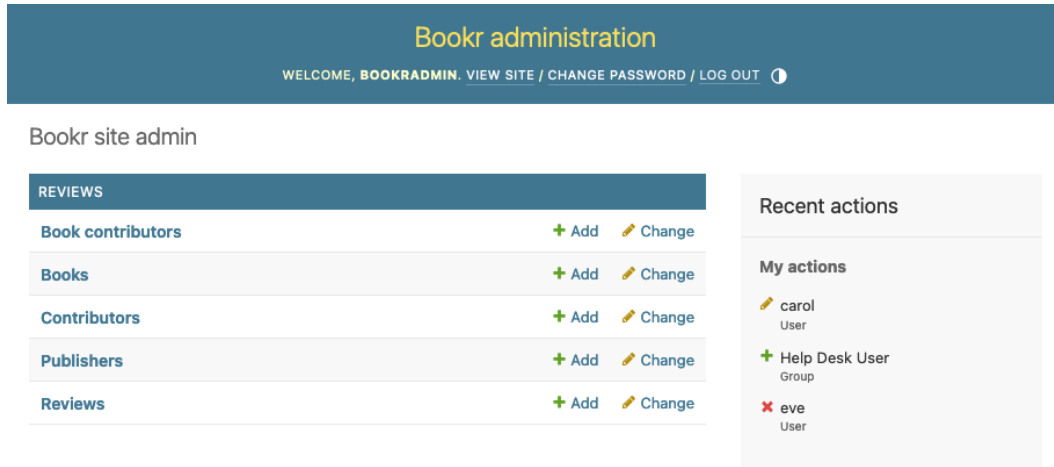


그림 4.38 - 맞춤형 AdminSite는 인증 및 권한 부여가 누락되었습니다

우리는 `inbookr/bookr/urls.py`에서 AdminSite 객체를 URL 패턴에 참조하는 경로를 선택할 수 있지만, 이 접근 방식은 인증 및 리뷰를 위한 두 개의 별도의 관리 앱이 생기게 됩니다. 따라서 URL `http://127.0.0.1:8000/admin`은 `admin.site` 객체에서 파생된 원래의 관리 앱으로 이동하게 되고, `http://127.0.0.1:8000/bookradmin`은 우리의 `BookrAdminSite`, `admin_site`로 이동하게 됩니다. 이는 우리가 원하는 것이 아니며, `idx_b3795fcf`를 `idx_bf284c34`로 `BookrAdminSite`를 서브클래스화할 때 추가한 사용자 정의가 없는 관리 앱이 남게 됩니다:

```
from django.contrib import admin
from reviews.admin import admin_site
from django.urls import path

urlpatterns = [path('admin/', admin.site.urls),
               path('bookradmin/',
                   admin_site.urls),]
```

이 문제를 피하기 위해, 다음 하위 섹션에서 우리가 할 수 있는 일을 살펴보겠습니다.

Subclassing AdminSite – a better way

This has been a clumsy problem with the Django admin interface that has led to a lot of ad hoc solutions in earlier versions. However, since Django 2.1 came out, there has been a simple way of integrating a customized interface for the admin app without breaking auto-discovery or any of its other default features. Let's see how to do this with the following steps:

1. As BookrAdminSite is project-specific, the code does not really belong under our reviews folder. So, move BookrAdminSite to a new file, bookr/bookr/admin.py, at the top level of the Bookr project directory:

```
from django.contrib import admin

class BookrAdminSite(admin.AdminSite):
    title_header = 'Bookr Admin'
    site_header = 'Bookr administration'
    index_title = 'Bookr site admin'
```

The URL settings path in bookr/bookr/urls.py changes to path('admin/', admin.site.urls), and we define ReviewsAdminConfig.

2. Add a file called reviews/adminconfig.py containing these lines:

```
from django.contrib.admin.apps import AdminConfig

class ReviewsAdminConfig(AdminConfig):
    default_site = "admin.BookrAdminSite"
```

3. Replace django.contrib.admin with reviews.apps.ReviewsAdminConfig, so that INSTALLED_APPS in the bookr/bookr/settings.py file will look as follows:

```
INSTALLED_APPS = [
    'reviews.adminconfig.ReviewsAdminConfig',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'reviews']
```

With the ReviewsAdminConfig specification of default_site, we no longer need to replace references to admin.site with a custom AdminSite object, admin_site. We can

AdminSite 서브클래싱 – 더 나은 방법

장고 관리 인터페이스에서 발생했던 불편한 idx_75be1009 문제로 인해 이전 버전에서는 많은 임시 해결책이 필요했습니다. 그러나 Django 2.1이 출시된 이후로는 자동 검색이나 다른 기본 idx_20bc6054 기능을 손상시키지 않고 관리 앱을 위한 맞춤형 인터페이스를 통합하는 간단한 방법이 생겼습니다. 다음 단계에 따라 이를 수행하는 방법을 살펴보겠습니다:

1. BookrAdminSite는 프로젝트 전용이므로 코드는 실제로 우리의 리뷰 폴더에 속하지 않습니다. 따라서 BookrAdminSite를 Bookr 프로젝트 디렉토리의 최상위에 있는 새 파일 bookr/bookr/admin.py로 이동합니다:

```
from django.contrib import admin

class BookrAdminSite(admin.AdminSite):
    title_header = 'Bookr Admin'
    site_header = 'Bookr administration'
    index_title = 'Bookr 사이트 관리자'
```

bookr/bookr/urls.py의 URL 설정 경로가 path('admin/', admin.site.urls)로 변경되며, ReviewsAdminConfig를 정의합니다.

2. 다음 줄을 포함하는 reviews/adminconfig.py라는 파일을 추가합니다:

```
from django.contrib.admin.apps import AdminConfig

class ReviewsAdminConfig(AdminConfig):
    default_site = "admin.BookrAdminSite"
```

3. django.contrib.admin을 reviews.apps.ReviewsAdminConfig로 교체하므로 idx_10f16648는 bookr/bookr/settings.py 파일의 INSTALLED_APPS는 다음과 같이 보일 것입니다:

```
INSTALLED_APPS = [
    'reviews.adminconfig.ReviewsAdminConfig',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'reviews']
```

기본 사이트의 ReviewsAdminConfig 사양을 사용하면 admin.site에 대한 참조를 사용자 정의 AdminSite 객체인 admin_site로 교체할 필요가 없습니다. 우리는 할 수 있습니다

replace those `admin_site` calls with the `admin.site` calls that we had originally. Now, `reviews/admin.py` reverts to the following:

```
from django.contrib import admin
from reviews.models import (Publisher, Contributor,
                             Book, BookContributor, Review)

admin.site.register(Publisher)
admin.site.register(Contributor)
admin.site.register(Book, BookAdmin)
admin.site.register(BookContributor)
admin.site.register(Review)
```

There are other aspects of `AdminSite` that we can customize, but we will look at these in *Chapter 9*, once we have a fuller understanding of Django's templates and forms.

The following activity is aimed at testing the skills that you have learned to configure and customize the Django admin app for a new Django project.

Activity 4.02 – Customizing the AdminSite object

You have learned how to modify attributes of the `AdminSite` object in a Django project. This activity will challenge you to use these skills to customize a new project and override its site title, site header, and index header. Also, you will replace the logout message by creating a project-specific template and setting it in our custom `AdminSite` object. You are developing a Django project that implements a message board called *Comment8or*. *Comment8or* is geared toward a technical demographic, so you need to make the phraseology succinct and abbreviated:

- The *Comment8or* admin site will be referred to as `c8admin`. This will appear on the site header and index title.
- For the title header, it will say `c8 site admin`.
- The default Django admin logout message is `Thanks for spending some quality time with the Web site today`. In *Comment8or*, it will say `Bye from c8admin`.

These are the steps that you need to follow to complete this activity:

1. Following the process that you learned in *Chapter 1*, create a new Django project called `comment8or`, an app called `messageboard`, and run the migrations. Create a superuser called `c8admin`.

원래 있던 `admin.site` 호출로 `admin_site` 호출을 교체하세요. 이제 `reviews/admin.py`는 다음과 같이 되돌아갑니다:

```
from django.contrib import admin
from reviews.models import (Publisher, Contributor,
                             Book, BookContributor, Review)

admin.site.register(Publisher)
admin.site.register(Contributor)
admin.site.register(Book, BookAdmin)
admin.site.register(BookContributor)
admin.site.register(Review)
```

AdminSite의 다른 측면을 사용자 정의할 수 있지만, Django의 템플릿과 폼에 대한 더 완전한 이해가 생긴 후인 9장에서 이들을 살펴보겠습니다.

다음 `idx_b1e5f903` 활동은 `idx_b8d89374` Django 관리 앱을 새 Django 프로젝트에 맞게 구성하고 사용자 정의하는 데 배운 기술을 테스트하는 것을 목표로 합니다.

활동 4.02 – AdminSite 객체 사용자 정의

`idx_c4974ec0`에서 Django 프로젝트의 AdminSite 객체 속성을 수정하는 방법을 배웠습니다. 이 활동은 이러한 기술을 사용하여 새 프로젝트를 사용자 정의하고 사이트 제목, 사이트 헤더 및 인덱스 헤더를 재정의하도록 도전할 것입니다. 또한, 프로젝트 전용 템플릿을 생성하고 이를 사용자 정의 AdminSite 객체에 설정하여 로그아웃 메시지를 교체할 것입니다. 당신은 Comment8or라는 메시지 보드를 구현하는 Django 프로젝트를 개발하고 있습니다. Comment8or는 기술적인 인구 통계를 겨냥하고 있으므로 문구를 간결하고 축약해야 합니다:

- Comment8or 관리 사이트는 `c8admin`으로 언급됩니다. 이는 사이트 헤더와 인덱스 제목에 나타납니다.
- 제목 헤더에는 `c8` 사이트 관리라고 표시됩니다.
- 기본 Django 관리 로그아웃 메시지는 오늘 웹사이트와 함께 좋은 시간을 보내주셔서 감사합니다입니다. Comment8or에서는 `c8admin`에서 안녕이라고 표시됩니다.

이 활동을 완료하기 위해 따라야 할 단계는 다음과 같습니다:

1. 1장에 배운 프로세스를 따라 `comment8or`라는 새로운 Django 프로젝트와 `messageboard`라는 앱을 만들고 마이그레이션을 실행합니다. `c8admin`이라는 슈퍼유저를 만듭니다.

2. In the Django source code, there is a template for the logout page located in `django/contrib/admin/templates/registration/logged_out.html`. Make a copy of it in your project's directory under `bookr/comment8or/templates/comment8or` and modify the message in the template following the requirements.
 3. Inside the project, create an `admin.py` file that implements a custom `AdminSite` object. Set the appropriate values for the `index_title`, `title_header`, `site_header`, and `logout_template` attributes, based on the requirements.
 4. Add a custom `AdminConfig` subclass to `bookr/messageboard/adminconfig.py`.
 5. Replace the admin app with the custom `AdminConfig` subclass in `bookr/comment8or/settings.py`.
 6. Configure the `TEMPLATES` setting so that the project's template is discoverable.
- When the project was first created, the login page looked like this:

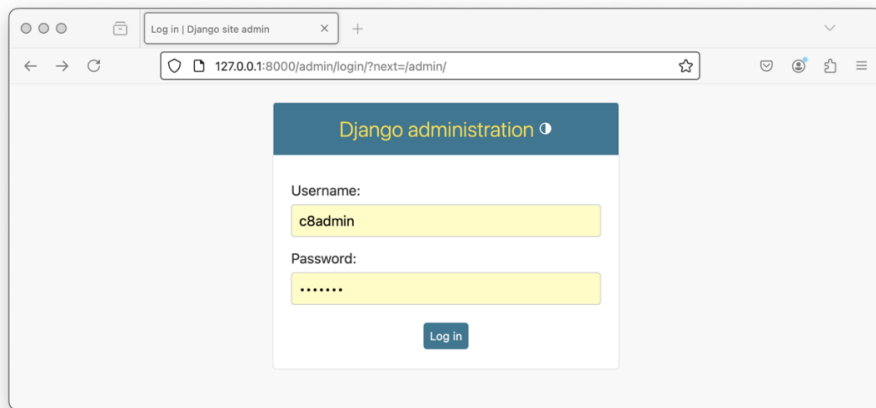


Figure 4.39 – Login page for the project

2. 장고 소스 코드에는 `django/contrib/admin/templates/registration/logged_out.html`에 위치한 로그아웃 페이지의 템플릿이 있습니다. `yourproject`의 디렉토리 아래 `bookr/comment8or/templates/comment8or`에 `idx_d585c47ea_idx_52be0d0e`의 복사본을 만들고 템플릿의 메시지를 요구 사항에 따라 수정하십시오.
3. 프로젝트 내에 custom AdminSite 객체를 구현하는 `admin.py` 파일을 만듭니다. 요구 사항에 따라 `index_title`, `title_header`, `site_header` 및 `logout_template` 속성에 적절한 값을 설정합니다.
4. `bookr/messageboard/adminconfig.py`에 사용자 정의 AdminConfig 하위 클래스를 추가합니다.
5. `bookr/comment8or/settings.py`에서 `admin` 앱을 사용자 정의 AdminConfig 하위 클래스로 교체합니다.
6. TEMPLATES 설정을 구성하여 프로젝트의 템플릿을 발견할 수 있도록 합니다. 프로젝트가 처음 생성되었을 때 로그인 페이지는 다음과 같았습니다:

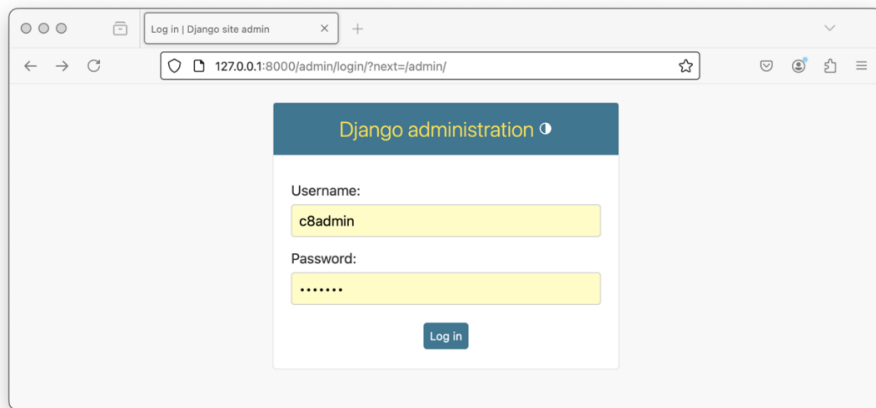


그림 4.39 - 프로젝트의 로그인 페이지

The app index page looked like this:

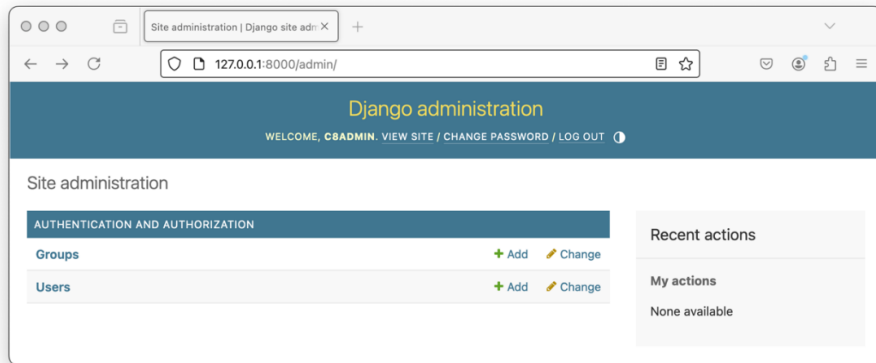


Figure 4.40 – App index page for the project

Finally, the logout page looked like this:

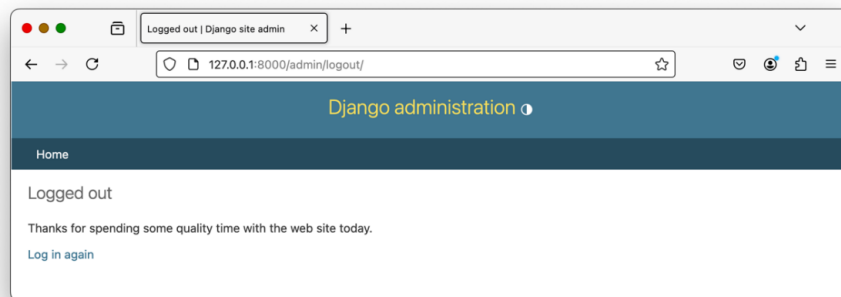


Figure 4.41 – Logout page for the project

idx_b0ac41acapp_idx_5105192bindex 페이지는 다음과 같았습니다:

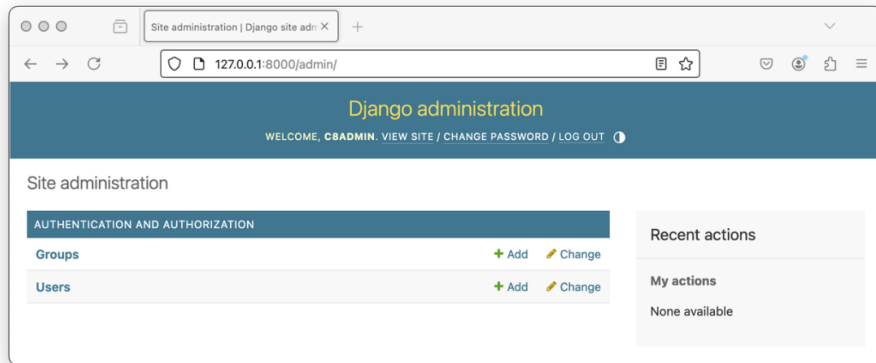


그림 4.40 - 프로젝트의 앱 인덱스 페이지

마지막으로, 로그아웃 페이지는 다음과 같습니다:

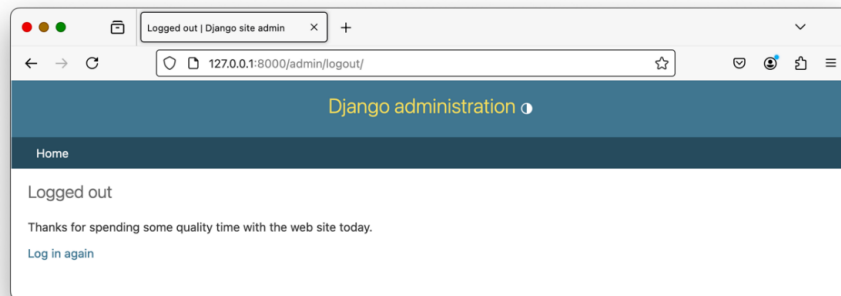


그림 4.41 - 프로젝트의 로그아웃 페이지

After you have completed this activity with all the customizations, the login page will look like this:

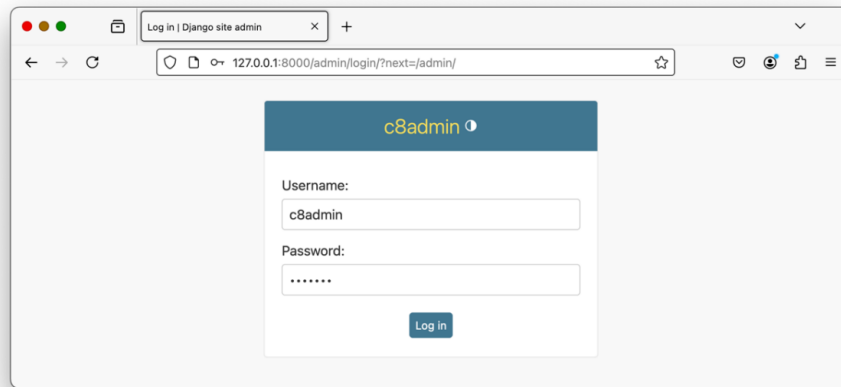


Figure 4.42 – Login page after customization

The app index page will look like this:

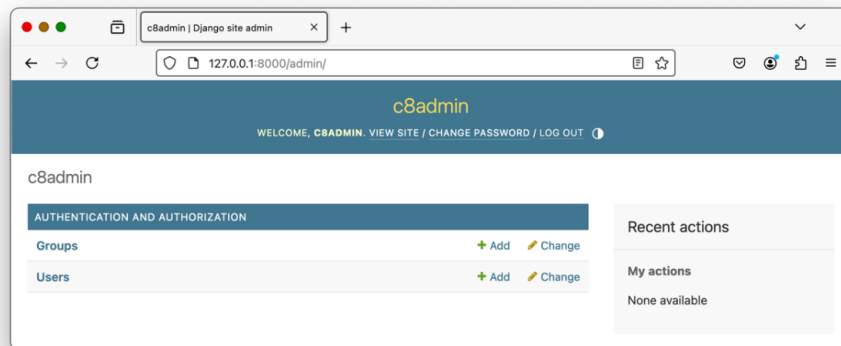


Figure 4.43 – App index page after customization

이 활동을 모든 사용자 정의와 함께 완료한 후, 로그인 페이지는 다음과 같이 보일 것입니다:

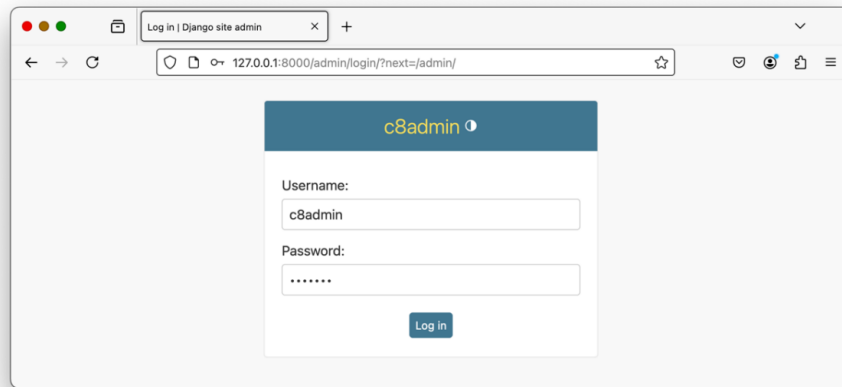


그림 4.42 - 사용자 정의 후 로그인 페이지

앱 인덱스 페이지는 다음과 같이 보일 것입니다:

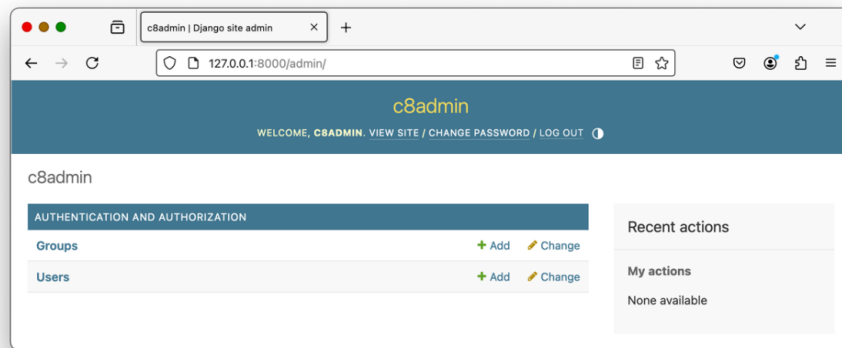


그림 4.43 - 사용자 정의 후 앱 인덱스 페이지

Finally, the logout page will look like this:

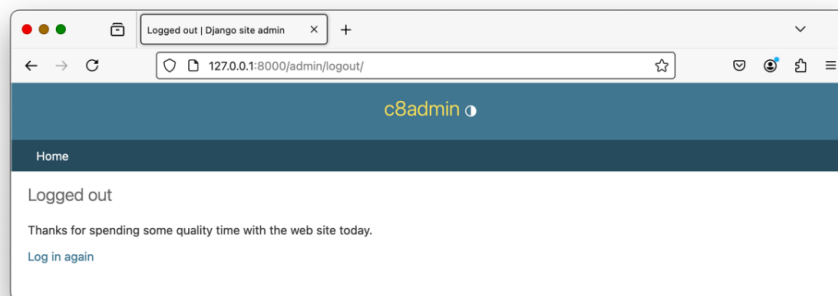


Figure 4.44 – Logout page after customization

You have successfully customized the admin app by subclassing `AdminSite`.

In the next section, we will examine the `AdminSite` object and learn about some of its important properties that can be used to modify the global appearance of the admin app. Then, we will examine approaches to subclassing `AdminSite` within our project so that we can customize it away from its default appearance. We will end by testing our knowledge of these techniques by creating a new project and customizing its admin interface.

Customizing the `ModelAdmin` classes

Now that we've learned how a subclassed `AdminSite` can be used to customize the global appearance of the admin app, we will look at how to customize the admin app's interface to individual models. Owing to the admin interface being generated automatically from the models' structure, it has an overly generic appearance and needs to be customized for the sake of aesthetics and usability. Click one of the **Books** links in the admin app and compare it to the **Users** link. Both links take you to the change list pages. These are the pages that a Bookr administrator visits when they want to add new books or add or alter the privileges of a user. As explained previously in the *Change lists* section, a change list page presents a list of model objects with the option of selecting a group of them for bulk deletion (or other bulk activity), examining an individual object to edit it, or adding a new object. Notice the difference between the two change list pages with a view to making our vanilla **Books** page as fully featured as the **Users** page. The following screenshot from the **Authentication and Authorization** app contains useful features such as a search bar, sortable column headers for important user fields, and a result filter:

마침내 로그아웃 페이지는 다음과 같이 보일 것입니다:

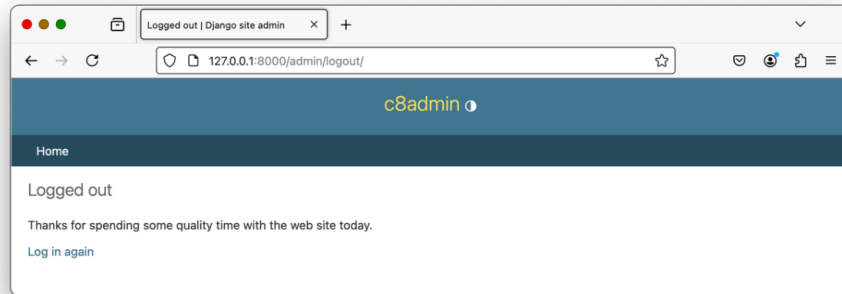


그림 4.44 - 사용자 정의 후 로그아웃 페이지

AdminSite를 서브클래싱하여 관리 앱을 성공적으로 사용자 정의했습니다.

다음 섹션에서는 AdminSite 객체를 살펴보고 관리 앱의 전반적인 모양을 수정하는 데 사용할 수 있는 몇 가지 중요한 속성에 대해 배웁니다. 그런 다음, 프로젝트 내에서 AdminSite를 서브클래싱하는 방법을 살펴보고 기본 모양에서 사용자 정의할 수 있도록 합니다. 마지막으로, 새로운 프로젝트를 만들고 관리 인터페이스를 사용자 정의하여 이러한 기술에 대한 우리의 지식을 테스트할 것입니다.

ModelAdmin 클래스 사용자 정의

서브클래싱된 AdminSite가 관리 앱의 전반적인 모양을 사용자 정의하는 데 어떻게 사용될 수 있는지 배웠으므로, 이제 관리 앱의 인터페이스를 개별 모델에 맞게 사용자 정의하는 방법을 살펴보겠습니다. 관리 인터페이스는 모델의 구조에서 자동으로 생성되기 때문에 지나치게 일반적인 모양을 가지고 있으며 미적 및 사용성을 위해 사용자 정의해야 합니다. 관리 앱의 책 링크 중 하나를 클릭하고 사용자 링크와 비교해 보세요. 두 링크 모두 변경 목록 페이지로 이동합니다. 이 페이지는 책 관리자가 새 책을 추가하거나 사용자 권한을 추가 또는 변경하고자 할 때 방문하는 페이지입니다. 변경 목록 섹션에서 설명한 바와 같이, 변경 목록 페이지는 모델 객체 목록을 표시하며, 이 목록에서 일괄 삭제(또는 기타 일괄 작업)를 위해 그룹을 선택하거나, 개별 객체를 검토하여 편집하거나, 새 객체를 추가할 수 있는 옵션이 있습니다. 우리의 기본 Books 페이지를 Users 페이지만큼 기능이 풍부하게 만들기 위해 두 변경 목록 페이지 간의 차이를 주목하세요. 인증 및 권한 부여 앱의 다음 스크린샷에는 검색 바, 중요한 사용자 필드를 위한 정렬 가능한 열 헤더, 결과 필터와 같은 유용한 기능이 포함되어 있습니다:

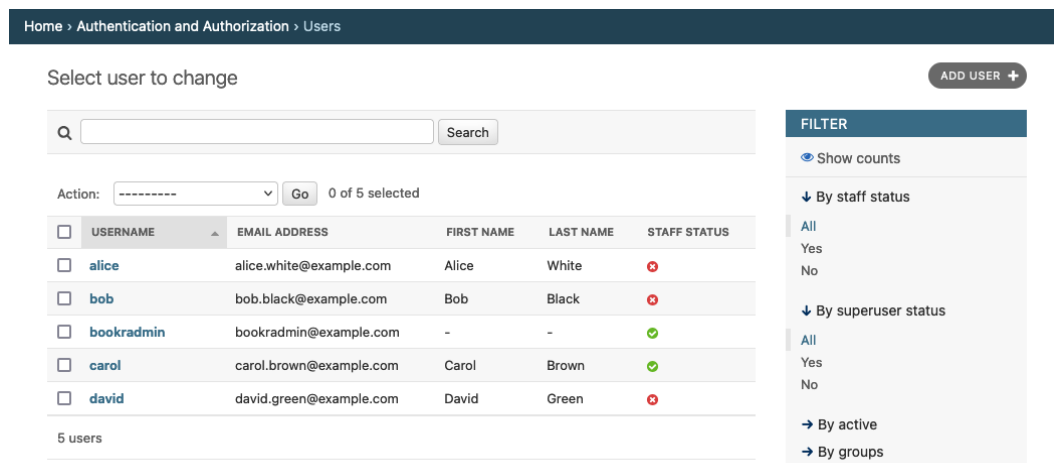


Figure 4.45 – The Users change list contains the customized ModelAdmin features

By subclassing `ModelAdmin`, we can create a class that sets the functionality for a specific model's admin pages.

In this section, we will learn how the `ModelAdmin` classes can be developed to customize the fields on the change list page and to implement filters and search bars. We will look at how model fields can be grouped in admin CRUD forms or be excluded from the form if the fields should retain a default or system-generated value.

The list display fields

On the **Users** change list page, you will see the following:

- There is a list of user objects presented, summarized by their **USERNAME**, **EMAIL ADDRESS**, **FIRST NAME**, **LAST NAME**, and **STAFF STATUS** attributes.
- These individual attributes are sortable. The sorting order can be changed by clicking the headers.
- There is a search bar at the top of the page.
- In the right-hand column, there is a selection filter that allows the selection of several user fields, including some that do not appear in the list display.

However, the behavior for the **Books** change list page is a lot less helpful. The books are listed by their titles but not in alphabetical order. The title column is not sortable, and there are no filter or search options present:

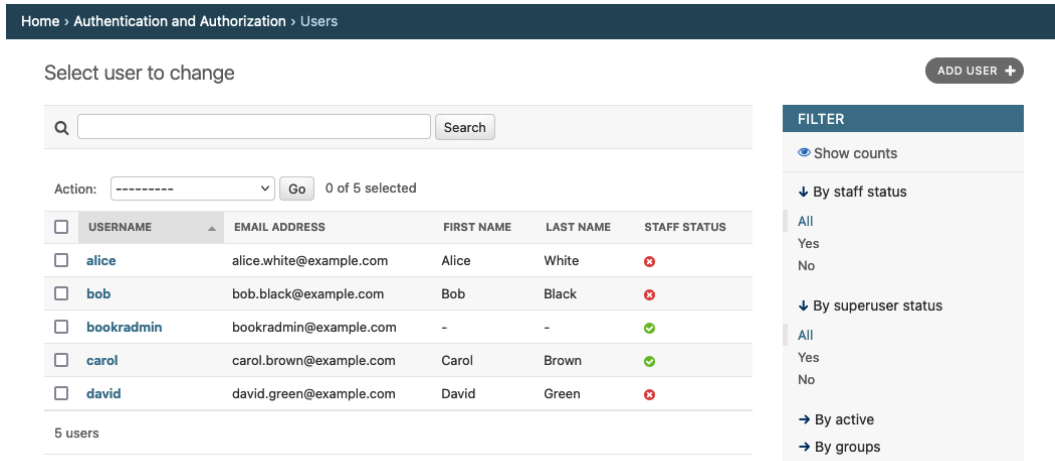


그림 4.45 - 사용자 변경 목록에는 사용자 정의된 ModelAdmin 기능이 포함되어 있습니다.

ModelAdmin을 서브클래싱하여 특정 모델의 관리 페이지에 대한 기능을 설정하는 클래스를 만들 수 있습니다.

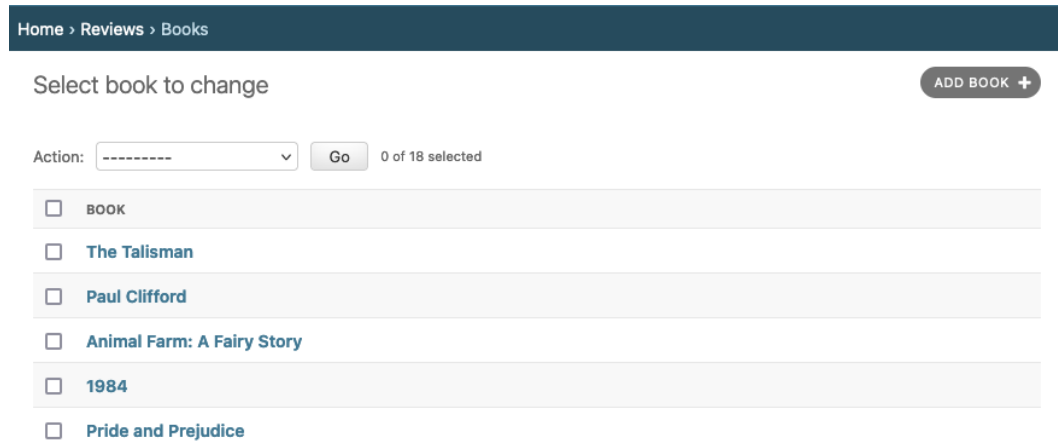
이 섹션에서는 ModelAdmin 클래스를 개발하여 변경 목록 페이지의 필드를 사용자 정의하고 필터 및 검색 바를 구현하는 방법을 배웁니다. idx_1a26d291 필드가 기본값 또는 시스템 생성 값을 유지해야 하는 경우 관리 CRUD 양식에서 모델 필드를 그룹화하거나 양식에서 제외하는 방법을 살펴보겠습니다.

목록 표시 필드

사용자 변경 idx_0ae599aa 목록 페이지에서 다음을 볼 수 있습니다:

- 사용자 객체 목록이 제공되며, USERNAME, EMAILADDRESS, FIRST NAME, LAST NAME 및 STAFF STATUS 속성으로 요약됩니다.
- 이 개별 속성은 정렬할 수 있습니다. 정렬 순서는 헤더를 클릭하여 변경할 수 있습니다.
- 페이지 상단에 검색 바가 있습니다.
- 오른쪽 옆에는 목록 표시에는 나타나지 않는 일부를 포함하여 여러 사용자 필드를 선택할 수 있는 선택 필터가 있습니다.

그러나 도서 변경 목록 페이지의 동작은 훨씬 덜 유용합니다. 도서는 제목별로 나열되지만 알파벳 순서는 아닙니다. 제목 열은 정렬할 수 없으며 필터나 검색 옵션이 없습니다:



Home > Reviews > Books

Select book to change ADD BOOK +

Action: 0 of 18 selected

- ☐ BOOK
- ☐ The Talisman
- ☐ Paul Clifford
- ☐ Animal Farm: A Fairy Story
- ☐ 1984
- ☐ Pride and Prejudice

Figure 4.46 – The Books change list

Recall from *Chapter 2*, that we defined the `__str__` methods on the `Publisher`, `Book`, and `Contributor` classes. In the case of the `Book` class, it had a `__str__()` representation that returns the book object's title:

```
class Book(models.Model):  
    ...  
    def __str__(self):  
        return self.title
```

If we had not defined the `__str__()` method on the `Book` class, it would have inherited it from the base `Model` class, `django.db.models.Model`.

This base class provides an abstract way to give a string representation of an object. For example, when we have `Book` with a primary key, in this case, the `id` field with a value of 17, then we will end up with a string representation of `Book object (17)`:

Home > Reviews > Books

Select book to change ADD BOOK +

Action: 0 of 18 selected

☐	BOOK
☐	The Talisman
☐	Paul Clifford
☐	Animal Farm: A Fairy Story
☐	1984
☐	Pride and Prejudice

그림 4.46 - 책 변경 목록

2장에서 정의한 `__str__` 메서드를 `Publisher`, `Book`, `Contributor` 클래스에 대해 기억하십시오. `Book` 클래스의 경우, 책 객체의 제목을 반환하는 `__str__()` 표현이 있었습니다:

```
class Book(models.Model):
    ...
    def __str__(self):
        return self.title
```

`Book` 클래스에 `__str__()` 메서드를 정의하지 않았다면, `django.db.models.Model`의 기본 `Model` 클래스에서 상속받았을 것입니다.

이 기본 클래스는 객체의 문자열 표현을 제공하는 추상적인 방법을 제공합니다. 예를 들어, 기본 키가 있는 `Book`이 있을 때, 이 경우 `id` 필드의 값이 17인 경우, 그러면 `idx_57f76422`는 `Book` 객체(17)의 문자열 표현을 갖게 됩니다:

The screenshot shows the top of a Django Admin change list for the 'Books' model. At the top, there is an 'Action:' dropdown menu with a downward arrow, a 'Go' button, and the text '0 of 18 selected'. Below this is a table with six rows. Each row starts with a checkbox, followed by the text 'BOOK', 'Book object (18)', 'Book object (17)', 'Book object (16)', 'Book object (15)', and 'Book object (14)' respectively.

Figure 4.47 – The Books change list using the model `__str__` representation

It could be useful in our application to represent a Book object as a composite of several fields. For example, if we wanted the books to be represented as Title (ISBN), the following code snippet would produce the desired results:

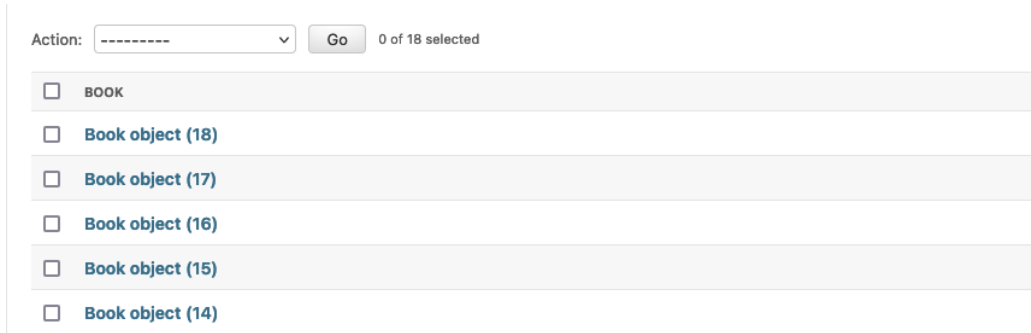
```
class Book(models.Model):
    ...
    def __str__(self):
        return f"{self.title} ({self.isbn})"
```

This is a useful change in and of itself, as it makes the representation of the object more intuitive in the app:

The screenshot shows a portion of the Django Admin change list for the 'Books' model, displaying the custom string representation. At the top, there is an 'Action:' dropdown menu with a downward arrow, a 'Go' button, and the text '0 of 18 selected'. Below this is a table with six rows. Each row starts with a checkbox, followed by the text 'BOOK', 'The Talisman (9781451697216)', 'Paul Clifford (9781719053167)', 'Animal Farm: A Fairy Story (9780151002177)', '1984 (9781328869333)', and 'Pride and Prejudice (9780141439518)' respectively.

Figure 4.48 – A portion of the Books change list with the custom string representation

We are not limited to using the `__str__` representation of the object in the `list_display` field. The columns that appear in the list display are determined by the `ModelAdmin` class of the



Action: ----- Go 0 of 18 selected

- ☐ BOOK
- ☐ Book object (18)
- ☐ Book object (17)
- ☐ Book object (16)
- ☐ Book object (15)
- ☐ Book object (14)

그림 4.47 - 모델 `__str__` 표현을 사용하는 도서 변경 목록

우리 애플리케이션에서 책 객체를 여러 필드의 조합으로 표현하는 것이 유용할 수 있습니다. 예를 들어, 책이 제목 (ISBN)으로 표현되기를 원한다면, 다음 코드 스니펫이 원하는 결과를 생성할 것입니다:

```
class Book(models.Model):
    ...
    def __str__(self):
        return f"{self.title} ({self.isbn})"
```

이것은 그 자체로 유용한 변화이며, 앱에서 객체의 표현을 더 직관적으로 만듭니다:



Action: ----- Go 0 of 18 selected

- ☐ BOOK
- ☐ The Talisman (9781451697216)
- ☐ Paul Clifford (9781719053167)
- ☐ Animal Farm: A Fairy Story (9780151002177)
- ☐ 1984 (9781328869333)
- ☐ Pride and Prejudice (9780141439518)

그림 4.48 - 사용자 정의 문자열 표현이 있는 도서 변경 목록의 일부

우리는 `list_display` 필드에서 객체의 `__str__` 표현만 사용하는 것으로 제한되지 않습니다. 목록에 표시되는 열은 `ModelAdmin` 클래스에 의해 결정됩니다 `idx_2067978dof`

Django admin app. In the Django shell, we can import the `ModelAdmin` class and examine its `list_display` attribute:

```
python manage.py shell
>>> from django.contrib.admin import ModelAdmin
>>> ModelAdmin.list_display
('__str__',)
```

This explains why the default behavior of `list_display` is to display a single-column table of the objects' `__str__` representations so that we can customize the list display by overriding this value. The best practice is to subclass `ModelAdmin` for each object. If we wanted the book list display to contain two separate columns for **Title** and **ISBN**, rather than having a single column containing both values, as in *Figure 4.48*, we would subclass `ModelAdmin` as `BookAdmin` and specify the custom `list_display`. The benefit of doing this is that we are now able to sort books by title and by ISBN. We can add this class to `reviews/admin.py`:

```
class BookAdmin(admin.ModelAdmin):
    list_display = ("title", "isbn")
```

Now that we've created a `BookAdmin` class, we should reference it when we register our `reviews.models.Book` class with the admin site. In the same file, we also need to modify the model registration to use `BookAdmin` instead of the default value of `admin.ModelAdmin`, so the `admin.site.register` call now becomes the following:

```
admin.site.register(Book, BookAdmin)
```

Once these two changes have been made to the `reviews/admin.py` file, we will get a **Books** change list page that looks like this:

장고 관리자 앱. 장고 셸에서 `ModelAdmin` 클래스를 가져오고 그 `list_display` 속성을 살펴볼 수 있습니다:

```
python manage.py shell
>>> from django.contrib.admin import
ModelAdmin>>> ModelAdmin.list_display
(['__str__',])
```

이것은 `list_display`의 기본 동작이 객체의 `__str__` 표현을 단일 열 테이블로 표시하는 이유를 설명합니다. 이를 통해 이 값을 재정의하여 목록 표시를 사용자 정의할 수 있습니다. 모범 사례는 각 객체에 대해 `ModelAdmin`을 서브클래싱하는 것입니다. 만약 우리가 도서 목록 표시가 제목과 ISBN에 대한 두 개의 별도 열을 포함하도록 하고 싶다면, 그림 4.48과 같이 두 값을 포함하는 단일 열 대신, 우리는 `ModelAdmin`을 `BookAdmin`으로 서브클래싱하고 사용자 정의 `list_display`를 지정해야 합니다. 이렇게 하면 이제 제목과 ISBN으로 책을 정렬할 수 있는 이점이 있습니다. 이 클래스를 `reviews/admin.py`에 추가할 수 있습니다:

```
class BookAdmin(admin.ModelAdmin):
    list_display = ("title", "isbn")
```

이제 `BookAdmin` 클래스를 생성했으므로, 관리자 사이트에 `reviews.models.Book` 클래스를 등록할 때 이를 참조해야 합니다. 같은 파일에서 기본값인 `admin.ModelAdmin` 대신 `BookAdmin`을 사용하도록 모델 등록을 수정해야 하므로, `admin.site.register` 호출은 이제 다음과 같이 됩니다:

```
admin.site.register(Book, BookAdmin)
```

이 두 가지 변경 사항이 `reviews/admin.py` 파일에 적용되면, 다음과 같은 Bookschange 목록 `idx_fe164c74page`를 얻을 수 있습니다:

Action: 0 of 18 selected

☐	TITLE	ISBN NUMBER OF THE BOOK.
☐	The Talisman	9781451697216
☐	Paul Clifford	9781719053167
☐	Animal Farm: A Fairy Story	9780151002177
☐	1984	9781328869333
☐	Pride and Prejudice	9780141439518

Figure 4.49 – A portion of the Books change list with a two-column list display

This gives us a hint as to how flexible `list_display` is. It can take four types of values:

- It takes field names from the model, such as `title` or `isbn`.
- It takes a function that takes the model instance as an argument, such as this function that gives an initialized version of a person's name:

```
def initialled_name(obj):
    """ obj.first_names='Jerome David',
    obj.last_names='Salinger'=> 'Salinger, JD'
    obj.first_names='Plato', obj.last_names=''
    => 'Plato'
    """
    initials = "".join([name[0] for name in
                        obj.first_names.split(" ")])
    if obj.last_names:
        return "{obj.last_names}, {initials}"
    return obj.first_names

class ContributorAdmin(admin.ModelAdmin):
    list_display = (initialled_name,)
```

- It takes a method from the `ModelAdmin` subclass that takes the model object as a single argument. Note that this needs to be specified as a string argument, as it would be out of scope and undefined within the class:

```
class BookAdmin(admin.ModelAdmin):

    list_display = ("title", "isbn13")
```

Action: 0 of 18 selected

☐	TITLE	ISBN NUMBER OF THE BOOK.
☐	The Talisman	9781451697216
☐	Paul Clifford	9781719053167
☐	Animal Farm: A Fairy Story	9780151002177
☐	1984	9781328869333
☐	Pride and Prejudice	9780141439518

그림 4.49 - 두 개의 열 목록 표시가 있는 도서 변경 목록의 일부

이것은 `list_display`가 얼마나 유연한지를 알려줍니다. 모델에서 제목이나 isbn

- 과 같은 필드 이름을 가져올 수 있습니다.
- 모델 인스턴스를 인수로 받는 함수를 가져올 수 있습니다. 예를 들어, 사람의 이름의 초기화된 버전을 제공하는 이 함수와 같습니다:

```
def initialled_name(obj):
    """ obj.first_names='Jerome David',
    obj.last_names='Salinger'=> 'Salinger,
    JD'obj.first_names='Plato',
    obj.last_names=''=> 'Plato'
    """
    initials = "".join([name[0] for name in
                        obj.first_names.split(" ")])
    if obj.last_names:
        return "{obj.last_names}, {initials}"
    return obj.first_names

class ContributorAdmin(admin.ModelAdmin):
    list_display = (initialled_name,)
```

- 모델 객체를 단일 인수로 받는 `ModelAdmin` 서브클래스의 메서드를 사용합니다. 이는 문자열 인수로 지정해야 하며, 클래스 내에서 범위를 벗어나 정의되지 않음을 유의하세요:

```
class BookAdmin(admin.ModelAdmin):

    list_display = ("title", "isbn13")
```

```
def isbn13(self, obj):
    """ '9780316769174' => '978-0-31-676917-4'
        '0316769174' => '0316769174'
        None => '' """
    if obj.isbn:
        if len(obj.isbn)==13:
            return "-".join([obj.isbn[0:3],
                             obj.isbn[3:4], obj.isbn[4:6],
                             obj.isbn[6:12], obj.isbn[12:13]])
        return obj.isbn
    return ""
```

- It takes a method (or a non-field attribute) of the model class, such as `__str__`, as long as it accepts the model object as an argument. For example, we could convert `isbn13` to a method on the `Book` model class:

```
class Book(models.Model):

    def isbn13(self):
        """ '9780316769174' => '978-0-31-676917-4'
            '0316769174' => '0316769174'
            None => '' """
        if self.isbn:
            if len(self.isbn)==13:
                return "-".join([self.isbn[0:3],
                                 self.isbn[3:4], self.isbn[4:6],
                                 self.isbn[6:12],self.isbn[12:13]])
            return self.isbn
        return ""
```

Now, when viewing the **Books** change list at <http://127.0.0.1:8000/admin/reviews/book>, we can see the hyphenated ISBN13 field:

```
def isbn13(self, obj):
    """ '9780316769174' => '978-0-31-676917-4'
        '0316769174' => '0316769174'
        None => '' """
    if obj.isbn:
        if len(obj.isbn)==13:
            return "-".join([obj.isbn[0:3],
                             obj.isbn[3:4], obj.isbn[4:6],
                             obj.isbn[6:12], obj.isbn[12:13]])
        return obj.isbn
    return ""
```

- 모델 클래스의 `idx_86de4913method` (또는 비필드 속성)인 `__str__`과 같은 것을 사용하면 모델 객체를 인수로 받아들이는 한 가능합니다. 예를 들어, 우리는 `isbn13`을

Book 모델 클래스의 메서드로 변환할 수 있습니다:

```
class Book(models.Model):

    def isbn13(self):
        """ '9780316769174' => '978-0-31-676917-4'
            '0316769174' => '0316769174'
            None => '' """
        if self.isbn:
            if len(self.isbn)==13:
                return "-".join([self.isbn[0:3],
                                 self.isbn[3:4], self.isbn[4:6],
                                 self.isbn[6:12], self.isbn[12:13]])
            return self.isbn
        return ""
```

이제 <http://127.0.0.1:8000/admin/reviews/book>에서 도서 변경 목록을 볼 때, 하이픈이 있는 ISBN13 필드를 볼 수 있습니다:

Action:

Go

0 of 18 selected

☐	TITLE	ISBN-13
☐	The Talisman	978-1-45-169721-6
☐	Paul Clifford	978-1-71-905316-7
☐	Animal Farm: A Fairy Story	978-0-15-100217-7
☐	1984	978-1-32-886933-3
☐	Pride and Prejudice	978-0-14-143951-8

Figure 4.50 – A portion of the Books change list with the hyphenated ISBN13

It is worth noting that computed fields such as `__str__` or our `isbn13` methods do not make for sortable fields on the summary page unless specified. Also, we cannot include fields of the `ManyToManyField` type in `list_display`.

While columns that are derived from model attributes derive their column headers and properties from the field attributes, we can specify properties of display columns that are computed fields using the display decorator.

The display decorator

When using a callable in `list_display`, as in the cases of `initialled_name` and `isbn13`, we can use the `admin.display` decorator to specify the column name that will appear in the header of the change list using the `description` argument. We can also use it to get around the limitation of calculated fields not being sortable by specifying `ordering` on the callable. The `empty_value` argument can be used to specify how a `None` value or an empty string is displayed. The default `empty_value` display is a single dash character:

```
@admin.display(
    ordering="isbn",
    description="ISBN-13",
    empty_value="-"
)
def isbn13(self, obj):
    """ '9780316769174' => '978-0-31-676917-4'
        '0316769174' => '0316769174'
        None => ' ' """
    ...
```

Action: 0 of 18 selected

☐	TITLE	ISBN-13
☐	The Talisman	978-1-45-169721-6
☐	Paul Clifford	978-1-71-905316-7
☐	Animal Farm: A Fairy Story	978-0-15-100217-7
☐	1984	978-1-32-886933-3
☐	Pride and Prejudice	978-0-14-143951-8

그림 4.50 - 하이픈이 있는 ISBN13이 포함된 도서 변경 목록의 일부

`__str__` 또는 우리의 `isbn13` 메서드와 같은 계산된 필드는 명시되지 않는 한 요약 페이지에서 정렬 가능한 필드를 만들지 않는다는 점은 주목할 가치가 있습니다. 또한, `list_display`에 `ManyToManyField` 유형의 필드를 포함할 수 없습니다.

모델 속성에서 파생된 열은 `idx_20386e36` 필드 속성에서 열 헤더와 속성을 파생하지만, `display` 데코레이터를 사용하여 계산된 필드인 표시 열의 속성을 지정할 수 있습니다.

디스플레이 데코레이터

`list_display`에서 `idx_90afefdf` 호출 가능성을 사용할 때, 초기화된 이름과 `isbn13`의 경우처럼, `admin.display` 데코레이터를 사용하여 변경 목록의 헤더에 나타날 열 이름을 `description` 인수를 사용하여 지정할 수 있습니다. 또한, 호출 가능성에서 정렬을 지정하여 계산된 필드가 정렬할 수 없는 제한을 우회할 수 있습니다. `empty_value` 인수를 사용하여 `None` 값이나 빈 문자열이 표시되는 방식을 지정할 수 있습니다. 기본 `empty_value` 표시 값은 단일 대시 문자입니다:

```
@admin.display(
    ordering="isbn",
    description="ISBN-13",
    empty_value="-"
)

def isbn13(self, obj):
    """ '9780316769174' => '978-0-31-676917-4'
        '0316769174' => '0316769174'
        None => '' """
    ...
```


Filters are implemented using the `list_filter` attribute of a `ModelAdmin` subclass. In our Bookr app, filtering by book title or ISBN would be impractical, as it would produce a large list of filter options that return only one record. The filter that would occupy the right-hand side of the page would take up more space than the actual change list. A practical option would be to filter books by publisher. We defined a custom `__str__` method for the `Publisher` model that returns the publisher's name attribute, so our filter options will be listed as publisher names.

We can specify our change list filter in `reviews/admin.py` in the `BookAdmin` class like so:

```
list_filter = ("publisher",)
```

Here is how the **Books** change list page should look now:

Select book to change

Action: 0 of 6 selected

☐	TITLE	ISBN NUMBER OF THE BOOK.
☐	Hands-On Reinforcement Learning with Python	9781788836524
☐	Natural Language Processing with TensorFlow	9781788478311
☐	Deep Reinforcement Learning Hands-On	9781788834247
☐	Architects of Intelligence	9781789954531
☐	Hands-On Machine Learning for Algorithmic Trading	9781789346411
☐	Advanced Deep Learning with Keras	9781788629416

6 books

FILTER

By publisher

All

[Packt Publishing](#)

Harper Collins

Penguin Classics

Scribner

Bay Back Books

Simon and Schuster

Houghton Mifflin Harcourt

HardPress

Figure 4.52 – The Books change list page with the publisher filter

필터는 ModelAdmin 서브클래스의 `list_filter` 속성을 사용하여 구현됩니다. 우리Bookr 앱에서는 책 제목이나 ISBN으로 필터링하는 것이 비현실적입니다. 왜냐하면 이는 단 하나의 레코드만 반환하는 필터 옵션의 큰 목록을 생성하기 때문입니다. 페이지의 오른쪽에 위치할 필터는 실제 변경 목록보다 더 많은 공간을 차지할 것입니다. 실용적인 옵션은 출판사별로 책을 필터링하는 것입니다. 우리는 Publisher 모델에 대해 `idx_c8baf876출판사의 이름` 속성을 반환하는 사용자 정의 `__str__` 메서드를 정의했으므로, 우리의 필터 옵션은 출판사 이름으로 나열될 것입니다.

우리는 `reviews/admin.py`의 BookAdmin 클래스에서 변경 목록 필터를 다음과 같이 지정할 수 있습니다:

```
list_filter = ("publisher",)
```

이제 Books 변경 목록 페이지는 다음과 같아야 합니다:

Select book to change

ADD BOOK +

Action: Go 0 of 6 selected

☐	TITLE	ISBN NUMBER OF THE BOOK.
☐	Hands-On Reinforcement Learning with Python	9781788836524
☐	Natural Language Processing with TensorFlow	9781788478311
☐	Deep Reinforcement Learning Hands-On	9781788834247
☐	Architects of Intelligence	9781789954531
☐	Hands-On Machine Learning for Algorithmic Trading	9781789346411
☐	Advanced Deep Learning with Keras	9781788629416

6 books

FILTER

By publisher

- All
- Packt Publishing**
- Harper Collins
- Penguin Classics
- Scribner
- Bay Back Books
- Simon and Schuster
- Houghton Mifflin Harcourt
- HardPress

그림 4.52 - 출판사 필터가 있는 Books 변경 목록 페이지

With that line of code, we have implemented a useful publisher filter on the **Books** change list page.

In order to consolidate our knowledge of filters, we will add further filters to the **Books** change list page.

Exercise 4.04 – Adding the date list_filter and date_hierarchy filters

We have seen that the `admin.ModelAdmin` class provides useful attributes to customize filters on change list pages. For example, filtering by date is a crucial functionality for many applications and can also help us make our app more user-friendly. In this exercise, we will examine how date filtering can be implemented by including a date field in the filter and look at the `date_hierarchy` filter:

1. Edit the `reviews/admin.py` file and modify the `list_filter` attribute in the `BookAdmin` class to include `'publication_date'`:

```
class BookAdmin(admin.ModelAdmin):
    list_display = ("title", "isbn13")
    list_filter = ("publisher", "publication_date")
```

2. Reload the **Books** change list page and confirm that the filter now includes date settings:

이 코드 한 줄로 우리는 도서 변경 목록 페이지에 유용한 게시자 필터를 구현했습니다.

필터에 대한 우리의 지식을 통합하기 위해, 우리는 도서 변경 목록 페이지에 추가 필터를 추가할 것입니다.

연습 4.04 – 날짜 `list_filter` 및 `date_hierarchy` 필터 추가

우리는 `admin.ModelAdmin` 클래스가 변경 목록 페이지에서 필터를 사용자 정의할 수 있는 유용한 속성을 제공한다는 것을 보았습니다. 예를 들어, 날짜로 필터링하는 것은 많은 애플리케이션에 중요한 기능이며, 우리의 앱을 더 사용자 친화적으로 만드는 데도 도움이 될 수 있습니다. 이 연습에서는 날짜 필터링을 필터에 날짜 필드를 포함시켜 구현하는 방법을 살펴보고 `date_hierarchy` 필터를 살펴보겠습니다:

1. `reviews/admin.py` 파일을 편집하고 `BookAdmin` 클래스의 `list_filter` 속성을 `'publication_date'`를 포함하도록 수정합니다:

```
class BookAdmin(admin.ModelAdmin):
    list_display = ("title", "isbn13")
    list_filter = ("publisher", "publication_date")
```

2. 책 변경 목록 페이지를 새로 고치고 필터에 날짜 설정이 포함되었는지 확인하십시오:

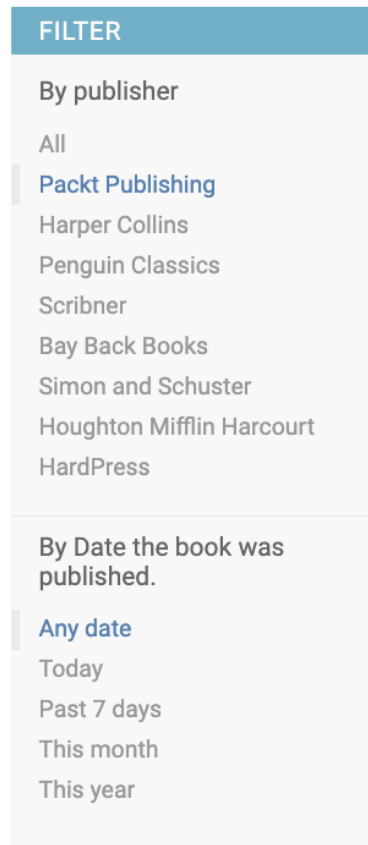


Figure 4.53 – Confirming that the Books change list page includes date settings

This publication date filter would be convenient if the Bookr project were receiving a lot of new releases, and we wanted to filter books by what was published in the last seven days or a month. Sometimes, though, we might like to filter by a specific year or a specific month in a specific year. Fortunately, the `admin.ModelAdmin` class comes with a custom filter attribute geared toward navigating hierarchies of temporal information. It is called `date_hierarchy`.

3. Add a `date_hierarchy` attribute to `BookAdmin` and set its value to `publication_date`:

```
class BookAdmin(admin.ModelAdmin):  
    date_hierarchy = "publication_date"  
    list_display = ("title", "isbn13")  
    list_filter = ("publisher", "publication_date")
```

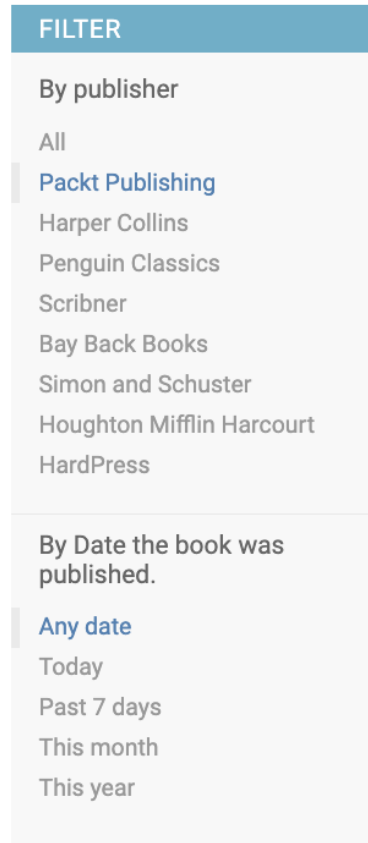


그림 4.53 - 도서 변경 목록 페이지에 날짜 설정이 포함되어 있는지 확인하기

이 idx_128f6e66 출판 idx_0c1b7a8a 날짜 필터는 Bookr 프로젝트가 많은 신간을 받고 있고, 지난 일주일 또는 한 달 동안 출판된 책으로 필터링하고 싶을 때 유용할 것입니다. 그러나 때때로 특정 연도나 특정 연도의 특정 월로 필터링하고 싶을 수 있습니다. 다행히도 `admin.ModelAdmin` 클래스는 시간 정보의 계층을 탐색하는데 맞춰진 사용자 정의 필터 속성을 제공합니다. 이를 `date_hierarchy`라고 합니다.

3. `BookAdmin`에 `date_hierarchy` 속성을 추가하고 그 값을 `publication_date`로 설정합니다:

```
class BookAdmin(admin.ModelAdmin):
    date_hierarchy = "publication_date"
    list_display = ("title", "isbn13")
    list_filter = ("publisher", "publication_date")
```

4. Reload the **Books** change list page and confirm that the date hierarchy appears above the **Action** drop-down menu:

Select book to change

199620012002200420062012201720182019

Action:

Go0 of 18 selected

☐	TITLE	ISBN-13
☐	The Talisman	978-1-45-169721-6
☐	Paul Clifford	978-1-71-905316-7
☐	Animal Farm: A Fairy Story	978-0-15-100217-7
☐	1984	978-1-32-886933-3
☐	Pride and Prejudice	978-0-14-143951-8
☐	Fahrenheit 451	978-1-45-167331-9
☐	The Catcher in the Rye	978-0-31-676917-4
☐	The Great Gatsby	978-0-74-327356-5
☐	To Kill A Mocking Bird	-
☐	For Whom The Bell Tolls	978-1-47-678777-0
☐	The Grapes of Wrath	978-0-14-303943-3
☐	Brave New World	978-0-06-085052-4

ADD BOOK +

FILTER

Show counts

By publisher

All

Packt Publishing

Harper Collins

Penguin Classics

Scribner

Bay Back Books

Simon and Schuster

Houghton Mifflin Harcourt

CreateSpace Independent Publishing Platform

Pocket Books

By Date the book was published.

Any date

Today

Past 7 days

This month

This year

Figure 4.54 – Confirming that the date hierarchy appears above the Action drop-down menu

5. Select a year from the date hierarchy and confirm that it contains a list of months in that year containing book titles and a total list of books:

Select book to change

< All datesMarch 2006October 2006

Action:

Go0 of 2 selected

☐	TITLE	ISBN-13
☐	The Grapes of Wrath	978-0-14-303943-3
☐	Brave New World	978-0-06-085052-4

2 books

Figure 4.55 – Confirming that the selection of a year from the date hierarchy shows the books published that year

6. Confirm that selecting one of these months further filters down to days in the month:

4. 책 변경 목록 페이지를 새로 고치고 날짜 계층이 작업 드롭다운 메뉴 위에 나타나는지 확인하십시오:

Select book to change

1996 2001 2002 2004 2006 2012 2017 2018 2019

Action: 0 of 18 selected

☐	TITLE	ISBN-13
☐	The Talisman	978-1-45-169721-6
☐	Paul Clifford	978-1-71-905316-7
☐	Animal Farm: A Fairy Story	978-0-15-100217-7
☐	1984	978-1-32-886933-3
☐	Pride and Prejudice	978-0-14-143951-8
☐	Fahrenheit 451	978-1-45-167331-9
☐	The Catcher in the Rye	978-0-31-676917-4
☐	The Great Gatsby	978-0-74-327356-5
☐	To Kill A Mocking Bird	-
☐	For Whom The Bell Tolls	978-1-47-678777-0
☐	The Grapes of Wrath	978-0-14-303943-3
☐	Brave New World	978-0-06-085052-4

FILTER

Show counts

↓ By publisher

All

Packt Publishing

Harper Collins

Penguin Classics

Scribner

Bay Back Books

Simon and Schuster

Houghton Mifflin Harcourt

CreateSpace Independent Publishing Platform

Pocket Books

↓ By Date the book was published.

Any date

Today

Past 7 days

This month

This year

그림 4.54 - 날짜 계층이 작업 드롭다운 메뉴 위에 나타나는지 확인하기

5. idx_d53c6b83 날짜 계층에서 연도를 선택하고 해당 연도의 책 제목과 총 책 목록이 포함된 월 목록이 포함되어 있는지 확인하십시오:

Select book to change

< All dates March 2006 October 2006

Action: 0 of 2 selected

☐	TITLE	ISBN-13
☐	The Grapes of Wrath	978-0-14-303943-3
☐	Brave New World	978-0-06-085052-4

2 books

그림 4.55 - 날짜 계층에서 연도를 선택하면 해당 연도에 출판된 책이 표시되는지 확인하기

6. idx_ac7c815e 이들 중 하나의 월을 선택하면 해당 월의 날짜로 추가 필터링되는지 확인하십시오:

Select book to change

< 2006 **March 28**

Action: 0 of 1 selected

☐	TITLE	ISBN-13
☐	The Grapes of Wrath	978-0-14-303943-3

1 book

Figure 4.56 – Filtering months down to days in the month

The `date_hierarchy` filter is a convenient way of customizing a change list that contains a large set of time-sortable data in order to facilitate faster record selection, as we saw in this exercise.

Let's now look at the implementation of a search bar in our app.

The search bar

This brings us to the remaining piece of functionality that we wanted to implement – the search bar. Like filters, a basic search bar is quite simple to implement. We only need to add the `search_fields` attribute to the `ModelAdmin` class. The obvious character fields in our `Book` class to search on are `title` and `isbn`. At present, the **Books** change list appears with a date hierarchy across the top of the change list. The search bar will appear above this:

Select book to change

< 2006 March 28

Action: 0 of 1 selected

☐	TITLE	ISBN-13
☐	The Grapes of Wrath	978-0-14-303943-3

1 book

그림 4.56 - 월을 일로 필터링하기

date_hierarchy 필터는 대량의 시간 정렬 가능한 데이터를 포함하는 변경 목록을 사용자 정의하여 더 빠른 레코드 선택을 용이하게 하는 편리한 방법입니다. 이 연습에서 보았듯이요.

이제 우리 앱에서 검색 바의 구현을 살펴보겠습니다.

검색 바

이것은 우리가 구현하고자 했던 나머지 기능인 검색 바로 이어집니다. 필터와 마찬가지로 기본 검색 바는 구현하기가 매우 간단합니다. ModelAdmin 클래스에 search_fields 속성만 추가하면 됩니다. 우리 Book 클래스에서 검색할 수 있는 명백한 문자 필드는 title과 isbn입니다. 현재 Books 변경 목록은 변경 목록 상단에 date hierarchy가 표시됩니다. 검색 바는 이 위에 나타날 것입니다:

Select book to change

ADD BOOK +

Q

Search

199620012002200420062012201720182019

Action:

Go0 of 18 selected

☐	TITLE	ISBN-13
☐	The Talisman	978-1-45-169721-6
☐	Paul Clifford	978-1-71-905316-7
☐	Animal Farm: A Fairy Story	978-0-15-100217-7
☐	1984	978-1-32-886933-3
☐	Pride and Prejudice	978-0-14-143951-8
☐	Fahrenheit 451	978-1-45-167331-9
☐	The Catcher in the Rye	978-0-31-676917-4
☐	The Great Gatsby	978-0-74-327356-5
☐	To Kill A Mocking Bird	-
☐	For Whom The Bell Tolls	978-1-47-678777-0
☐	The Grapes of Wrath	978-0-14-303943-3

FILTER

Show counts

By publisher

All

Packt Publishing

Harper Collins

Penguin Classics

Scribner

Bay Back Books

Simon and Schuster

Houghton Mifflin Harcourt

CreateSpace Independent Publishing Platform

Pocket Books

By Date the book was published.

Any date

Today

Past 7 days

This month

This year

Figure 4.57 – The Books change list before the search bar is added

We can start by adding this attribute to BookAdmin in reviews/admin.py and examining the result:

```
search_fields = ("title", "isbn")
```

The result would look like this:

Select book to change

ADD BOOK +

Q Search

1996 2001 2002 2004 2006 2012 2017 2018 2019

Action: Go 0 of 18 selected

☐	TITLE	ISBN-13
☐	The Talisman	978-1-45-169721-6
☐	Paul Clifford	978-1-71-905316-7
☐	Animal Farm: A Fairy Story	978-0-15-100217-7
☐	1984	978-1-32-886933-3
☐	Pride and Prejudice	978-0-14-143951-8
☐	Fahrenheit 451	978-1-45-167331-9
☐	The Catcher in the Rye	978-0-31-676917-4
☐	The Great Gatsby	978-0-74-327356-5
☐	To Kill A Mocking Bird	-
☐	For Whom The Bell Tolls	978-1-47-678777-0
☐	The Grapes of Wrath	978-0-14-303943-3

FILTER

Show counts

↓ By publisher

All

- Packt Publishing
- Harper Collins
- Penguin Classics
- Scribner
- Bay Back Books
- Simon and Schuster
- Houghton Mifflin Harcourt
- CreateSpace Independent Publishing Platform
- Pocket Books

↓ By Date the book was published.

Any date

- Today
- Past 7 days
- This month
- This year

그림 4.57 - 검색 바가 추가되기 전의 도서 변경 목록

이 속성을 reviews/admin.py의 BookAdmin에 추가하고 결과를 살펴보겠습니다:

```
search_fields = ("title", "isbn")
```

결과는 다음과 같이 보일 것입니다:

Select book to change ADD BOOK +

1996 2001 2002 2004 2006 2012 2017 2018 2019

Action: -----

0 of 18 selected

☐	TITLE	ISBN-13
☐	The Talisman	978-1-45-169721-6
☐	Paul Clifford	978-1-71-905316-7
☐	Animal Farm: A Fairy Story	978-0-15-100217-7
☐	1984	978-1-32-886933-3
☐	Pride and Prejudice	978-0-14-143951-8
☐	Fahrenheit 451	978-1-45-167331-9
☐	The Catcher in the Rye	978-0-31-676917-4
☐	The Great Gatsby	978-0-74-327356-5
☐	To Kill A Mocking Bird	978-0-06-093546-7
☐	For Whom The Bell Tolls	978-1-47-678777-0

FILTER

[Show counts](#)

By publisher

All

- Packt Publishing
- Harper Collins
- Penguin Classics
- Scribner
- Bay Back Books
- Simon and Schuster
- Houghton Mifflin Harcourt
- CreateSpace Independent Publishing Platform
- Pocket Books

By Date the book was published.

Any date

- Today
- Past 7 days
- This month
- This year

Figure 4.58 – The Books change list with the search bar

Now, we can perform a simple text search on fields that match the title field or ISBN. This search requires precise string matches, so "color" won't match "colour." It also lacks the deep semantic processing that we expect from more sophisticated search facilities such as **Elasticsearch**. ISBN lookup is a very good feature if you happen to have a barcode scanner. Limiting our search to fields on the Books model is quite restrictive. We might want to search by publisher name, too. Fortunately, `search_fields` is flexible enough to accomplish this. To search on `ForeignKeyField` or `ManyToManyField`, we just need to specify the field name on the current model and the field on the related model, separated by two underscores. In this case, Book has a foreign key, publisher, and we want to search on the `Publisher.name` field so it can be specified as `'publisher__name'` on `BookAdmin.search_fields`:

```
search_fields = ("title", "isbn", "publisher__name")
```

If we wanted to restrict a search field to an exact match rather than return results that contain the search string, then the field can be suffixed with `'__exact'`. So, replacing `'isbn'` with `'isbn__exact'` will require the complete ISBN to be matched, and we won't be able to get a match using a portion of the ISBN.

Select book to change

ADD BOOK +

Q
Search

1996 2001 2002 2004 2006 2012 2017 2018 2019

Action: Go 0 of 18 selected

☐	TITLE	ISBN-13
☐	The Talisman	978-1-45-169721-6
☐	Paul Clifford	978-1-71-905316-7
☐	Animal Farm: A Fairy Story	978-0-15-100217-7
☐	1984	978-1-32-886933-3
☐	Pride and Prejudice	978-0-14-143951-8
☐	Fahrenheit 451	978-1-45-167331-9
☐	The Catcher in the Rye	978-0-31-676917-4
☐	The Great Gatsby	978-0-74-327356-5
☐	To Kill A Mocking Bird	978-0-06-093546-7
☐	For Whom The Bell Tolls	978-1-47-678777-0

FILTER

Show counts

By publisher

All
Packt Publishing
Harper Collins
Penguin Classics
Scribner
Bay Back Books
Simon and Schuster
Houghton Mifflin Harcourt
CreateSpace Independent Publishing Platform
Pocket Books

By Date the book was published.

Any date
Today
Past 7 days
This month
This year

그림 4.58 - 검색 바가 있는 도서 변경 목록

이제 제목 필드나 ISBN과 일치하는 필드에서 `idx_4831d58esimple` 텍스트 검색을 수행할 수 있습니다. 이 검색은 정확한 문자열 일치를 요구하므로 "color"는 "colour"와 일치하지 않습니다. 또한, Elasticsearch와 같은 더 정교한 검색 기능에서 기대하는 깊은 의미 처리 기능이 부족합니다. ISBN 조회는 바코드 스캐너가 있는 경우 매우 유용한 기능입니다. 도서 모델의 필드로 검색을 제한하는 것은 상당히 제한적입니다. 우리는 출판사 이름으로도 검색하고 싶을 수 있습니다. 다행히도 `search_fields`는 이를 수행할 만큼 유연합니다. `ForeignKeyField` 또는 `ManyToManyField`에서 검색하려면 현재 모델의 필드 이름과 관련 모델의 필드를 두 개의 언더스코어로 구분하여 지정하기만 하면 됩니다. 이 경우, `Book`은 외래 키인 `publisher`를 가지고 있으며, `Publisher.name` 필드에서 검색하고 싶으므로 `BookAdmin.search_fields`에서 `'publisher__name'`으로 지정할 수 있습니다:

```
search_fields = ("title", "isbn", "publisher__name")
```

검색 필드를 검색 문자열을 포함하는 결과를 반환하는 대신 정확한 일치로 제한하려면 필드에 `'__exact'`를 접미사로 추가할 수 있습니다. 따라서 `'isbn'`을 `'isbn__exact'`로 바꾸면 전체 ISBN이 일치해야 하며, ISBN의 일부를 사용하여 일치를 얻을 수 없습니다.

Similarly, we constrain the search field to only return results that start with the search string by using the `'__startswith'` suffix. Qualifying the publisher name search field as `'publisher__name__startswith'` means that we will get results searching for "pack" but not for "ackt."

This concludes our examination of common customizations of the change list pages. We may also want to customize the behavior of the create and update forms in our app.

Excluding and grouping fields

There are occasions when it is appropriate to restrict the visibility of some of the fields in the model in the admin interface. This can be achieved with the `exclude` attribute.

This is the review form screen with the **Date edited** field visible. Note that the **Date created** field does not appear – it is already a hidden view because `date_created` is defined on the model with the `auto_now_add` parameter:

Add review

Content:

An excellent introduction to the topic.

The Review text.

Rating:

5

The rating the reviewer has given.

Date edited:

Date:2024-09-13

Today | 📅

Time:12:24:27

Now | 🕒

Note: You are 10 hours ahead of server time.
The date and time the review was last edited.

Creator:

bob

✎ + 👁

Book:

Hands-On Machine Learning for Algorithmic Trading (9781789346411)

✎ + 👁

The Book that this review is for.

SAVE

Save and add another

Save and continue editing

Figure 4.59 – The review form

If we wanted to exclude the **Date edited** field from the review form, we would do this in the `ReviewAdmin` class:

```
exclude = ("date_edited",)
```

유사하게, 우리는 검색 문자열로 시작하는 결과만 반환하도록 검색 필드를 제한합니다. 'publisher__name__startswith'로 게시자 이름 검색 필드를 한정하면 "pack"을 검색할 수 있지만 "ackt."는 검색할 수 없습니다.

이것으로 변경 목록 페이지의 일반적인 사용자 정의에 대한 idx_91f0ace1 검토를 마칩니다. 우리는 또한 앱에서 생성 및 업데이트 양식의 동작을 사용자 정의하고 싶을 수 있습니다.

필드 제외 및 그룹화

관리 인터페이스에서 모델의 일부 필드의 가시성을 제한하는 것이 적절한 경우가 있습니다. 이는 exclude 속성을 사용하여 달성할 수 있습니다.

이것은 날짜 수정 필드가 보이는 검토 양식 화면입니다. 날짜 생성 필드는 나타나지 않는데, 이는 date_created가 모델에서 auto_now_add 매개변수로 정의되어 이미 숨겨진 보기이기 때문입니다:

Add review

Content:
The Review text.

Rating:
The rating the reviewer has given.

Date edited: Today | 📅
 Now | ⌚
Note: You are 10 hours ahead of server time.
 The date and time the review was last edited.

Creator: ✎ + 👁

Book: ✎ + 👁
The Book that this review is for.

그림 4.59 - 검토 양식

검토 양식에서 날짜 수정 필드를 idx_2bb3c862 제외하고 싶다면, 우리는 ReviewAdmin 클래스에서 이렇게 할 것입니다:

```
exclude = ("date_edited",)
```


Then, the review form would appear without **Date edited**:

Add review

Content:
The Review text.

Rating:
The rating the reviewer has given.

Creator:   

Book:   
The Book that this review is for.

Figure 4.60 – The review form with the Date edited field excluded

Conversely, it might be more prudent to restrict the admin fields to those that have been explicitly permitted. This is achieved with the `fields` attribute. The advantage of this approach is that if new fields are added in the model, they won't be available in the admin form unless they have been added to the `fields` tuple in the `ModelAdmin` subclass:

```
fields = ("content", "rating", "creator", "book")
```

This will give us the same result that we saw earlier.

Another option is to use the `fieldsets` attribute of the `ModelAdmin` subclass to specify the form layout as a series of grouped fields. Each grouping in `fieldsets` consists of a title followed by a dictionary containing a `"fields"` key pointing to a list of field name strings:

```
fieldsets = (
    ("Linkage", {"fields": ("creator", "book")}),
    ("Review content", {"fields": ("content", "rating")}),
)
```

그런 다음, 검토 양식이 수정된 날짜 없이 나타납니다:

Add review

Content:
The Review text.

Rating:
The rating the reviewer has given.

Creator:   

Book:   
The Book that this review is for.

그림 4.60 - 수정된 날짜 필드가 제외된 검토 양식

반대로, 관리 필드를 명시적으로 허용된 필드로 제한하는 것이 더 신중할 수 있습니다. 이는 `fields` 속성을 사용하여 달성됩니다. 이 접근 방식의 장점은 모델에 새 필드가 추가되더라도 `ModelAdmin` 하위 클래스의 `fields` 튜플에 추가되지 않는 한 관리 양식에서 사용할 수 없다는 것입니다:

```
fields = ("content", "rating", "creator", "book")
```

이것은 우리가 이전에 보았던 것과 동일한 결과를 제공합니다.

또 다른 옵션은 `ModelAdmin` 하위 클래스의 `fieldsets` 속성을 사용하여 양식 레이아웃을 일련의 그룹화된 필드로 지정하는 것입니다. `fieldsets`의 각 그룹화는 제목과 `"fields"` 키가 포함된 사전으로 구성되어 있으며, 이 키는 필드 이름 문자열 목록을 가리킵니다:

```
fieldsets = (
    ("연결", {"fields": ("제작자", "책")}),
    ("리뷰 내용", {"fields": ("내용", "평점")}),
)
```